

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH
CITY**

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

**FACULTY OF COMPUTER SCIENCE AND
ENGINEERING**



REPORT

GRADUATION PROJECT

**DEVELOPMENT OF AN ONLINE
LIBRARY MANAGEMENT SYSTEM**

MAJOR: COMPUTER SCIENCE

COUNCIL: 3CC Computer Science

Advisor 1: Dr. Truong Tuan Anh

Advisor 2: MSc. Vu Ngoc Tu

—o0o—

Student 1: Nguyen Anh Khoa (2211614)

Student 2: Nguyen Tran Bao Khoa (2211610)

Student 3: Mai Nam Phuong (2252656)

HO CHI MINH CITY, OCTOBER 2026

Acknowledgments

First and foremost, our team would like to express our sincere gratitude to Dr. Truong Tuan Anh – lecturer-in-charge – and MSc. Vu Ngoc Tu – who directly guided our team throughout the graduation project during Semester 253.

Thanks to their dedicated guidance and valuable feedback, our team was able to make timely adjustments, address shortcomings, and complete the content of the project. Under their orientation, our team recognized the practical market demand for an online library management system while gaining a clearer understanding of the strengths and limitations of existing systems to propose appropriate enhancement solutions for our graduation project.

We would also like to express our deepest gratitude to our families, who have always encouraged, motivated, and created the best conditions for us to focus on completing this project. This served as an immense source of motivation helping us persevere throughout the process.

In addition, we would like to thank our friends who accompanied, gave feedback, and shared experiences with us during our work. Your encouragement and support helped us overcome challenging moments when feeling confused about how to execute the next steps of the project.

Finally, we sincerely thank everyone for all their valuable help, encouragement, and guidance, enabling us to successfully complete our graduation project titled “Development of an Online Library Management System”.

Declaration of Authorship

We hereby declare that this project is our own research work, conducted under the supervision and guidance of Dr. Truong Tuan Anh and MSc. Vu Ngoc Tu. The research results presented in this project have not been previously published in any form. All materials used in this project have been collected from various reference sources and systems.

These materials are fully cited in the references section. Apart from the aforementioned sources, all other report contents and implementation results were independently created by our team.

We take full responsibility should any violation arise concerning the statements declared herein.

Ho Chi Minh City, October 2026

Nguyen Anh Khoa

Nguyen Tran Bao Khoa

Mai Nam Phuong

Abstract

The project titled “Development of an Online Library Management System” was conducted to fulfill digital transformation demands in library resource management and utilization. In the context of traditional libraries gradually transitioning to digital environments, constructing a modern, efficient management system capable of providing better support for users is an imperative requirement. The project focuses on researching, designing, and deploying a library system as a Web Application targeted at small-to-medium scale libraries, such as school libraries, faculty libraries, or learning resource centers.

The system is constructed to manage diverse types of library resources, including printed books, journals, and digital assets such as E-books and PDF documents. Furthermore, the system goes beyond supporting basic administrative functions by aiming to enhance user experience through the integration of modern technologies, particularly Artificial Intelligence (AI).

Within the system, the primary user roles consist of readers, librarians, and administrators. Each role possesses distinct functional requirements and usage patterns, which are appropriately supported by the system.

For the reader role, the system provides flexible document search and retrieval functions through keywords or intelligent search tools. Additionally, the system incorporates a document recommendation mechanism based on reading history and user behavior, helping readers quickly access documents tailored to their learning and research needs. Readers can manage their borrowing history, track loan-return statuses, and interact with the system via a chatbot for prompt assistance with queries.

For the librarian role, the system supports managing resource catalogs, updating document metadata, and processing borrowing and returning operations. Furthermore, librarians can monitor resource availability, control data, and ensure efficient library operations. Implementing the system minimizes manual procedures, thereby increasing productivity and reducing management errors.

For the administrator role, the system provides management privileges over all system operations, including user account management, system configuration, and data monitoring. In addition, administrators can review statistics related to borrowing and returning activities to support managerial decision-making and rational library resource development.

A key highlight of the system is the integration of Artificial Intelligence capabilities, such as document recommendations, natural language search, and an interactive assistant chatbot. These features significantly enhance user-system interaction and elevate the overall user experience compared to traditional library systems.

Architecturally, the system is designed following a web application model with key components: user interface, server-side business processing logic, and data storage databases. This organization ensures flexibility, scalability, and maintainability for future upgrades.

Throughout the development of the Online Library Management System, this project aims to optimize resource management workflows, enhance operational efficiency, and deliver an improved user experience in searching for and utilizing academic materials. Simultaneously, the project demonstrates the practical applicability of modern technologies, especially AI, in developing information systems for educational and research environments.

Contents

1	General Topic Introduction	1
1.1	Project Introduction	1
1.1.1	Motivation for Topic Selection	1
1.2	Project Objectives	1
1.2.1	Overview of Proposed System	2
1.3	Project Benefits	2
1.3.1	Building a Modern Online Library Management System	2
1.3.2	Effective Support for Library Management	2
1.3.3	Optimizing Reader Resource Exploitation	2
1.3.4	Enhancing Interaction Between Users and the Library System	2
1.4	Project Scope and Limitations	2
1.4.1	User Groups	2
1.4.2	Scope of Artificial Intelligence Application	3
1.4.3	Project Limitations	3
1.5	General Topic Significance	3
1.5.1	Practical Significance	3
1.5.2	Scientific Significance	3
2	Overview Analysis of Relevant Market Systems	4
2.1	Library Resource Management and Search System – Ho Chi Minh City University of Technology (HCMUT)	4
2.1.1	Strengths	5
2.1.2	Weaknesses	5
2.2	Online Library System – University of London (The Online Library – UoL)	5
2.2.1	Strengths	5
2.2.2	Weaknesses	6
2.3	University of Delaware Library System (UD Library)	6
2.3.1	Strengths	6
2.3.2	Weaknesses	6
2.4	Open Source Library Management System – Koha ILS	6
2.4.1	Strengths	7
2.4.2	Weaknesses	7
3	System Requirements	8
3.1	Functional Requirements	8
3.1.1	User Account Management	8
3.1.2	Resource Catalog Management	8
3.1.3	Circulation Management	8
3.1.4	AI Natural Language Search and Intelligent Interaction	9
3.1.5	Notifications and Communication	9
3.1.6	Reporting and Analytics	9
3.1.7	System Administration and Configuration	9
3.2	Non-Functional Requirements	10
3.2.1	Performance and User Experience	10
3.2.2	Security and Access Control	10
3.2.3	Reliability and Maintainability	10
4	Detailed Use-Case Specifications	11
4.1	User Account and Access Management	11
4.1.1	Overview	11
4.1.2	Detailed Specifications	11
4.2	Catalog and Metadata Management	14
4.2.1	Overview	14
4.2.2	Detailed Specifications	14
4.3	Document Circulation Management	17
4.3.1	Overview	17
4.3.2	Detailed Specifications	17
4.4	Resource Access and Exploitation	21
4.4.1	Overview	21

4.4.2	Detailed Specifications	21
4.5	Intelligent Interaction	22
4.5.1	Overview	22
4.5.2	Detailed Specifications	22
4.6	Notifications and Communication	25
4.6.1	Overview	25
4.6.2	Detailed Specifications	25
4.7	Reporting and System Administration	28
4.7.1	Overview	28
4.7.2	Detailed Specifications	28
5	System Analysis and Design	31
5.1	System Architecture	31
5.1.1	Presentation Tier	31
5.1.2	Application Tier	31
5.1.3	Data Tier	32
5.2	Database Design	33
5.2.1	Entity Descriptions	33
5.2.2	Enhanced Entity-Relationship Diagram (EERD)	33
5.2.3	Database Schema	35
5.3	Class Diagram	36
5.3.1	Account and Access Management	36
5.3.2	Book Catalog Management	37
5.3.3	Document Circulation Management	38
5.3.4	Resource Search and Exploitation	39
5.3.5	AI Virtual Assistant and Recommendations	39
5.3.6	Notifications and Communication	40
5.3.7	System Administration	40
5.4	Activity Diagrams	41
5.4.1	Account Registration	41
5.4.2	System Login	43
5.4.3	Password Reset	45
5.4.4	View Loan/Return History	45
5.4.5	User Account Management	47
5.4.6	Add / Edit / Delete Catalog Record	48
5.4.7	Batch Catalog Data Ingestion	49
5.4.8	Automated Cataloging via ISBN	50
5.4.9	Physical Material Checkout	51
5.4.10	Material Return	52
5.4.11	Calculate and Assess Overdue Fines	53
5.4.12	Online Loan Renewal	54
5.4.13	Place Item Hold	54
5.4.14	Document Catalog Search	55
5.5	Sequence Diagrams	56
5.5.1	Account Registration	56
5.5.2	Account Login	57
5.5.3	Account Password Reset	58
5.5.4	Document Search	59
5.5.5	Add / Edit / Delete Document	60
5.5.5.a	Add Document	60
5.5.5.b	Edit Document	60
5.5.5.c	Delete Document	61
5.5.6	Batch Document Import	61
5.5.7	Reserve Document	62
5.5.8	Fine Calculation Update	63
5.5.9	Process Document Loan	64
5.5.10	Extend Document Loan	64
5.5.11	Return Document	65

6	Implementation	67
6.1	Technology Stack and Architectural Infrastructure	67
6.1.1	Front-end Presentation Layer: React.js and Vite	67
6.1.2	Back-end Core Server: Node.js and Express	67
6.1.3	Dedicated AI Microservice: Python and FastAPI	67
6.1.4	Data Modeling and ORM: Prisma	68
6.1.5	Relational Database: PostgreSQL	68
6.1.6	Vector Store and Embedding Infrastructure: ChromaDB and pgvector	68
6.1.7	Adaptive Hybrid Search Strategy	69
6.1.8	Security, Authentication, and Role-Based Access Control	69
6.1.9	External Integrations: Open Library, Google Books, and Nodemailer	69
6.1.10	Background Task Processing: node-cron	70
6.2	Authentication and Access Governance Module	70
6.2.1	User Interface Implementation	70
6.2.2	Technical Implementation Highlights	71
6.3	Reader Module	72
6.3.1	User Interface Implementation	72
6.3.2	Technical Implementation Highlights	76
6.4	Librarian Module	77
6.4.1	User Interface Implementation	77
6.4.2	Technical Implementation Highlights	80
6.5	Administrator Module	82
6.5.1	User Interface Implementation	82
6.5.2	Technical Implementation Highlights	85
6.6	Deployment Stage	86
6.6.1	Frontend Deployment	87
6.6.2	Backend Deployment	87
6.6.3	AI Service Deployment	87
6.6.4	System Integration	87
7	System Testing	88
7.1	Test Objectives and Environment	88
7.2	Unit Testing	88
7.3	Integration Testing	89
7.4	System Testing	90
7.5	Code Quality Testing	91
7.6	Test Summary	92
8	AI Features Evaluation	93
8.1	Why Traditional Testing is Insufficient for AI	93
8.2	AI-as-a-Judge Methodology	93
8.2.1	Chatbot Evaluation Metrics (RAG Framework)	93
8.2.2	Semantic Search Evaluation Metrics	93
8.3	Chatbot RAG Evaluation Results	94
8.3.1	Test Dataset Setup	94
8.3.2	Detailed Chatbot Evaluation Results Table	94
8.3.3	Chatbot Evaluation Analysis	94
8.4	Semantic Search Evaluation Results	94
8.4.1	Test Dataset Setup	94
8.4.2	Overall Performance Metrics	95
8.4.3	Detailed Search Evaluation Results Table	95
8.4.4	Search Performance Analysis	95
9	Summary and Future Work	96
9.1	Achieved Results	96
9.2	Remarks and Evaluation	96
9.3	Future Work	96
10	References	97
11	Appendix	98

List of Figures

1	Main interface of the lib.hcmut.edu.vn library portal and print material catalog	4
2	Institutional Repository Search Interface and Account Login Module on the OPAC portal	4
3	Unified discovery search interface and live support tool at The Online Library - UoL	5
4	DELCAT unified discovery service interface and supporting AI tool at UD Library	6
5	Koha Staff Client administrative interface module	7
6	User Account and Access Management Use-Case Diagram	11
7	Catalog & AI Metadata Management Use-Case Diagram	14
8	Document Circulation Management Use-Case Diagram	17
9	Resource Access & Exploitation Use-Case Diagram	21
10	Intelligent Interaction & AI Use-Case Diagram	22
11	Notifications & Communication Use-Case Diagram	25
12	Reporting & Administration Use-Case Diagram	28
13	Overall System Architecture Diagram of the Online Library Management System	31
14	EERD Diagram of the Online Library Management System	34
15	Relational Database Schema for the Online Library Management System	35
16	Class Diagram for Account and Access Management	36
17	Class Diagram for Book Catalog Management	37
18	Class Diagram for Document Circulation Management	38
19	Class Diagram for Resource Search and Exploitation	39
20	Class Diagram for AI Virtual Assistant and Recommendations	39
21	Class Diagram for Notifications and Communication	40
22	Class Diagram for System Administration	40
23	Activity Diagram for Account Registration	42
24	Activity Diagram for System Login	44
25	Activity Diagram for Password Reset	45
26	Activity Diagram for View Loan/Return History	46
27	Activity Diagram for User Account Management	47
28	Activity Diagram for Add / Edit / Delete Catalog Record	48
29	Activity Diagram for Batch Catalog Data Ingestion	49
30	Activity Diagram for Automated Cataloging via ISBN	50
31	Activity Diagram for Physical Material Checkout	51
32	Activity Diagram for Material Return	52
33	Activity Diagram for Calculate and Assess Overdue Fines	53
34	Activity Diagram for Online Loan Renewal	54
35	Activity Diagram for Place Item Hold	55
36	Activity Diagram for Document Catalog Search	56
37	Sequence Diagram for Account Registration	57
38	Sequence Diagram for Account Login	58
39	Sequence Diagram for Account Password Reset	59
40	Sequence Diagram for Document Search	59
41	Sequence Diagram for Add Document	60
42	Sequence Diagram for Edit Document	61
43	Sequence Diagram for Delete Document	61
44	Sequence Diagram for Batch Document Import	62
45	Sequence Diagram for Reserve Document	63
46	Sequence Diagram for Fine Calculation Update	63
47	Sequence Diagram for Process Document Loan	64
48	Sequence Diagram for Extend Document Loan	65
49	Sequence Diagram for Return Document	66
50	Front-End Technology Foundation: React.js and Vite Build Engine	67
51	Back-End Application Gateway: Node.js and Express Framework	67
52	Dedicated AI Microservice Engine: Python and FastAPI	68
53	Object-Relational Mapping Engine: Prisma (TypeScript)	68
54	Relational Database Management System: PostgreSQL	68
55	Vector Embedding and Retrieval Pipeline Architecture	69
56	External Bibliographic Integration via Google Books and Open Library	69
57	BkLib Login Interface with Dual-Role Selector Tabs	70
58	BkLib Account Registration Interface	71

59	Three-Stage Modal Workflow for Self-Service Password Recovery	71
60	Reader Module Central Dashboard	73
61	Reader Catalog Search and Filtering Interface	73
62	Active Loans Management and Online Renewal Interface	74
63	Borrowing History and Fine Audit Ledger	74
64	Active Hold Reservations and Queue Status Interface	75
65	Virtual Assistant Chat Window Interface	75
66	Personal Profile and Notification Settings Interface	76
67	Librarian Operational Dashboard and Quick Actions	77
68	High-Throughput Circulation Desk Interface	78
69	Catalog and Physical Inventory Management Interface	78
70	Multi-Channel Communications Dispatcher Interface	79
71	Librarian Reports and Analytics Dashboard	79
72	Librarian Staff Profile and Operating Scope Interface	80
73	Administrator System Overview Dashboard	83
74	User Account and Role Governance Interface	83
75	System Dynamic Configuration Management Console	84
76	Campus Branch Network Management Interface	84
77	Security Audit Trail and Mutation Log Viewer	85
78	Database Backup and Disaster Recovery Interface	85
79	Unit test results	89
80	Integration test results	90
81	System test results	91
82	Quality test results	92
83	Summary of test results	92

List of Tables

1	Use-case Specification: Account Registration	11
2	Use-case Specification: Password Reset	12
3	Use-case Specification: System Login	12
4	Use-case Specification: Update User Profile	13
5	Use-case Specification: View Loan/Return History	13
6	Use-case Specification: User Account Management	14
7	Use-case Specification: Add / Edit / Delete Catalog Record	15
8	Use-case Specification: Batch Catalog Data Ingestion	16
9	Use-case Specification: Automated Cataloging via ISBN	16
10	Use-case Specification: Automated Document Summarization	17
11	Use-case Specification: Physical Material Checkout	18
12	Use-case Specification: Material Return	19
13	Use-case Specification: Calculate and Assess Overdue Fines	19
14	Use-case Specification: Online Loan Renewal	20
15	Use-case Specification: Place Item Hold	20
16	Use-case Specification: Inter-Branch Loan Request	21
17	Use-case Specification: Document Catalog Search	22
18	Use-case Specification: Rate and Review Materials	22
19	Use-case Specification: Natural Language Search	23
20	Use-case Specification: Chatbot Virtual Assistant Interaction	24
21	Use-case Specification: View Personalized Recommendations	25
22	Use-case Specification: Configure Notification Preferences	26
23	Use-case Specification: View System Notifications	27
24	Use-case Specification: Broadcast Mass Notifications	27
25	Use-case Specification: View Operational Dashboard	28
26	Use-case Specification: Export Periodic Statistical Reports	29
27	Use-case Specification: Configure System Parameters	29
28	Use-case Specification: Manage Branch Libraries	29
29	Use-case Specification: View Audit Logs	30
30	System Entities and Database Definitions	33
31	Comprehensive Technology Stack Summary of the BkLib Platform	70
32	Scope and Results of Unit Testing	88
33	Scope and Results of Integration Testing	89
34	System Testing Workflows	90
35	Source Code Quality Test Results	91
36	Backend Test Summary	92
37	Deterministic Testing vs. AI Feature Evaluation	93
38	Detailed RAG Chatbot Evaluation Results (Scale 1–5)	94
39	Semantic Search Evaluation Metrics Summary (pgvector)	95
40	Detailed Semantic Search Evaluation Results (pgvector, 15 queries)	95

1 General Topic Introduction

1.1 Project Introduction

In the context of rapid digital transformation, traditional libraries are gradually transforming into modern online learning resource centers. However, the greatest challenge today extends beyond mere digitization of book titles to managing and exploiting those resources effectively. Existing library management systems often focus primarily on basic administrative tasks, rendering interactions between readers and collections passive and fragmented.

For library managers, manually operating vast resource catalogs—ranging from physical books and journals to digital assets—frequently leads to delays in updating loan statuses and preservation records. The lack of automated analytics tools places heavy pressure on member management and circulation tracking, particularly when forecasting reading demand to optimize collection acquisition.

For readers, searching for materials on legacy systems is often difficult due to rigid keyword matching, lacking flexibility and personalization. Modern users desire a smarter interactive space where they can receive book recommendations tailored to their interests, reading history, and personal behavior without spending excessive time on manual searches. Simultaneously, the demand for immediate assistance via virtual assistants to resolve inquiries regarding rules or document shelf locations is becoming essential.

Recognizing these pressing requirements, the project "Development of an Online Library Management System" was conceived to research, design, and deploy a comprehensive technological platform. This platform not only thoroughly resolves resource management and operational challenges—such as book catalogs, journals, digital assets, and circulation workflows—but also integrates Artificial Intelligence (AI) to enhance collection exploitation. The core focus of the project is creating a smart interactive ecosystem based on a combination of modern administrative models that automate operations and centralize resource management, personalized user experiences through intelligent search tools and automated recommendation systems based on actual reading behavior, and automated support from Chatbots and virtual assistants to deliver immediate query responses and operational guidance to users anytime, anywhere.

By applying software technologies and AI algorithms, the system focuses on optimizing administrative workflows for librarians while providing readers with rapid search capabilities and access to materials tailored to individual needs.

1.1.1 Motivation for Topic Selection

During our academic studies and library usage in a university environment, our team observed multiple limitations in existing library systems. Document searches rely heavily on simple keywords, yielding results that often fail to align with user needs. Furthermore, systems lack the capability to recommend materials based on user preferences or borrowing history, requiring readers to spend significant time locating suitable resources.

Additionally, management activities such as tracking loans/returns, updating item statuses, or providing user support remain heavily dependent on manual operations. This not only reduces operational efficiency but also negatively impacts user experience.

Recognizing these realities, our team chose the topic "Development of an Online Library Management System" with the objective of building a modern system capable of automation and intelligent support, contributing to enhanced management efficiency and improved user satisfaction.

1.2 Project Objectives

The primary objective of this project is to build an online library management system as a Web Application to effectively support both library administrators and readers in exploiting and utilizing academic resources.

The system is developed to provide a centralized management platform for library operations, including catalog management, loan and return lifecycle tracking, and membership administration. Concurrently, the system aims to enhance user experience through intelligent search tools and recommendation mechanisms based on user preferences and reading history.

Furthermore, the project aims to integrate artificial intelligence technologies to assist users during system interactions. Features such as chatbots or virtual assistants are researched and implemented to deliver instant answers to frequently asked questions, system usage guidance, and rapid document retrieval support.

Through constructing this system, the project strives to create a smart digital library platform that optimizes administrative workflows for library managers while delivering convenient, effective resource discovery and access experiences for readers.

1.2.1 Overview of Proposed System

The online library management system is built as a web application, allowing users to access and interact via standard web browsers without requiring additional software installations. Overall, the system comprises core components including the user interface, server-side business logic processing engine, and database storing resource metadata and user profiles.

In addition to basic components, the system integrates artificial intelligence modules to support advanced capabilities such as document recommendations, intelligent search, and chatbot user assistance. These modules operate on data gathered from user behavior and system resource metadata to produce relevant, personalized results.

1.3 Project Benefits

The project delivers tangible benefits to library resource management and exploitation.

1.3.1 Building a Modern Online Library Management System

The project focuses on developing a web-based library management system enabling resource administration and access through an online environment. The system integrates data processing technologies and artificial intelligence to enhance administrative control and support users during document search and exploitation.

1.3.2 Effective Support for Library Management

The system enables management of diverse resource catalogs, including physical books, journals, and digital assets. Features tracking loan-return-preservation statuses, member accounts, and usage analytics empower library staff to operate efficiently. Statistical insights from the system also inform decisions regarding collection acquisition and adjustments.

1.3.3 Optimizing Reader Resource Exploitation

Intelligent search capabilities, automated recommendation engines, and virtual assistant features allow users to quickly locate materials matching their learning and research needs. The system also personalizes user experiences based on reading history and search patterns, boosting resource exploitation efficiency.

1.3.4 Enhancing Interaction Between Users and the Library System

Integrating chatbots or virtual assistants ensures users receive prompt support when interacting with the system. Inquiries regarding borrowing policies, document location, or system operation are answered immediately, enhancing overall user experience during library interactions.

1.4 Project Scope and Limitations

The scope of the project focuses on researching and building an online library management system as a Web Application. Direct system actors comprise: Administrators (Admin), Librarians (Librarian), and Readers (Reader).

1.4.1 User Groups

Reader: Primary system users who search, query, and borrow materials. Readers also receive personalized book recommendations, manage loan histories, and interact with the system via search features or chatbots.

Librarian: Responsible for managing catalog entries, processing loan and return transactions, and updating resource statuses. Librarians also utilize administrative tools for data entry and system information control.

Administrator (Admin): Manages the entire system, including parameter configuration, user account management, and overall system monitoring.

The system manages common library resources such as physical books, printed journals, and digital assets in electronic file formats like E-books or PDF documents. Core functionalities encompass catalog management, member account management, loan-return status tracking, document search support, and personalized recommendation generation.

Within the project scope, the system is designed for small to medium-scale libraries, such as departmental libraries, school libraries, or learning resource centers. Target end-users include students, faculty members, and researchers requiring access to academic materials.

AI features are implemented at a foundational level, detailed in the artificial intelligence application scope section below.

1.4.2 Scope of Artificial Intelligence Application

Within the project scope, artificial intelligence features are deployed at a foundational level to demonstrate integration potential within a library management system. Specifically, the system utilizes existing models and services to implement functionalities such as behavioral recommendation algorithms, natural language search, and Q&A chatbots for common inquiries.

Due to time and resource constraints, the project does not focus on building or training AI models from scratch, but primarily leverages existing tools and APIs to deploy and integrate them into the system. This ensures project feasibility while demonstrating a modern technological approach.

1.4.3 Project Limitations

Although the system is designed with comprehensive features, certain limitations exist. The system is currently developed exclusively as a web application and does not support native mobile platforms. AI capabilities are implemented at a foundational level and have not reached the optimization of large commercial platforms.

Additionally, the system does not handle extreme large-scale scenarios involving millions of records or deep integration with external third-party library networks. Advanced capabilities will be considered for future research and subsequent software releases.

1.5 General Topic Significance

1.5.1 Practical Significance

The online library management system developed in this project enhances operational efficiency for libraries in a digital environment. Centralizing resource and user management reduces manual operations while ensuring data accuracy and real-time synchronization.

Furthermore, the system significantly improves reader experiences through intelligent search and recommendation tools. Users can rapidly discover resources tailored to their academic needs without navigating cumbersome search procedures.

Additionally, integrating chatbots or virtual assistants delivers instant user support, reducing the workload for library staff in addressing routine inquiries.

1.5.2 Scientific Significance

From a research standpoint, the project clarifies methodologies for building and deploying a library management system using modern software technologies. System design encompasses software architecture engineering, database design, and resource/user management implementation.

The project also explores integrating artificial intelligence models into information systems to personalize user experiences. Applying recommendation algorithms and user-assistive chatbots demonstrates AI's potential in elevating knowledge resource utilization.

By combining web development technologies, data management, and artificial intelligence, the project proposes a smart library management system model that can serve as a benchmark for future research and practical implementations.

In subsequent chapters, the report focuses on system requirement analysis, overall architecture design, and core feature implementation to build a complete library management system.

2 Overview Analysis of Relevant Market Systems

2.1 Library Resource Management and Search System – Ho Chi Minh City University of Technology (HCMUT)

The library resource management and search system at Ho Chi Minh City University of Technology (VNU-HCM) currently operates through two independent yet complementary component portals: the Official Library Information Portal (<https://lib.hcmut.edu.vn/>) and the Online Public Access Catalog system (OPAC at <https://opac.lib.hcmut.edu.vn/>). This combination aims to manage and serve two core resource streams in parallel: traditional printed materials and institutional repository assets.

For the main library information portal (lib.hcmut.edu.vn), the system serves as a central hub managing and presenting all print materials, including textbooks, reference books, and academic journals across various disciplines physically stored in campus stacks (such as the Loan Collection in Building B2, Reference Collection in Building B4 at Ly Thuong Kiet Campus, and the Library at Di An Campus). Users on this interface primarily access general announcements, loan rules, or utilize the quick search bar to verify the physical availability of print volumes.

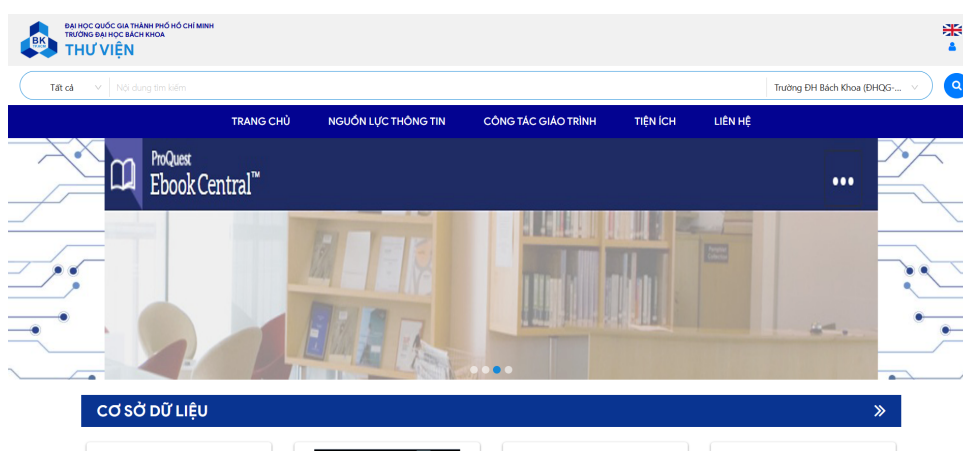


Figure 1: Main interface of the lib.hcmut.edu.vn library portal and print material catalog

Concurrently, to support in-depth academic research, the university operates the OPAC portal (opac.lib.hcmut.edu.vn), serving as a specialized search gateway for institutional repository assets. This portal archives and digitizes intellectual assets produced within the university, including student graduation theses, master's theses, doctoral dissertations, and scientific research projects conducted by faculty members. The OPAC interface offers advanced metadata filtering by Author, Advisor, Defense Year, or Academic Field.

The OPAC system is deeply integrated with the university's Single Sign-On authentication system (SSO/BKPay). Upon successfully logging into the "Personal Account" module, readers can manage loan activities such as viewing borrowed items, checking loan histories, tracking overdue fines, requesting loan renewals, or placing holds on items currently checked out.



Figure 2: Institutional Repository Search Interface and Account Login Module on the OPAC portal

Although both portals satisfy core operational needs, data fragmentation between print materials and digital/institutional repositories creates friction in student user experiences. This limitation motivates our team to

construct a modern library management system that unifies resource data and optimizes operational workflows through modern software solutions.

2.1.1 Strengths

- Specialized interface for institutional repositories optimized for querying academic metadata fields.
- Online loan-return and hold placement workflows fully compatible with practical library policies.
- Single Sign-On (SSO) authentication mechanism synchronizing reader accounts while securing digital resources.

2.1.2 Weaknesses

- Maintaining two independent portals obligates readers to execute repetitive searches across separate domain names.
- Search mechanisms rely strictly on exact keyword matching, showing limited natural language understanding.
- Static platform architecture lacking automated recommendation features and 24/7 AI virtual assistant tools.
- Advanced search interfaces are poorly optimized for mobile devices, exhibiting procedural interaction flows.

2.2 Online Library System – University of London (The Online Library – UoL)

The University of London (UoL) Online Library system at <https://onlinelibrary.london.ac.uk/> represents a flagship digital library (E-library) model designed specifically to support global distance-learning students. Unlike traditional physical collection management models, this system operates almost entirely digitally, focusing on managing, storing, and distributing access rights to academic databases, electronic journals (E-journals), and electronic books (E-books).

A key technical highlight of the system is the deployment of a centralized Discovery Service. Rather than requiring users to log into publisher platforms individually, a single unified search bar (One-stop search) retrieves full-text items across hundreds of database sources. Furthermore, to compensate for the absence of in-person assistance, the system features a live support tool ("Chat with us") functioning as a 24/7 help desk to resolve technical inquiries and guide student literature searches.

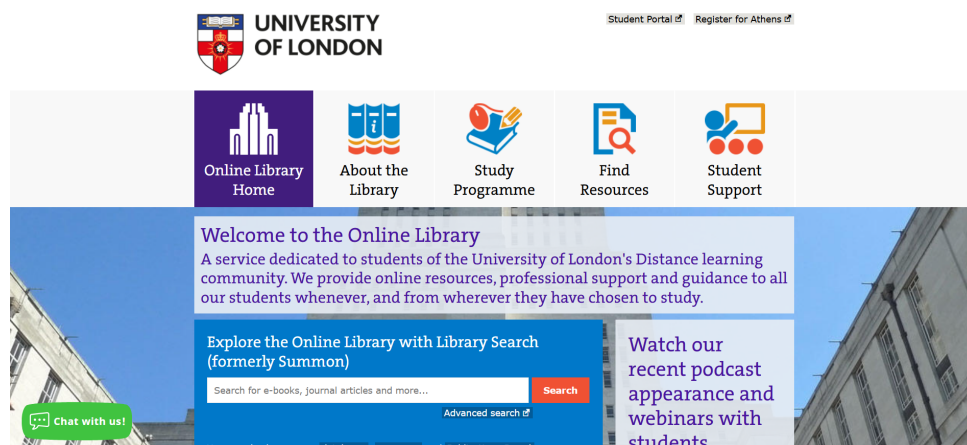


Figure 3: Unified discovery search interface and live support tool at The Online Library - UoL

Analyzing the UoL system provides critical insights into optimizing digital user experience while highlighting technological gaps in intelligent algorithmic user interaction.

2.2.1 Strengths

- Provides a single access gateway for all digital resources, reducing search latency for users.
- Employs modern authentication protocols (OpenAthens), enabling seamless remote off-campus access without requiring a VPN.
- Categorizes digital materials by specific academic disciplines to guide student literature exploration efficiently.

2.2.2 Weaknesses

- Model is restricted strictly to digital assets, entirely omitting physical book circulation and stack management workflows.
- Centralized discovery searching can induce information overload and lacks behavioral analytics for personalized recommendation.
- Chatbot support operates on static pre-scripted rules, failing to interpret complex queries and relying heavily on human librarian intervention.

2.3 University of Delaware Library System (UD Library)

The University of Delaware Library (UD Library) system at <https://library.udel.edu/> exemplifies a highly digitized multi-branch university library governance model. Unlike UoL's pure digital focus, UD Library tackles a more complex challenge: managing and distributing data across central and departmental libraries, while pioneering AI virtual assistant integration in student support workflows.

At the core of this system lies **DELCAT** (Unified Discovery Service), serving as a single interface connecting users with millions of records across diverse formats. Notably, UD Library has begun deploying conversational natural language AI solutions alongside traditional search mechanisms.

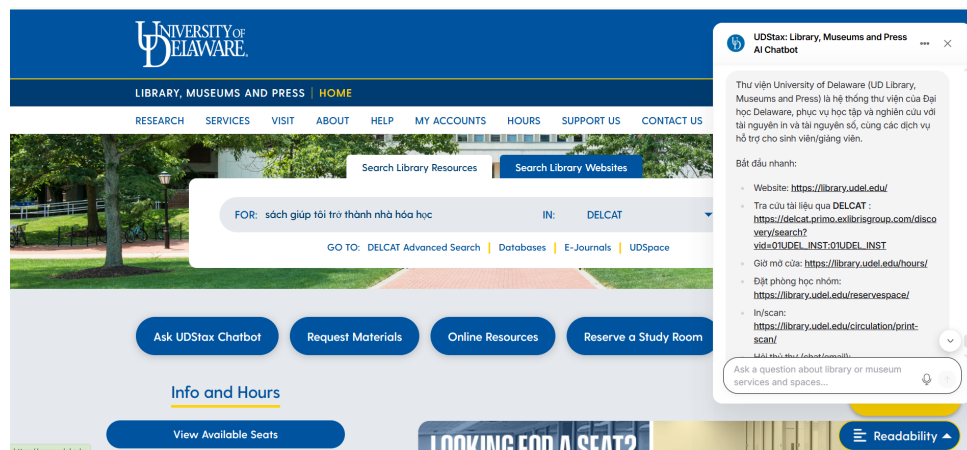


Figure 4: DELCAT unified discovery service interface and supporting AI tool at UD Library

Investigating UD Library offers important architectural perspectives on constructing an "All-in-one" unified discovery platform and operating conversational AI assistants in large academic environments.

2.3.1 Strengths

- Integrates virtual assistants capable of processing natural language to automate routine procedural Q&A.
- DELCAT platform unifies print book and electronic resource discovery within a single user interface.
- Connects non-traditional library services, such as group study room reservations, printing services, and interlibrary loans.

2.3.2 Weaknesses

- Chatbot operates independently as a procedural advisory channel without deep integration into search engines for direct book recommendation.
- Unified discovery architecture frequently causes information overload by returning excessive search result volumes.

2.4 Open Source Library Management System – Koha ILS

Koha ILS is the world's most widely adopted open-source Integrated Library System (ILS), utilized by numerous academic institutions and public library networks. Unlike user-facing OPAC portals or Discovery platforms analyzed previously, Koha represents a backend core system focused on digitizing and automating complex administrative workflows for library staff (Librarians).

Through its Staff Client administration portal, Koha offers comprehensive modules for managing physical and digital libraries, including: Circulation, International Cataloging, Patron Management, and Statistical Reporting. Studying Koha provides valuable reference standards for designing database schemas and business rule logic in our project.

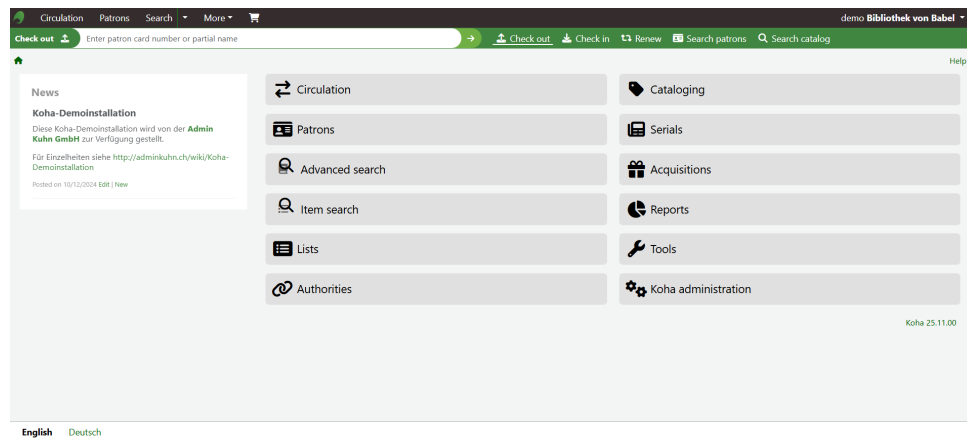


Figure 5: Koha Staff Client administrative interface module

Analyzing Koha helps clearly delineate essential core features required in library management software while exposing user experience friction present in legacy administrative systems.

2.4.1 Strengths

- Strictly complies with international library metadata standards (MARC21, Z39.50), guaranteeing data consistency and interoperability.
- Circulation module provides exhaustive rule matrices for configuring borrowing, returning, renewals, and overdue fine logic tailored to user groups.
- Features built-in REST APIs allowing third-party applications to interact and extract data for platform extension.
- Reporting system supports direct custom SQL queries, delivering maximum flexibility for administrative reporting.

2.4.2 Weaknesses

- Staff Client interface (UI/UX) is verbose, adhering to traditional form-heavy layouts that increase librarian training curves.
- Data cataloging workflows remain heavily manual, lacking intelligent automated classification or metadata extraction mechanisms.
- Monolithic system architecture and complex relational database schemas hamper rapid decoupling or custom modular feature additions.

3 System Requirements

3.1 Functional Requirements

3.1.1 User Account Management

- The system must allow new users (readers) to register accounts by providing full name, email address, student or staff ID, and password.
- The system must authenticate users via username or email and password.
- The system must support at least three user roles: Reader, Librarian, and Administrator, each endowed with distinct authorization privileges.
- Users must be able to view and update their personal profile information, including contact details and notification preferences.
- The system must enable users to reset passwords via verification links dispatched to their registered email addresses.
- Administrators must be empowered to activate, deactivate, or temporarily suspend any user account.
- The system must log borrowing history for each reader, accessible to the respective reader and authorized librarians.

3.1.2 Resource Catalog Management

- The system must maintain a catalog of all library assets, including physical books and periodicals.
- Each catalog record must store: title, author(s), ISBN/ISSN, publisher, publication year, edition, genre/subject, language, physical location (shelf/classification code), total copy count, and available copy count.
- Librarians must be able to add, edit, and delete catalog records via the administration interface.
- The system must support batch catalog data ingestion via CSV and Excel (XLSX) file formats.
- The system must provide full-text search capability across title, author, ISBN, subject, and keyword fields.
- Search results must support filtering by category, language, availability status, publication year range, and media format.
- The system must display real-time copy availability for each item, including available, checked out, and reserved counts.

3.1.3 Circulation Management

- The system must allow readers to check out physical materials by barcode scanning or manual entry at circulation desks, or by placing online pickup requests.
- The system must enforce role-based loan limits (e.g., maximum 5 items for students, maximum 15 items for faculty).
- The system must assign due dates to borrowed items based on configurable loan duration rules per media type.
- The system must allow readers to renew checked-out items online up to a configurable maximum count, provided no hold requests exist for those items.
- The system must allow readers to place holds on items currently checked out by other users.

- The system must automatically notify the next user in a hold queue once a reserved item becomes available.
- The system must process item returns and immediately update copy availability in the catalog.
- The system must automatically calculate overdue fines based on daily configurable rates per material type.
- The system must enable readers to view active loans, due dates, holds, and outstanding fines from their user dashboard.
- The system must support inter-branch borrowing, enabling readers to request materials from other branch libraries in the network.

3.1.4 AI Natural Language Search and Intelligent Interaction

- The system must provide a natural language search interface allowing readers to query the catalog using conversational phrasing.
 - The natural language search module must analyze user intent, extract attributes (genre, target audience, topic, length), and map them to catalog metadata.
 - The system must display relevance-ranked natural language search results alongside standard keyword search outputs.
 - The system must integrate a Virtual Assistant Chatbot for direct user interaction, resolving FAQ inquiries regarding library policies, operating hours, and loan procedures.

3.1.5 Notifications and Communication

- The system must dispatch automated email notifications for: 3-day pre-due-date reminders, overdue alerts, hold availability notices, and account status updates.
 - Readers must be able to configure notification channels and frequencies within profile settings.
 - The system must provide an in-app notification center displaying unread system alerts.
 - Librarians must be able to broadcast mass announcements (e.g., library closure notices) to all users or target groups.

3.1.6 Reporting and Analytics

- The system must generate analytical reports regarding: top-borrowed items, overdue items, fine collection summaries, new user registrations, and catalog growth metrics.
 - The system must feature a real-time dashboard for librarians displaying key operational metrics: active loans, overdue counts, copy availability, and daily transaction volumes.
 - Reports must support export to PDF and Excel formats.

3.1.7 System Administration and Configuration

- Administrators must be able to configure global parameters including loan durations, fine rates, role borrowing limits, and hold expiration thresholds.
 - The system must support multi-branch library management.

- The system must record full audit logs for administrative actions (configuration changes, account edits, batch ingestion).

3.2 Non-Functional Requirements

Non-functional requirements define the operational standards that ensure the system not only performs its business logic correctly, but also runs stably, securely, and delivers a positive user experience.

3.2.1 Performance and User Experience

- **Query Optimization:** Frequently accessed and infrequently changing data (book categories, branch locations, system configuration values) are cached at the application layer to reduce redundant database round-trips, thereby improving page rendering speed.
- **UI/UX Consistency:** Typography, color palette, navigation layouts, and interactive controls (Renew, Hold, Process Return) must remain visually and behaviorally consistent across all user roles and views, providing a coherent and professional user experience.

3.2.2 Security and Access Control

- **Role-Based Access Control:** The system enforces strict RBAC combined with stateless JWT session tokens, ensuring that Readers, Librarians, and Administrators can only access interfaces and perform operations within their designated authorization boundaries.
- **Data Safety:** User passwords are never stored in plain text — all credentials undergo salted hashing via bcrypt prior to persistence. All form inputs and API payloads are validated and sanitized server-side to prevent common injection and scripting vulnerabilities.
- **Rate Limiting and CORS:** API endpoints are protected by rate-limiting middleware to mitigate brute-force attacks, and Cross-Origin Resource Sharing policies are explicitly configured to restrict unauthorized cross-domain access.

3.2.3 Reliability and Maintainability

- **Data Consistency:** Critical business operations — including loan record creation, fine assessment, and reservation queue updates — are wrapped in ACID-compliant PostgreSQL transactions via Prisma ORM. This guarantees that no partial or inconsistent state is persisted in the event of mid-operation failures.
- **Code Quality and Maintainability:** The backend codebase is organized into clearly separated functional modules (authentication, catalog, circulation, notifications, AI service) with strict TypeScript static typing enforced throughout. This architecture limits the surface area for latent bugs and enables future developers to onboard, extend, and maintain individual modules independently without risk of cascading side-effects.
- **Recoverability:** The system supports on-demand database snapshots and point-in-time restoration of system configurations and branch data, providing a recovery mechanism against accidental data loss or configuration corruption.

4 Detailed Use-Case Specifications

4.1 User Account and Access Management

4.1.1 Overview

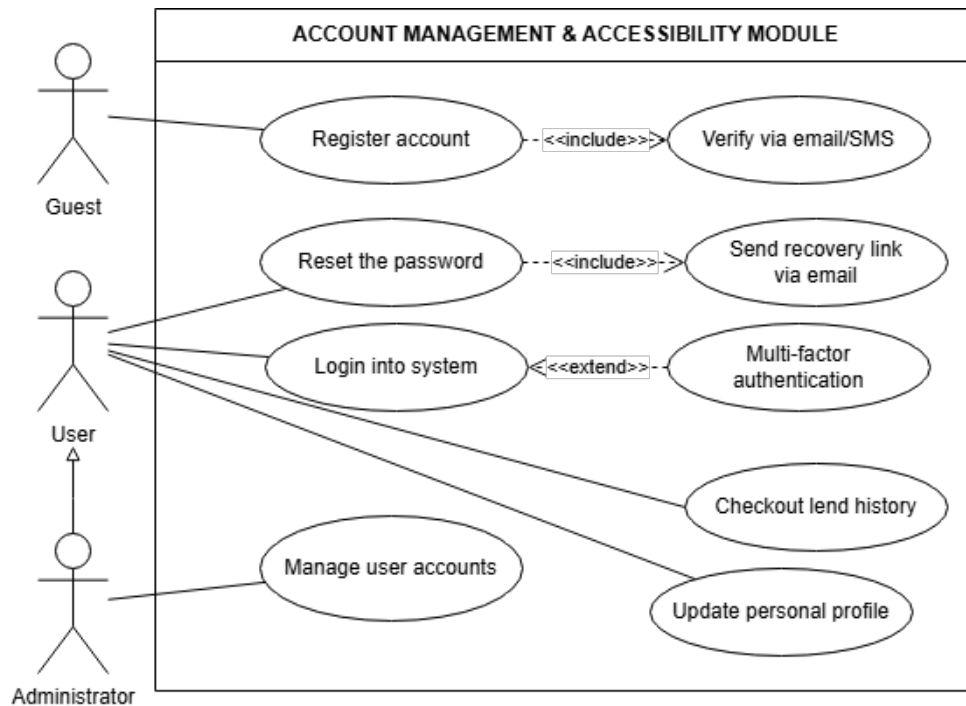


Figure 6: User Account and Access Management Use-Case Diagram

4.1.2 Detailed Specifications

Use-case Name	Account Registration
Use-case ID	UC-ACC-01
Overview	Allows unregistered users to sign up and become system members.
Actor	Guest
Pre-conditions	Guest accesses the registration page of the system.
Trigger Condition	Guest clicks the "Register" button.
Flow of Events	1. System displays the registration form. 2. Guest inputs required fields (Full name, Email, Password, etc.). 3. System validates input data integrity. 4. [Include] System executes "Send Verification Email/SMS" containing an OTP or activation link to the registered contact info. 5. Guest inputs the verification code or clicks the activation link. 6. System verifies the token, creates a new account, and persists it to the database. 7. System displays a success message.
Post-conditions	Account is created successfully and status transitions to "Activated".
Exceptions	Ex1: Email/Phone number already exists → System reports an error and requests re-entry. Ex2: Invalid verification code entered beyond maximum attempts → Cancel registration session.

Table 1: Use-case Specification: Account Registration

Use-case Name	Password Reset
Use-case ID	UC-ACC-02
Overview	Assists users in recovering account access when forgetting login passwords.
Actor	User (All Roles)
Pre-conditions	User possesses an existing system account but cannot recall password.
Trigger Condition	User clicks "Forgot Password" on the login view.
Flow of Events	1. System prompts user for registered email. 2. User inputs email and submits. 3. System verifies email existence in the database. 4. [Include] System triggers "Send Recovery Link via Email" flow. 5. User accesses the reset link received in email. 6. System renders password reset screen. 7. User inputs new password and confirms. 8. System hashes and updates new password, showing success alert.
Post-conditions	User password is updated successfully.
Exceptions	Ex1: Email does not exist → Display "No account found associated with this email". Ex2: Reset link expired → Prompt user to restart the reset process.

Table 2: Use-case Specification: Password Reset

Use-case Name	System Login
Use-case ID	UC-ACC-03
Overview	Authenticates user identity to grant system access matching role privileges.
Actor	User (All Roles)
Pre-conditions	User possesses an active account in the system.
Trigger Condition	User accesses the login view and enters credentials.
Flow of Events	1. User submits identity credentials (Email/Username) and Password. 2. System validates input formats and matches credentials against DB. 3. [Extend] If Multi-Factor Authentication (MFA) is configured, system triggers "MFA Verification", requesting OTP from app/SMS. 4. Initializes user session (Session/JWT Token). 5. Redirects user to the dashboard corresponding to their assigned Role.
Post-conditions	User logs in successfully; system records login timestamp audit.
Exceptions	Ex1: Incorrect credentials → Display "Incorrect account or password". Ex2: Account suspended → Display "Your account is locked. Please contact Admin". Ex3: MFA code failed 3 attempts → Lock login session temporarily for 15 minutes.

Table 3: Use-case Specification: System Login

Use-case Name	Update User Profile
Use-case ID	UC-ACC-04
Overview	Allows users to modify personal profile information on the platform.
Actor	User
Pre-conditions	User is currently logged into the system.
Trigger Condition	User accesses "Profile Settings" and selects "Edit".
Flow of Events	1. System populates form with current profile data. 2. User updates permissible fields (Phone, address, avatar image...). 3. User clicks "Save". 4. System validates field constraints. 5. Updates database record and displays success notification.
Post-conditions	User profile is updated.
Exceptions	Ex1: Invalid avatar format or file size exceeded → Warning alert and file rejection.

Table 4: Use-case Specification: Update User Profile

Use-case Name	View Loan/Return History
Use-case ID	UC-ACC-05
Overview	Enables users to review past and active item borrowings along with their statuses.
Actor	User
Pre-conditions	User is logged into the system.
Trigger Condition	User selects "Borrowing History" in personal dashboard.
Flow of Events	1. System receives user request. 2. Queries database for circulation records tied to user ID. 3. Renders loan list (Item title, borrow date, due date, return date, status, fines). 4. User can filter results by date range or status.
Post-conditions	System state remains unchanged.
Exceptions	Ex1: No loan records found → Display "You have no borrowing history".

Table 5: Use-case Specification: View Loan/Return History

Use-case Name	User Account Management
Use-case ID	UC-ACC-06
Overview	Empowers administrators to search, view, authorize, activate, or suspend user accounts.
Actor	Administrator (Admin)
Pre-conditions	Administrator is logged in with administrative access privileges.
Trigger Condition	Admin navigates to "User Management" menu.
Flow of Events	1. System displays table of all user accounts. 2. Admin filters or searches accounts by role or account status. 3. Admin selects a specific user to perform actions (Lock/Unlock, Reset Password, Change Role). 4. Admin confirms the administrative operation. 5. System persists changes to DB and writes audit log. 6. System displays success confirmation.
Post-conditions	Target account status or permission set is updated.
Exceptions	Ex1: Admin attempts to lock their own account → System rejects action and raises logic violation error.

Table 6: Use-case Specification: User Account Management

4.2 Catalog and Metadata Management

4.2.1 Overview

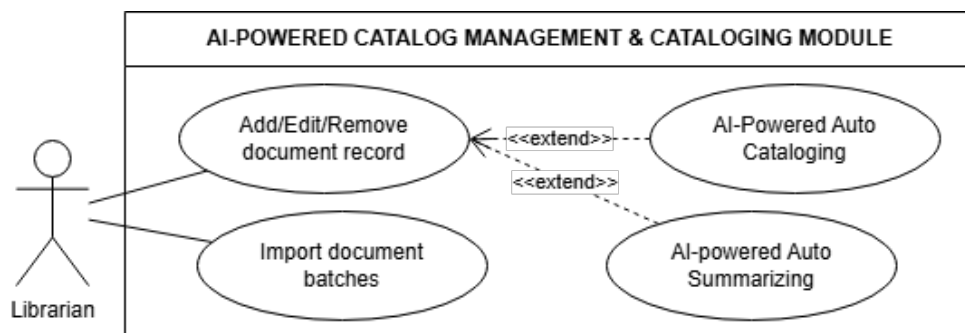


Figure 7: Catalog & AI Metadata Management Use-Case Diagram

4.2.2 Detailed Specifications

Use-case Name	Add / Edit / Delete Catalog Record
Use-case ID	UC-CAT-01
Overview	Librarians perform catalog management operations (CRUD), updating metadata or removing lost/damaged copies.
Actor	Librarian
Pre-conditions	Librarian is logged into the system.
Trigger Condition	Librarian navigates to "Catalog Management".
Flow of Events	1. System displays list of current catalog items. 2. Librarian selects action: Add New, Edit, or Delete. 3. If Delete: Librarian confirms deletion, system checks active loan constraints and executes Soft Delete. 4. If Add/Edit: System renders item form (Title, Author, ISBN, Category, Copies...). ** Extension Points: ** - [Extend] Librarian can execute "Automated Cataloging via ISBN" (UC-CAT-03) to auto-fill form. - [Extend] Librarian can execute "Automated Document Summarization" (UC-CAT-04) if description is missing. 5. Librarian reviews form fields and clicks "Save". 6. System validates fields, persists to DB, and displays success alert.

Post-conditions	Catalog record is updated successfully.
Exceptions	Ex1: Required fields missing → System highlights missing inputs and requests completion. Ex2: Duplicate ISBN code → Display "ISBN already exists in catalog".

Table 7: *Use-case Specification: Add / Edit / Delete Catalog Record*

Use-case Name	Batch Catalog Data Ingestion
Use-case ID	UC-CAT-02
Overview	Allows librarians to import large volumes of catalog records via standard data files (CSV, Excel).
Actor	Librarian
Pre-conditions	Librarian is logged into the system.
Trigger Condition	Librarian clicks "Batch Import" in catalog management.
Flow of Events	1. System presents upload interface and sample template file. 2. Librarian uploads catalog dataset file. 3. System parses file and displays preview table distinguishing valid vs invalid records. 4. Librarian confirms import operation. 5. System writes valid rows to DB and generates execution report (success vs error row counts).
Post-conditions	Multiple catalog records added to system.
Exceptions	Ex1: Unsupported file format → System rejects upload: "Unsupported file format". Ex2: File contains duplicate ISBNs → System skips duplicate lines, logs errors, and imports remaining valid records.

Table 8: Use-case Specification: Batch Catalog Data Ingestion

Use-case Name	Automated Cataloging via ISBN
Use-case ID	UC-CAT-03
Overview	AI-assisted feature to automatically retrieve and populate book metadata (Title, Author, Publisher, Genre) from external APIs using ISBN lookup.
Actor	Librarian
Pre-conditions	Currently executing flow of "Add / Edit / Delete Catalog Record".
Trigger Condition	Librarian enters ISBN and clicks "Auto-Fetch Metadata".
Flow of Events	1. Librarian enters ISBN into corresponding input field. 2. System dispatches API request to open catalog endpoints (Google Books, Open Library). 3. System receives JSON response, parsing entity attributes. 4. System auto-populates form fields with parsed attributes. 5. Librarian reviews, edits if needed, and proceeds with base cataloging flow.
Post-conditions	Catalog form is auto-filled, reducing manual entry time for librarians.
Exceptions	Ex1: ISBN not found in external databases → System alerts "No metadata found, please enter manually". Ex2: Third-party API timeout → System alerts timeout error and asks librarian to retry.

Table 9: Use-case Specification: Automated Cataloging via ISBN

Use-case Name	Automated Document Summarization
Use-case ID	UC-CAT-04
Overview	Uses Large Language Models (LLMs) to automatically generate concise abstract summaries (150-300 words) for catalog items lacking publisher descriptions.
Actor	Librarian
Pre-conditions	Executing "Add / Edit / Delete Catalog Record" flow. Title and Author fields populated.
Trigger Condition	Librarian clicks "Generate AI Summary" in Description field.
Flow of Events	1. System collects existing item fields (Title, Author, Category). 2. System formats prompt payload and sends to AI Engine (e.g., Gemini API). 3. AI Engine processes payload and returns summary text. 4. System populates text into Description field, automatically tagging "(AI-generated summary)". 5. Librarian reviews, edits, or requests regeneration.
Post-conditions	Item receives concise abstract summary, aiding reader discovery.
Exceptions	Ex1: Insufficient metadata input causing inaccurate AI synthesis → Generic output returned; librarian discards and clears output.

Table 10: Use-case Specification: Automated Document Summarization

4.3 Document Circulation Management

4.3.1 Overview

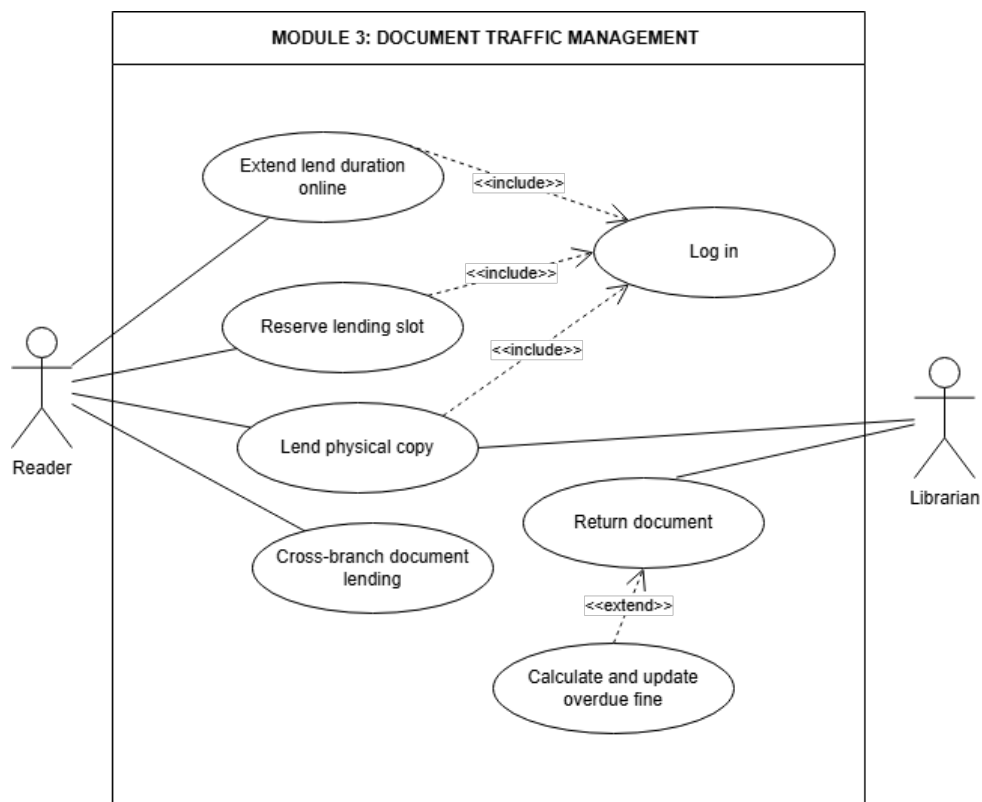


Figure 8: Document Circulation Management Use-Case Diagram

4.3.2 Detailed Specifications

Use-case Name	Physical Material Checkout
Use-case ID	UC-CIR-01
Overview	Allows readers to borrow physical materials available at the library counter assisted by a librarian.
Actor	Reader, Librarian

Pre-conditions	Reader and Librarian possess valid active accounts. Reader has no card suspension or unpaid fines exceeding threshold.
Trigger Condition	Reader presents items and membership card at circulation desk.
Flow of Events	1. [Include] Librarian logs into circulation module. 2. Librarian scans Reader membership barcode/ID. 3. System loads Reader profile, checking borrowing eligibility and fine status. 4. Librarian scans barcode on each checkout item. 5. System checks active loan limits against role allowance. 6. System creates loan record, calculating due dates based on item policy rules. 7. System updates copy status in inventory to "Checked Out" and decrements available count. 8. Librarian hands items to Reader; transaction completes.
Post-conditions	Loan record created in system; transaction logged.
Exceptions	Ex1: Reader reached maximum borrowing limit → Error alert "Loan limit exceeded"; reader must return existing items first. Ex2: Item scanned is on hold for another patron → Warning alert; librarian retains item for hold queue.

Table 11: *Use-case Specification: Physical Material Checkout*

Use-case Name	Material Return
Use-case ID	UC-CIR-02
Overview	Processes returned physical materials, closes active loan records, and updates copy availability in inventory.
Actor	Librarian
Pre-conditions	Items are associated with active open loan records.
Trigger Condition	Reader returns items at circulation desk.
Flow of Events	1. [Include] Librarian logs into circulation module. 2. Librarian scans item barcode. 3. System retrieves associated active loan record. 4. System compares return date against scheduled due date. ** Extension Points: ** - [Extend] If return date exceeds due date, system triggers "Calculate and Assess Overdue Fines" (UC-CIR-03). 5. Updates loan status to "Completed". 6. System verifies hold queue status for returned item: - If hold exists: Set status to "Awaiting Pickup" and send notification to next patron. - If no hold: Set status to "Available" and increment copy inventory count.
Post-conditions	Loan record closed. Item marked available for next transaction.
Exceptions	Ex1: Barcode damaged/unreadable → Librarian performs manual lookup by title and borrower ID.

Table 12: Use-case Specification: Material Return

Use-case Name	Calculate and Assess Overdue Fines
Use-case ID	UC-CIR-03
Overview	Automatically computes fine amounts when items are returned past due dates.
Actor	Librarian
Pre-conditions	Triggered from "Material Return" when system detects overdue return.
Trigger Condition	Return Date > Due Date in loan record.
Flow of Events	1. System computes overdue days: (Actual Return Date - Due Date). 2. Fetches daily fine rate applicable to material type (configured by Admin). 3. Calculates total fine amount (Overdue Days × Daily Rate). 4. Displays fine calculation on librarian view. 5. Librarian notifies reader. If reader pays immediately, receipt is generated; otherwise fine is posted to reader ledger.
Post-conditions	Fine record updated on reader profile or marked as paid.
Exceptions	Ex1: Official holidays fall within overdue range → System automatically excludes official library holidays from overdue day count.

Table 13: Use-case Specification: Calculate and Assess Overdue Fines

Use-case Name	Online Loan Renewal
Use-case ID	UC-CIR-04
Overview	Allows readers to extend loan periods online without physically visiting the library.
Actor	Reader
Pre-conditions	Item is not overdue and renewal limit has not been reached.
Trigger Condition	Reader clicks "Renew" in active loans dashboard.
Flow of Events	1. [Include] Reader logs into system portal. 2. System verifies renewal eligibility: - Has maximum renewal limit been reached for item? - Does an active hold request exist from another user? 3. If eligible, system extends due date by configured extension period. 4. System updates loan record and increments renewal counter. 5. Displays "Renewal Successful" along with new due date.
Post-conditions	Item due date is extended.
Exceptions	Ex1: Item has active hold request → System rejects renewal: "Item requested by another user, please return on time".

Table 14: Use-case Specification: Online Loan Renewal

Use-case Name	Place Item Hold / Reservation
Use-case ID	UC-CIR-05
Overview	Enables readers to join a reservation queue for items currently out of stock.
Actor	Reader
Pre-conditions	Target item status is "All Copies Checked Out".
Trigger Condition	Reader clicks "Place Hold" on item detail view.
Flow of Events	1. [Include] Reader logs into system portal. 2. System verifies Reader does not currently hold an active loan for same title. 3. System appends Reader ID to item reservation queue. 4. Interface displays "Hold Placed Successfully" showing queue position. 5. Upon item return, system holds copy and alerts queue position #1 patron.
Post-conditions	Reservation saved; renewal blocked for current borrower.
Exceptions	Ex1: Reader has unpaid fines or card suspension → System rejects hold request.

Table 15: Use-case Specification: Place Item Hold

Use-case Name	Inter-Branch Loan Request
Use-case ID	UC-CIR-06
Overview	Allows readers to request item transfers from another branch library to their local campus library for pickup.
Actor	Reader
Pre-conditions	Multi-branch network enabled. Item unavailable locally but available at another branch.
Trigger Condition	Reader clicks "Request Inter-Branch Transfer" on book detail page.
Flow of Events	1. [Include] Reader logs into system portal. 2. System prompts Reader to select preferred pickup branch. 3. Reader confirms request. 4. System creates inter-branch transfer ticket sent to holding branch. 5. Holding librarian packages item; status changes to "In Transit". 6. Upon arrival at destination branch, automated notification alerts Reader for pickup.
Post-conditions	Physical transfer logged and inventory updated across branches.
Exceptions	Ex1: Last available copy borrowed before request processing → Cancel request and suggest standard hold queue.

Table 16: Use-case Specification: Inter-Branch Loan Request

4.4 Resource Access and Exploitation

4.4.1 Overview

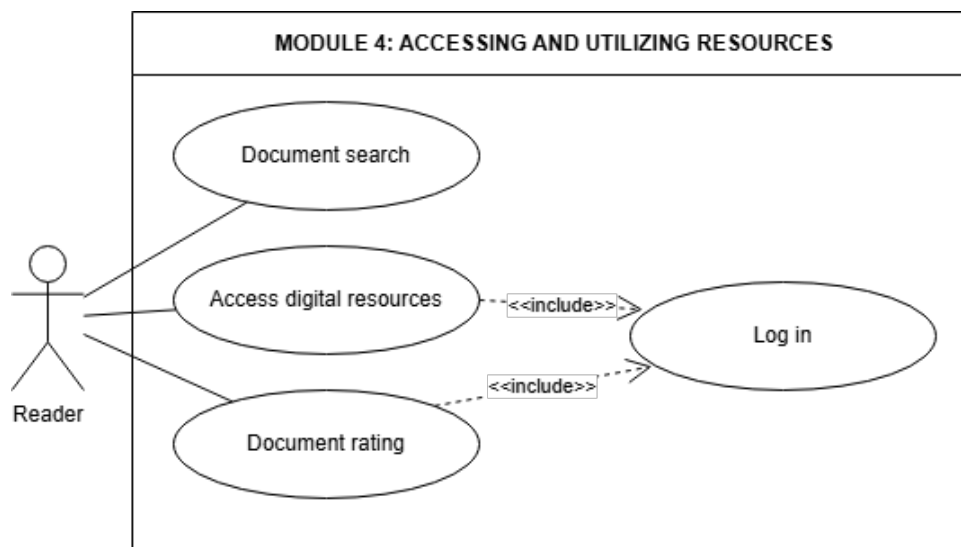


Figure 9: Resource Access & Exploitation Use-Case Diagram

4.4.2 Detailed Specifications

Use-case Name	Document Catalog Search
Use-case ID	UC-EXP-01
Overview	Allows readers to search library catalog records via keywords (Title, Author, Genre, ISBN).
Actor	Reader (Including unauthenticated guests)
Pre-conditions	System online with active DB connection.
Trigger Condition	Reader inputs query string and submits search.
Flow of Events	1. Reader selects search filter scope (All, Title, Author...) and enters query. 2. System queries catalog database indexes. 3. System returns paginated match results. 4. Renders item cards (Cover image, Title, Author, Year, Availability Status). 5. Reader clicks item card to open detail view.

Post-conditions	System state remains unchanged.
Exceptions	Ex1: No items match query → Display "No results found" with spelling/keyword suggestions.

Table 17: Use-case Specification: Document Catalog Search

Use-case Name	Rate and Review Materials
Use-case ID	UC-EXP-03
Overview	Allows readers to leave 1-5 star ratings and written reviews for items they have read.
Actor	Reader
Pre-conditions	Reader has borrowed the physical item (preventing spam).
Trigger Condition	Reader clicks "Write a Review" on book page.
Flow of Events	1. [Include] System verifies login status. 2. System verifies loan history against target item. 3. Renders rating selector (1-5 stars) and review text area. 4. Reader submits rating and optional review. 5. System checks content moderation filters. 6. Saves rating, updates item average rating score. 7. Displays success alert and refreshes view.
Post-conditions	Review saved; average item score updated.
Exceptions	Ex1: Reader has not borrowed item → Hide review button or display "You must borrow this item before submitting a review". Ex2: Review contains profanity → Block submission with moderation warning.

Table 18: Use-case Specification: Rate and Review Materials

4.5 Intelligent Interaction

4.5.1 Overview

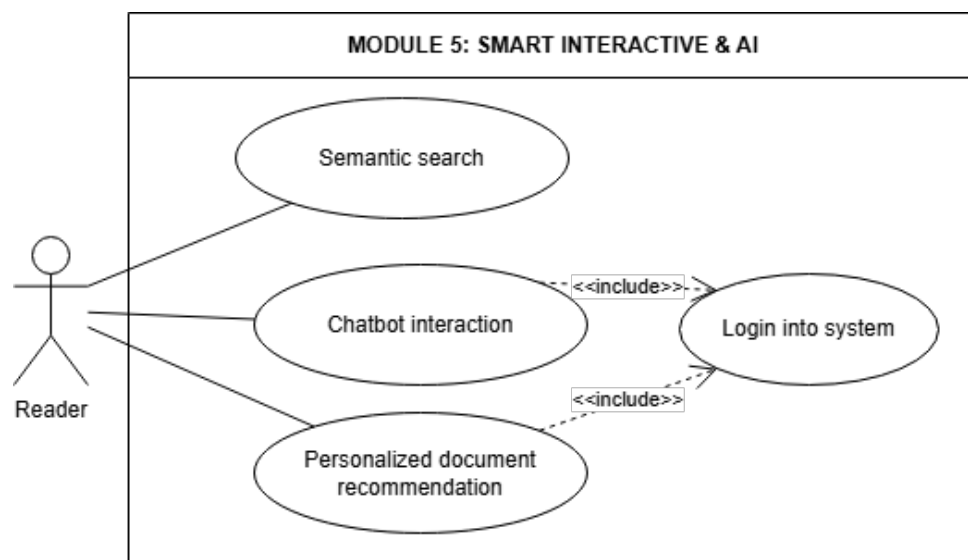


Figure 10: Intelligent Interaction & AI Use-Case Diagram

4.5.2 Detailed Specifications

Use-case Name	Natural Language Search
Use-case ID	UC-AI-01
Overview	Allows readers to search catalog items using conversational phrases (e.g., "introductory artificial intelligence textbooks") rather than exact keywords.
Actor	Reader

Pre-conditions	System AI NLP module operational.
Trigger Condition	Reader inputs phrase into Smart Search bar and submits.
Flow of Events	1. Reader enters natural language query. 2. NLP module processes query via Intent Analysis & Entity Extraction (genre, topic, target level). 3. System converts entities into vector embeddings or metadata filter trees. 4. Executes vector similarity search (pgvector) and semantic ranking. 5. Renders ranked search results sorted by semantic relevance scores.
Post-conditions	Semantic search results rendered with related topic tags.
Exceptions	Ex1: Irrelevant or meaningless phrase → AI assistant prompts "No matching documents found. Try phrases like..." Ex2: AI service offline → System falls back to full-text keyword search and alerts "Smart search temporarily under maintenance".

Table 19: *Use-case Specification: Natural Language Search*

Use-case Name	Chatbot Virtual Assistant Interaction
Use-case ID	UC-AI-02
Overview	Readers interact with an AI Chatbot for policy inquiries, opening hours, or checking personal account loan status.
Actor	Reader
Pre-conditions	AI Chatbot service online.
Trigger Condition	Reader opens Chatbot widget window.
Flow of Events	1. [Include] System prompts login if unauthenticated for personalized responses. 2. Chatbot interface displays welcome message and quick action buttons (FAQ, My Loans, Book Suggestions). 3. Reader types message or selects quick prompt. 4. Chatbot (LLM) analyzes context: - General FAQ: Retrieves answer from RAG knowledge base. - Account query (e.g., "What books do I owe?"): Chatbot queries backend API using Reader ID and synthesizes response. 5. Renders conversational response in chat widget.
Post-conditions	Chat transcript saved for audit logging and AI refinement.
Exceptions	Ex1: Query out of scope → Response: "I am a library assistant. Please contact staff at hotline for special inquiries."

Table 20: Use-case Specification: Chatbot Virtual Assistant Interaction

Use-case Name	View Personalized Recommendations
Use-case ID	UC-AI-03
Overview	Provides readers with tailored book recommendations generated from loan history, search logs, and collaborative user behavior.
Actor	Reader
Pre-conditions	AI Recommendation engine online.
Trigger Condition	Reader opens homepage dashboard or "Recommended For You" view.
Flow of Events	1. [Include] Reader logs into system. 2. System aggregates user signals: loan history, recent searches, ratings. 3. System sends telemetry to Recommendation Engine. 4. Engine executes Collaborative Filtering and Content-Based Filtering models. 5. Filters out items currently checked out or held by user. 6. Displays "Recommended For You" grid with brief rationale tags (e.g., "Because you read Title X...").
Post-conditions	Interface presents personalized recommendations.
Exceptions	Ex1: New account with no history (Cold Start) → Display "Trending Titles" or "New Arrivals" fallback.

Table 21: Use-case Specification: View Personalized Recommendations

4.6 Notifications and Communication

4.6.1 Overview

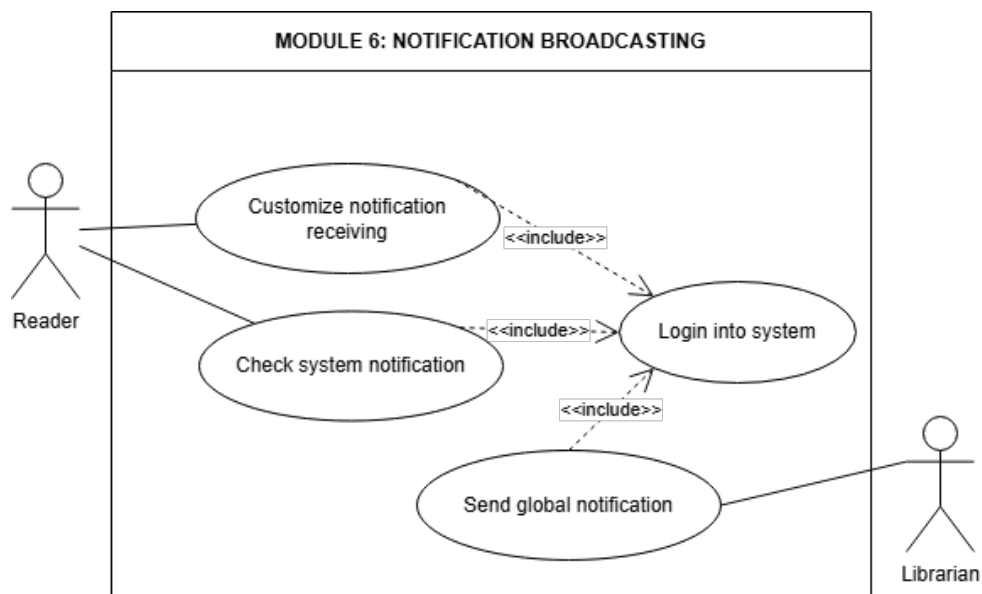


Figure 11: Notifications & Communication Use-Case Diagram

4.6.2 Detailed Specifications

Use-case Name	Configure Notification Preferences
Use-case ID	UC-NOT-01
Overview	Allows readers to customize delivery channels (Email, SMS, In-app) and event alerts (Due Reminders, Hold Pickups, System News).
Actor	Reader
Pre-conditions	Reader possesses valid account.
Trigger Condition	Reader opens "Notification Settings" in profile view.

Flow of Events	1. [Include] System prompts login verification. 2. System loads and renders current notification configuration toggles. 3. Reader toggles preferences for channels and event categories. 4. Reader clicks "Save Settings". 5. System validates and persists configuration to DB. 6. Displays "Notification preferences saved successfully".
Post-conditions	System dispatches notifications strictly per user preferences.
Exceptions	Ex1: SMS toggled on without phone number in profile → Warning alert requesting phone number update first.

Table 22: *Use-case Specification: Configure Notification Preferences*

Use-case Name	View System Notifications
Use-case ID	UC-NOT-02
Overview	Enables users to review alerts and reminders directly within the web app interface.
Actor	Reader
Pre-conditions	User has incoming notification records.
Trigger Condition	User clicks "Bell" icon (Notification Center) on top navbar.
Flow of Events	1. [Include] System verifies user login. 2. Queries notification table by User ID sorted by timestamp desc. 3. Renders notification panel distinguishing read vs unread items. 4. User clicks notification item to view details. 5. System marks notification as "Read" and decrements unread badge count.
Post-conditions	Notification status updated to "Read".
Exceptions	Ex1: Notification panel empty → Display "You have no new notifications".

Table 23: Use-case Specification: View System Notifications

Use-case Name	Broadcast Mass Notifications
Use-case ID	UC-NOT-03
Overview	Allows Librarians to create and send general announcements (holiday closures, maintenance, policy updates) to all or targeted reader groups.
Actor	Librarian
Pre-conditions	Librarian active with broadcast privileges.
Trigger Condition	Librarian accesses "Communication Management" tool in admin portal.
Flow of Events	1. [Include] System verifies Librarian session and privileges. 2. Renders broadcast creation form. 3. Librarian inputs: Title, Content, Channel (In-app, Email), Target Group (All, Students, Faculty). 4. Librarian clicks "Send Broadcast". 5. System prompts confirmation dialog. 6. System enqueues notification payload in Message Queue for dispatch. 7. Displays success confirmation and logs administrative action.
Post-conditions	Broadcast logged and dispatched to target users.
Exceptions	Ex1: Empty title or body → Disable send button and alert "Please enter complete broadcast content".

Table 24: Use-case Specification: Broadcast Mass Notifications

4.7 Reporting and System Administration

4.7.1 Overview

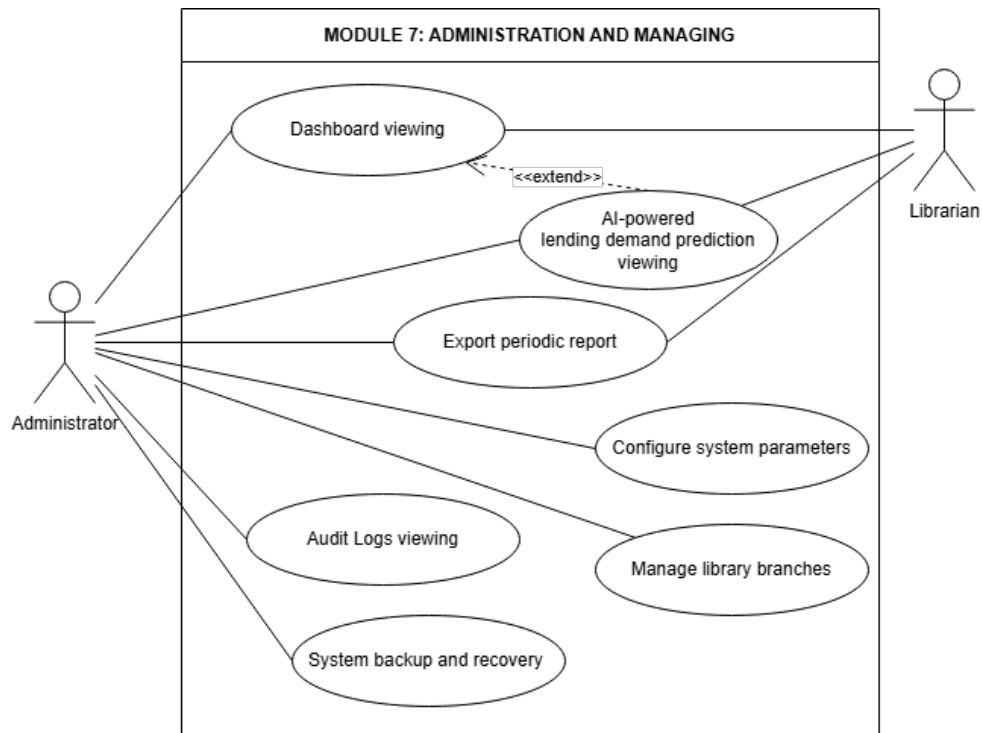


Figure 12: Reporting & Administration Use-Case Diagram

4.7.2 Detailed Specifications

Use-case Name	View Operational Dashboard
Use-case ID	UC-ADM-01
Overview	Provides real-time dashboard visualization of key operational metrics (active loans, overdue items, traffic).
Actor	Administrator (Admin), Librarian
Pre-conditions	User logged in as Admin or Librarian.
Trigger Condition	User opens administrative homepage.
Flow of Events	1. System receives request. 2. Queries DB aggregating real-time daily/weekly metrics. 3. Renders charts and metric summary cards.
Post-conditions	System data state remains unchanged.
Exceptions	Ex1: DB load high → Display cached dashboard data with refresh timestamp notice.

Table 25: Use-case Specification: View Operational Dashboard

Use-case Name	Export Periodic Statistical Reports
Use-case ID	UC-ADM-02
Overview	Supports exporting library metrics and audit data to standard formats (PDF, CSV, Excel) for archiving and reporting.
Actor	Administrator (Admin), Librarian
Pre-conditions	User logged in with report export privileges.
Trigger Condition	User opens "Statistical Reports" and selects "Export Data".
Flow of Events	1. System renders export configuration modal. 2. User selects report type (Top Borrowed, Fine Income, Overdues), date range, and export format. 3. User clicks "Export". 4. System compiles dataset and generates file artifact. 5. File download triggers automatically on client browser.
Post-conditions	Report artifact generated and downloaded.
Exceptions	Ex1: No records match date range → Warning alert "No data available for selected period".

Table 26: Use-case Specification: Export Periodic Statistical Reports

Use-case Name	Configure System Parameters
Use-case ID	UC-ADM-04
Overview	Configures core operational library rules: loan limits, loan durations, and daily overdue fine rates per item type.
Actor	Administrator (Admin)
Pre-conditions	Admin logged into system.
Trigger Condition	Admin opens "System Settings".
Flow of Events	1. System displays current global system settings. 2. Admin modifies values (e.g., fine rate adjustment). 3. Admin clicks "Save Configuration". 4. System validates parameters. 5. Updates DB and records entry in Audit Log.
Post-conditions	System calculations immediately apply updated configuration settings.
Exceptions	Ex1: Invalid parameter input (negative values) → Reject save and display validation error.

Table 27: Use-case Specification: Configure System Parameters

Use-case Name	Manage Branch Libraries
Use-case ID	UC-ADM-05
Overview	Adds, edits, or deactivates affiliated branch libraries, managing physical network topology.
Actor	Administrator (Admin)
Pre-conditions	Admin logged into system.
Trigger Condition	Admin selects "Branch Management".
Flow of Events	1. System renders active branch list. 2. Admin selects Add New, Edit, or Remove. 3. Admin enters branch attributes (Name, Address, Contact, Manager). 4. Admin clicks "Save". 5. System updates records and displays success alert.
Post-conditions	Branch directory updated.
Exceptions	Ex1: Attempting to delete branch with active inventory or staff → System prevents deletion to maintain referential integrity.

Table 28: Use-case Specification: Manage Branch Libraries

Use-case Name	View Audit Logs
Use-case ID	UC-ADM-06
Overview	Inspects historical logs of critical administrative actions (who did what, when) for security auditing.
Actor	Administrator (Admin)
Pre-conditions	Admin logged into system.
Trigger Condition	Admin accesses "Audit Logs" menu.
Flow of Events	1. System retrieves records from Audit Log table. 2. Renders event stream sorted by timestamp desc. 3. Admin filters log stream by actor, date range, or risk severity (Info, Warning, Critical). 4. Admin views detailed payload diffs (Old Data vs New Data).
Post-conditions	Read-only view; system state remains unchanged.
Exceptions	No major exceptions.

Table 29: *Use-case Specification: View Audit Logs*

5 System Analysis and Design

5.1 System Architecture

To visualize data flows and the separation of concerns across technology components in the Online Library Management System, the overall structure is modeled via the architecture diagram below:

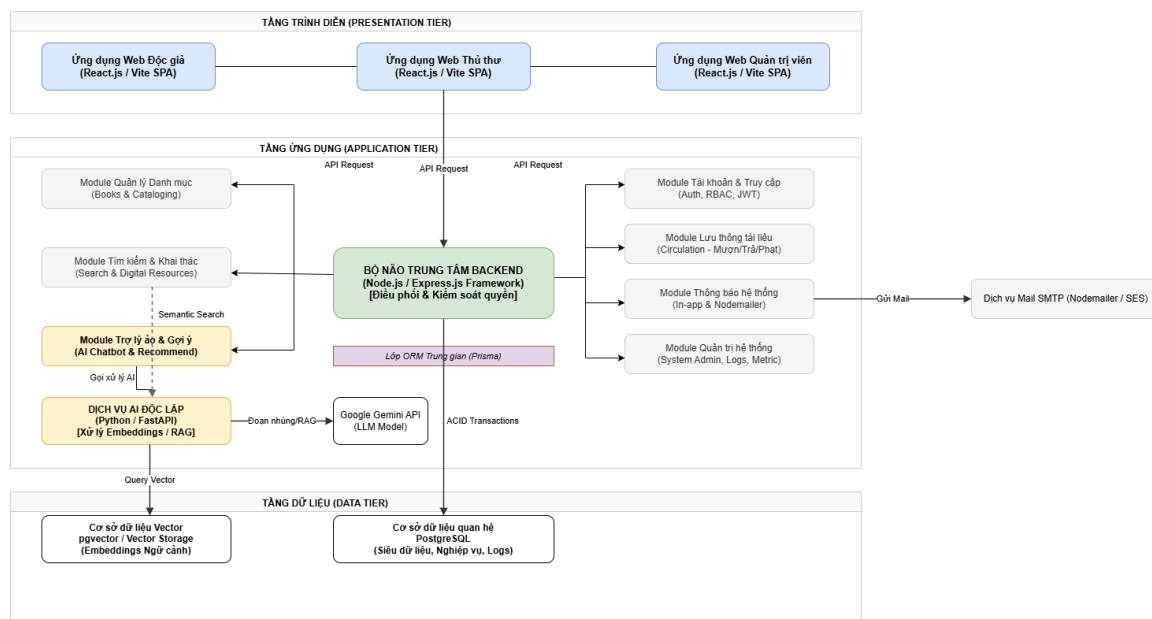


Figure 13: Overall System Architecture Diagram of the Online Library Management System

Overall, the architecture of the Online Library Management System is built upon a standard Three-Tier Architecture, combined flexibly with a Service-Oriented Architecture (SOA) paradigm to optimize independent Artificial Intelligence workloads. The three foundational tiers of the project comprise: the Presentation Tier, Application Tier, and Data Tier.

Applying this model ensures clear separation of responsibilities between user interface interactions, library business logic, and database management systems. Thanks to high inter-tier decoupling, our team can easily maintain or upgrade individual components independently (such as optimizing AI algorithms or updating the web interface) without disrupting core underlying operational processes.

5.1.1 Presentation Tier

Serving as the direct touchpoint for end-users, this tier is implemented as a robust Single Page Application (SPA) utilizing the React.js library and Vite build tool. The interface is tailored with strict Role-Based Access Control (RBAC) across three distinct user roles: Reader, Librarian, and Administrator (Admin).

All user interactions—including searching for documents, submitting checkout requests, chatting with the AI assistant, or executing librarian operations—are captured, asynchronously processed, and converted into standardized RESTful API requests before transmission to the central processing tier.

5.1.2 Application Tier

Functioning as the core orchestration hub, the Application Tier receives, processes, and executes complex business logic required for a smart library system. This tier operates primarily within a Node.js environment paired with the Express.js framework, while integrating a decoupled Python service (FastAPI) dedicated to high-performance AI workloads. To ensure modularity and scalability, the backend is partitioned into 7 functional modules matching system business domains:

- **Account and Access Management Module:** Protects system security via Role-Based Access Control (RBAC) combined with password hashing. The system employs JSON Web Tokens (JWT) to securely manage user sessions. Additionally, this module manages personal user profiles, OTP verification codes, and activity audit logs for readers and library staff.
- **Book Catalog Management Module:** Organizes hierarchical category structures (Category) and stores book metadata (Book). This sub-system assists librarians in automated cataloging by integrating third-party APIs (Open Library, Google Books API) for ISBN metadata fetching.

- **Document Circulation Management Module:** Handles the full lifecycle of physical item checkouts and returns. It tracks stack availability per copy (`PhysicalCopy`) via barcodes, automates reservation queues (`Reservation`), calculates online renewal eligibility, enforces fine rules (`Fine`) for overdue returns, and orchestrates inter-branch item transfers.
- **Search and Resource Exploitation Module:** Provides advanced keyword and semantic search filtering to optimize query performance across the book catalog.
- **Virtual Assistant and Recommendation Module:** Operates as a decoupled high-performance microservice leveraging Large Language Models (LLMs) via Google Generative AI (Gemini) APIs. Capabilities include: Natural language Semantic Search, AI document summarization, preference prediction algorithms for personalized recommendations, and a 24/7 Virtual Assistant Chatbot.
- **Notification Module:** Automates real-time user communications. The system listens for system events (due date reminders, fine assessments, hold availability) to dispatch multi-channel notifications via in-app alerts and automated emails using Nodemailer.
- **System Administration Module:** Provides administrative tools for system management, including dynamic system configuration key-values (`system_configs`), branch network management, real-time metric dashboards (`Dashboard Metrics`), automated database backups, and snapshot disaster recovery.

To maximize development velocity and type safety, the Application Tier utilizes **Prisma ORM** as the data access layer. Capitalizing on TypeScript's strict static type checking, Prisma catches query syntax errors during compile time. Prisma Migrate manages database schema versioning, ensuring seamless automated schema synchronization between development and production environments.

5.1.3 Data Tier

The data storage layer provides persistent, secure, and ACID-compliant storage for all system records. It comprises two specialized database stores:

- **Relational Database (PostgreSQL):** For library operations requiring strict financial transaction accuracy and concurrent consistency (e.g., decrementing physical inventory while generating loan tickets), **PostgreSQL** is selected for its robust ACID compliance. High-performance index structures ensure low latency under heavy query loads. Additionally, all sensitive state-mutating actions write automated entries to audit logs (`AuditLog`).
- **Vector Database (pgvector):** To power Retrieval-Augmented Generation (RAG) and semantic search in AI modules, the data tier incorporates vector storage mechanisms. Text summaries, book metadata, and digital document chunks converted into high-dimensional vector embeddings are stored and queried using Cosine Similarity directly in `pgvector`.

5.2 Database Design

5.2.1 Entity Descriptions

Entity Category	Entity Name	Description
User & Access Control	users	Stores user account profiles including readers, librarians, and system administrators.
	roles	Manages system roles and authorization permissions (Admin, Librarian, Reader).
	otp_tokens	Stores OTP tokens for login authentication, password resets, and MFA.
	notification_settings	Stores user notification preferences across Email, SMS, and In-app channels.
	notifications	Manages system notifications dispatched to users (due date alerts, hold notices).
Library Resources	books	Stores metadata records for books, journals, and digital academic materials.
	categories	Manages hierarchical category classifications for subjects and disciplines.
	physical_copies	Tracks physical copy instances, circulation statuses, and shelf locations.
	borrow_records	Tracks loan and return lifecycles for physical library items.
	reservations	Manages item hold reservation queues when copies are out of stock.
	finances	Manages overdue fines and penalty fee assessments.
	branches	Stores branch library profiles and campus location metadata.
	branch_transfers	Manages physical copy transfers between branch libraries.
Interaction & Reviews	reviews	Stores user ratings, reviews, and feedback on library materials.
	chat_history	Stores conversation transcripts between users and the AI Chatbot assistant.
System & Administration	audit_logs	Records administrative and security audit events for system compliance.
	system_configs	Stores global dynamic configuration parameters for system operation.

Table 30: System Entities and Database Definitions

5.2.2 Enhanced Entity-Relationship Diagram (EERD)

Based on the Enhanced Entity-Relationship Diagram (EERD), the system operational workflows are partitioned across three primary user roles:

5.2.2.1 Reader Role End-users interacting with the platform to search, borrow, and read physical and digital items:

- **Account Management & Security:** Executes multi-factor authentication via OTP tokens (otp_tokens); configures notification channels (Email, SMS, In-app) in notification_settings.
- **Physical Borrowing & Catalog Search:** Searches and browses categories (categories). Borrows physical materials with loan records tracked in borrow_records.
- **Physical Borrowing & Returns:** Submits loan requests and tracks borrowing lifecycles in borrow_records tied to specific copies (physical_copies).
- **Reservation Queue Management:** When items are out of stock, places hold reservations (reservations) with automated queue positioning via queuePosition.
- **Feedback & Ratings:** Submits star ratings and reviews (reviews); chats with the AI Virtual Assistant (chat_history); views and pays fine ledgers (finances).

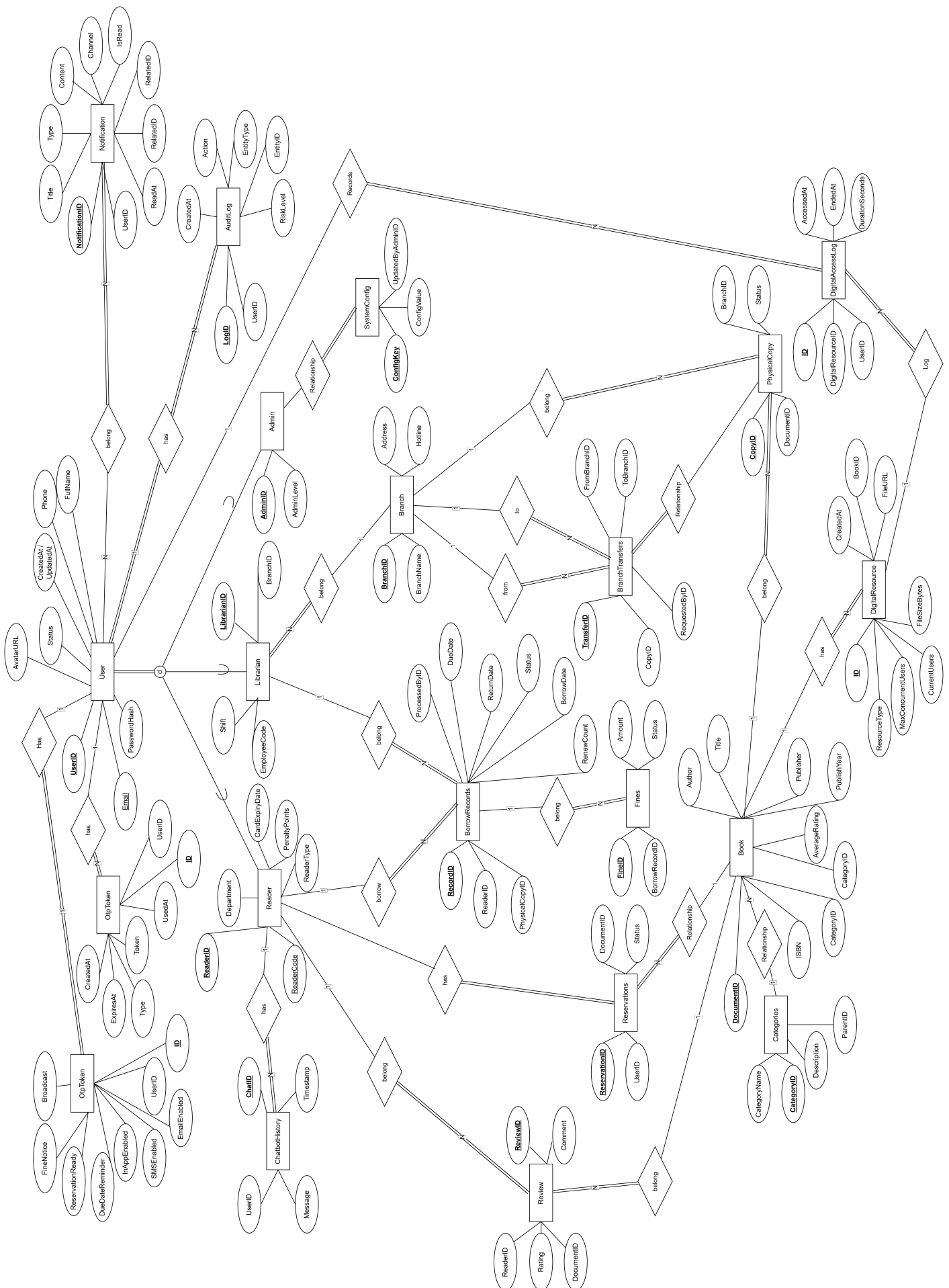


Figure 14: EERD Diagram of the Online Library Management System

5.2.2.2 Librarian Role

Operational staff managing physical stacks, catalog entries, and counter circulation:

- **Physical Stack Management:** Manages catalog records (`books`), copy availability (`status`), physical condition (`condition`), and shelf location (`location`) in `physical_copies`.
- **Circulation Desk Processing:** Scans barcodes (`barcode`) and reader cards (`readerCode`), updating `borrow_records`.
- **Reservation Queue Orchestration:** Monitors reservation queues (`reservations`), updating copy statuses upon return and alerting next patrons.
- **Inter-Branch Logistics:** Approves and tracks item transfer tickets between branch libraries via `branch_transfers`.
- **Fine Assessment & Waivers:** Reviews overdue penalties in `finest`, possessing authorization to grant fine waivers (`waivedById`, `waivedReason`).

5.2.2.3 Administrator Role System administrators holding global configuration, user security, and audit privileges:

- **User Management & RBAC:** Activates, locks, or suspends accounts (`users`), managing failed login limits (`failedLoginCount`) and role permissions (`roles`).
- **Infrastructure Administration:** Manages branch network configurations (`branches`) and assigns branch managers (`managerId`).
- **Global Parameters:** Configures operational system policies via `system_configs` (daily fine rates, loan limits, max file upload sizes).
- **System Audit & Compliance:** Monitors audit logs (`audit_logs`) for data security and regulatory compliance.

5.2.3 Database Schema

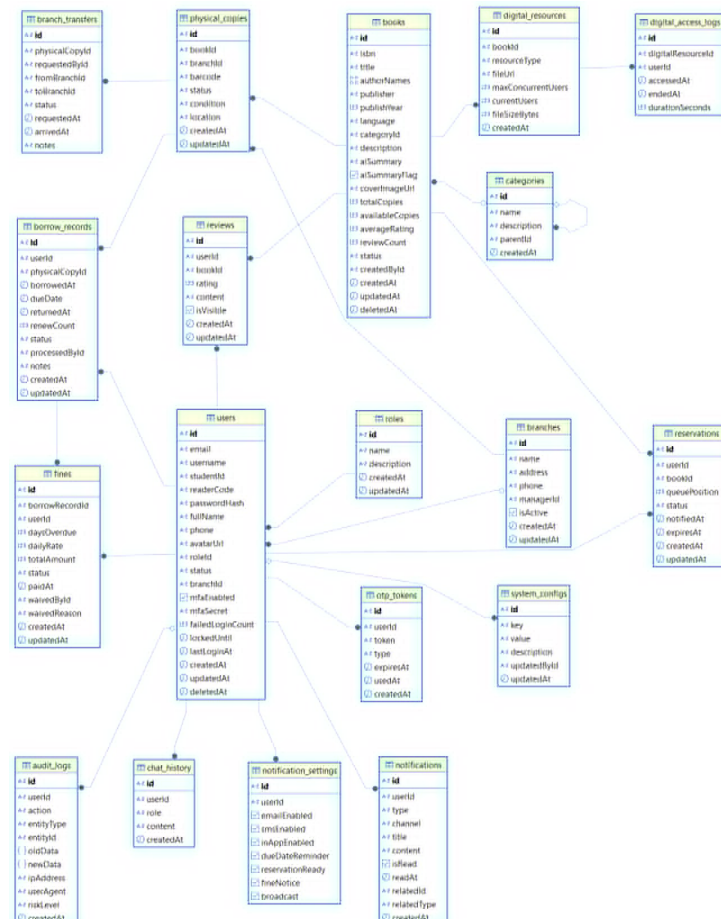


Figure 15: *Relational Database Schema for the Online Library Management System*

The schema above represents the relational structure implemented on PostgreSQL.

User accounts are centralized in the `users` table, linked to roles via foreign key `roleId` for RBAC permission checks. Security flags such as `status`, `failedLoginCount`, and `lockedUntil` enforce login security constraints.

Remaining tables map directly to the entity definitions and EERD relationships detailed above.

5.3 Class Diagram

5.3.1 Account and Access Management

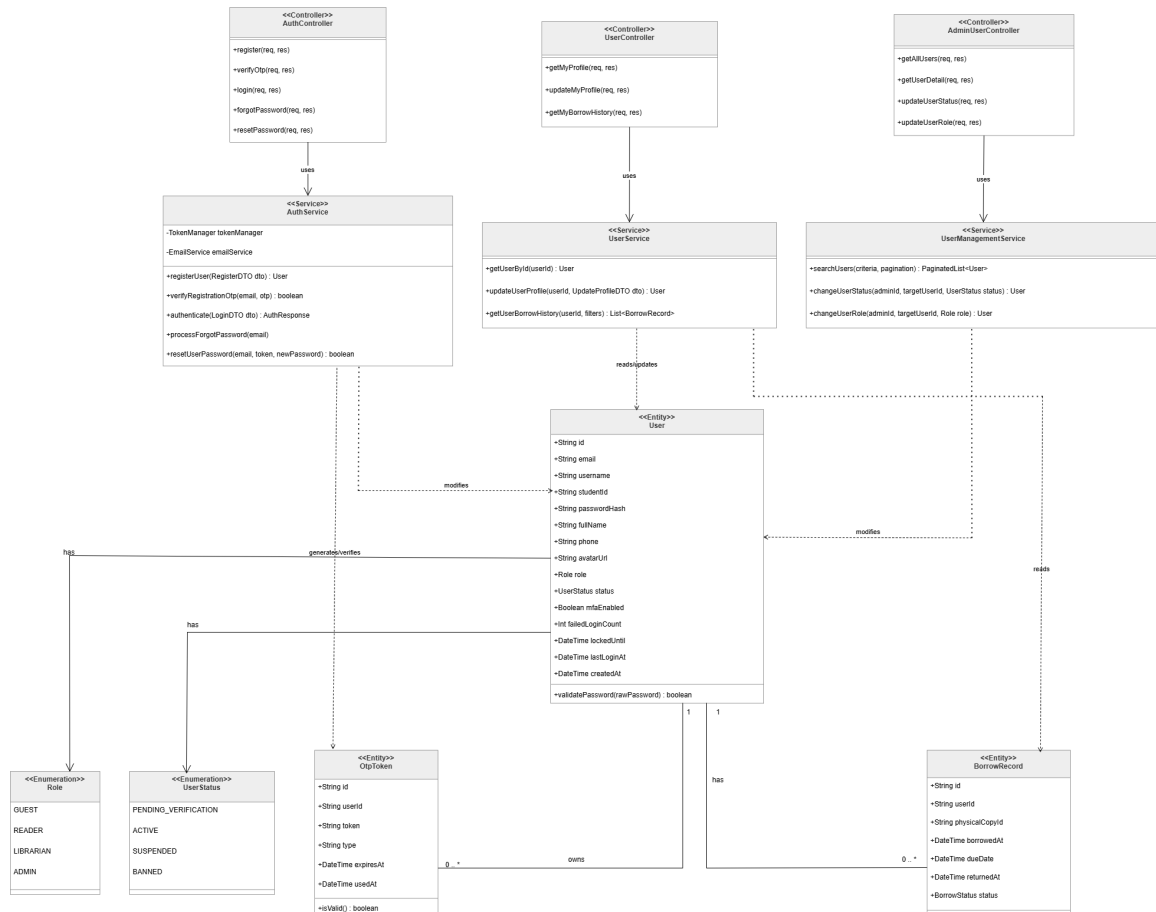


Figure 16: Class Diagram for Account and Access Management

The **Account and Access Management** class diagram models object structures, attributes, and methods for authentication and profile management.

The central entity is `User`, containing attributes `id`, `email`, `username`, `fullName`, `phone`, `avatarUrl`, `studentId`, alongside security attributes `passwordHash`, `mfaEnabled`, `failedLoginCount`, `lockedUntil`, `lastLoginAt`, and `createdAt`. The core method `validatePassword(rawPassword)` performs credential verification.

`User` holds associations with `Role` (`READER`, `LIBRARIAN`, `ADMIN`) and `UserStatus` (`PENDING_VERIFICATION`, `ACTIVE`, `SUSPENDED`, `BANNED`). `User` associates `1` – `0..*` with `OtpToken` (method `isValid()`) and `1` – `0..*` with `BorrowRecord`.

Architecturally, `AuthController` relies on `AuthService` (`registerUser`, `verifyRegistrationOtp`, `authenticate`, `processForgotPassword`, `resetPasswordUser`). `UserController` relies on `UserService` (`getUserById`, `updateUserProfile`, `getUserBorrowHistory`). `AdminUserController` uses `UserManagementService` (`searchUsers`, `getUserDetail`, `changeUserStatus`, `changeUserRole`).

5.3.2 Book Catalog Management

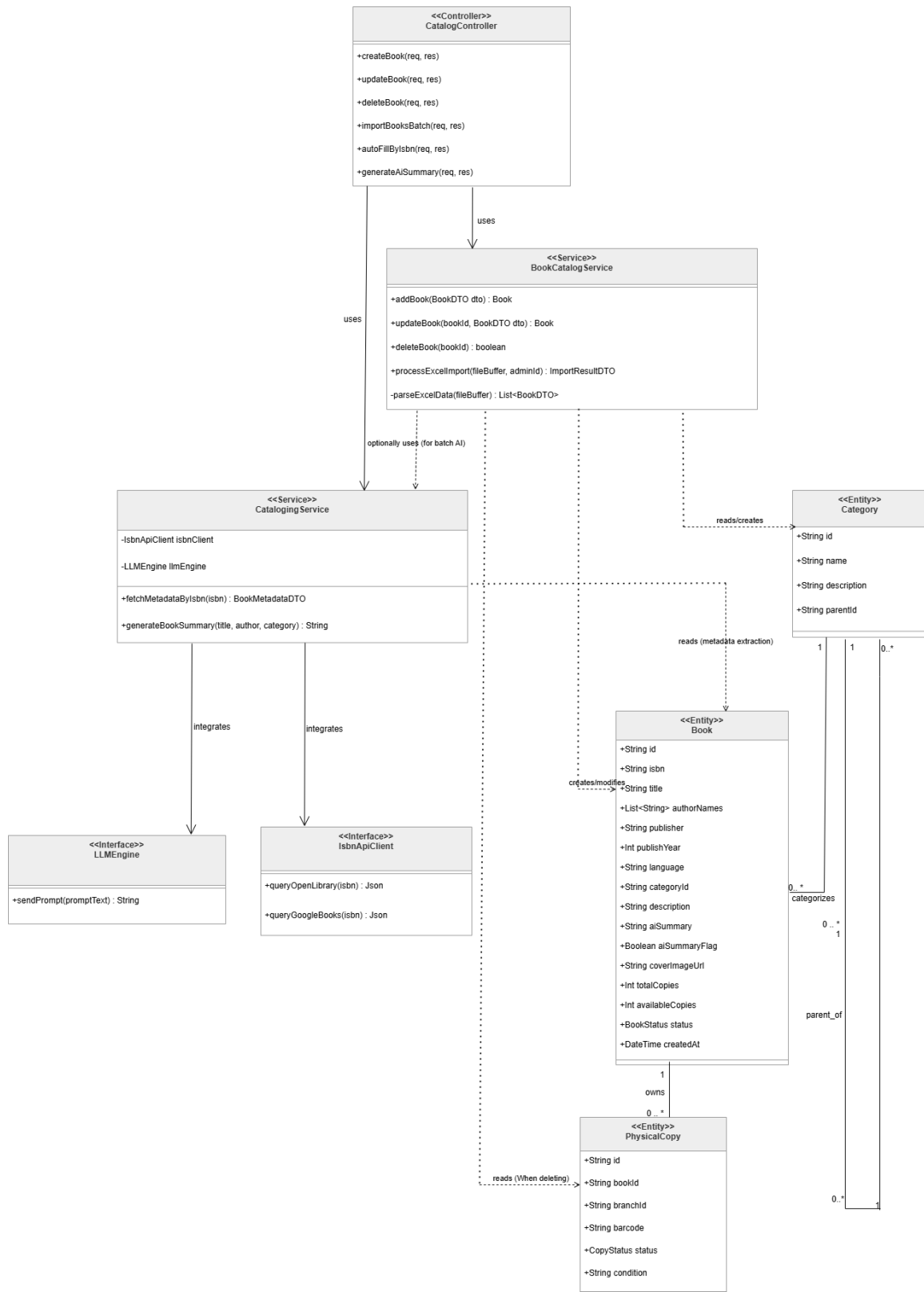


Figure 17: Class Diagram for Book Catalog Management

Core data structures include Category, Book, and PhysicalCopy. Category features a self-referential hierarchy (1 → 0..*) via parentId and associates 1 → 0..* with Book.

Book holds catalog metadata (isbn, title, authorNames, publisher, publishYear, language, totalCopies, availableCopies, aiSummary) and associates 1 → 0..* with PhysicalCopy (barcode, CopyStatus status, condition).

CatalogController delegates to BookCatalogService (addBook, updateBook, deleteBook, autoFillByIsbn, generateAiSummary, processExcelImport). BookCatalogService interacts with CatalogingService, which wraps IsbnApiClient (queryOpenLibrary, queryGoogleBooks) and LLMEngine (generateBookSummary) for automated cataloging.

5.3.3 Document Circulation Management

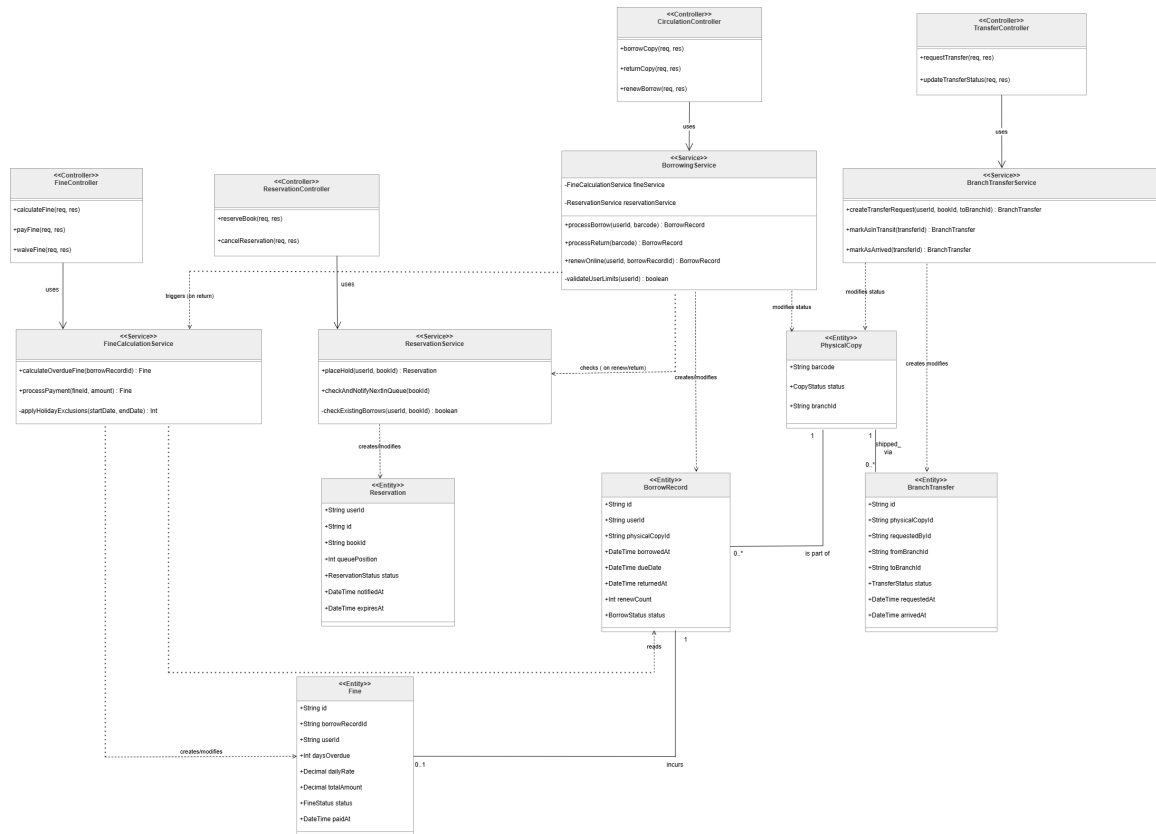


Figure 18: Class Diagram for Document Circulation Management

The central entity BorrowRecord (borrowedAt, dueDate, returnedAt, renewCount, BorrowStatus status) associates with PhysicalCopy (1 – 0..*). Reservation (queuePosition, ReservationStatus status) and Fine (1 – 0..1 with BorrowRecord) handle hold queues and fine assessments. BranchTransfer tracks inter-branch item transfers.

CirculationController uses BorrowingService (processBorrow, processReturn, renewOnline). ReservationController uses ReservationService (placeHold, cancelReservation). FineController uses FineCalculationService (calculateOverdueFine, processPayment). TransferController uses BranchTransferService (createTransferRequest, markInTransit, markAsArrived).

5.3.4 Resource Search and Exploitation

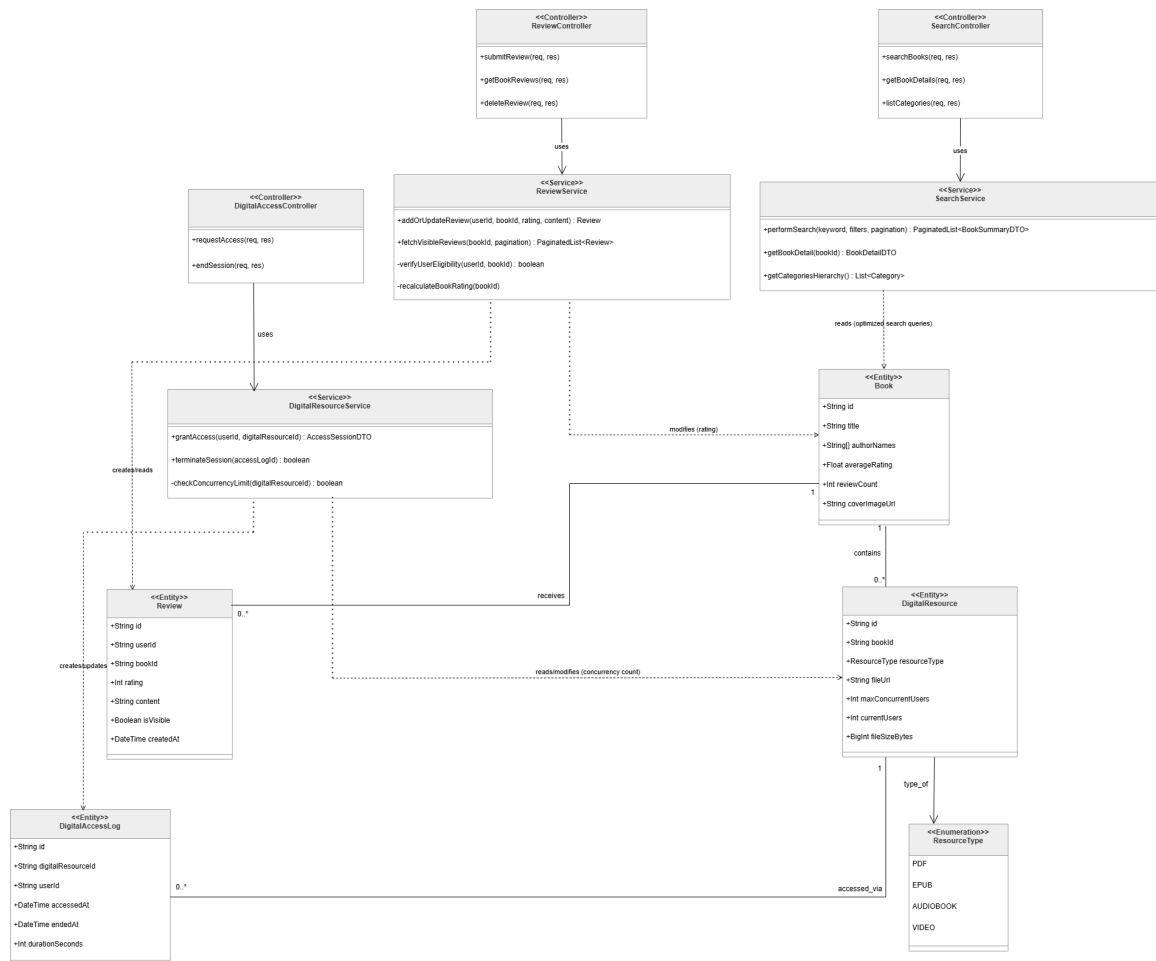


Figure 19: Class Diagram for Resource Search and Exploitation

Book associates 1 – 0..* with Review (0..* – 1 with Book) storing user ratings.

SearchController uses SearchService (performSearch, getBookDetail). ReviewController uses ReviewService (addOrUpdateReview, recalculateBookRating).

5.3.5 AI Virtual Assistant and Recommendations

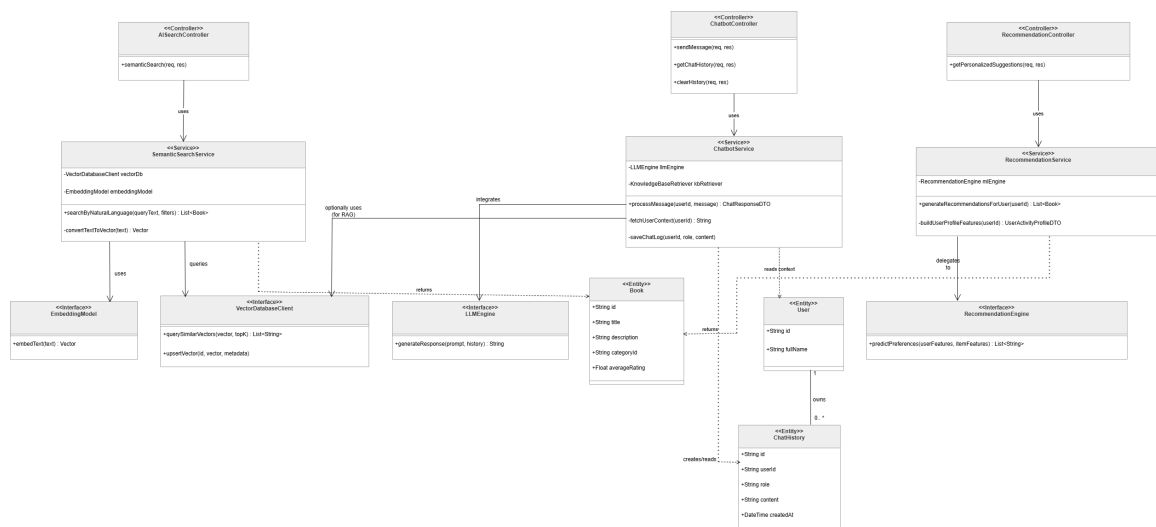


Figure 20: Class Diagram for AI Virtual Assistant and Recommendations

AISearchController uses SemanticSearchService (searchByNaturalLanguage) interfacing with EmbeddingModel and VectorDatabaseClient.

ChatbotController uses ChatbotService (sendMessage, getChatHistory), interfacing with LLMEngine and VectorDatabaseClient (RAG context), logging chats in ChatHistory.

RecommendationController uses RecommendationService (getPersonalizedSuggestions), leveraging RecommendationEngine (predictPreferences) to return tailored book lists.

5.3.6 Notifications and Communication

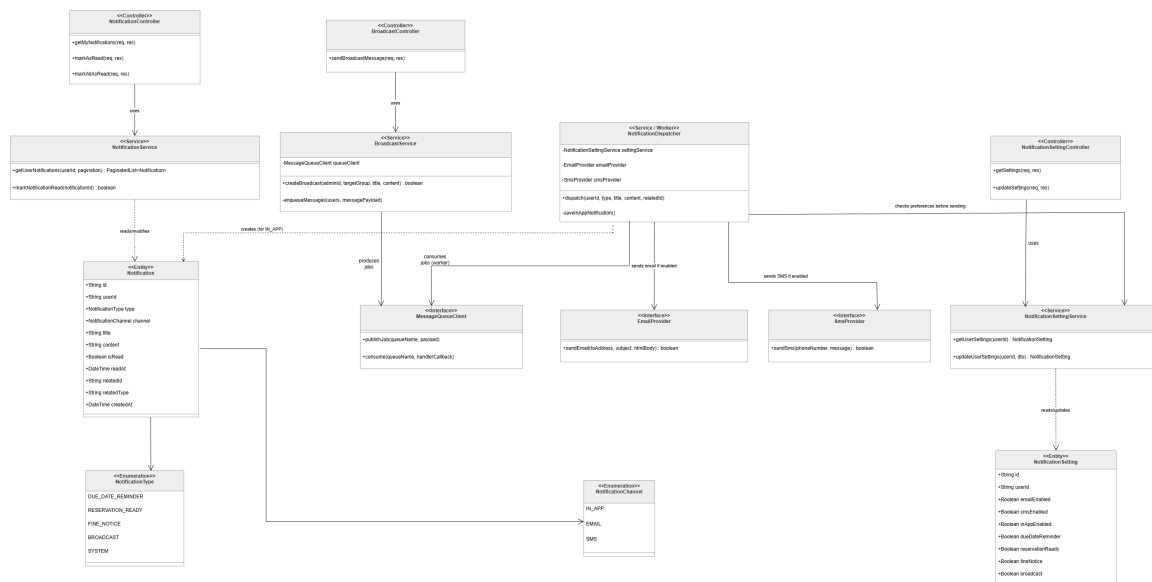


Figure 21: *Class Diagram for Notifications and Communication*

Entities comprise Notification (isRead, NotificationType, NotificationChannel) and NotificationSetting (emailEnabled, smsEnabled, inAppEnabled).

NotificationSettingController uses NotificationSettingService. NotificationController uses NotificationService. BroadcastController uses BroadcastService pushing jobs to MessageQueueClient. Async worker NotificationDispatcher consumes queue jobs and dispatches alerts via EmailProvider and SmsProvider.

5.3.7 System Administration

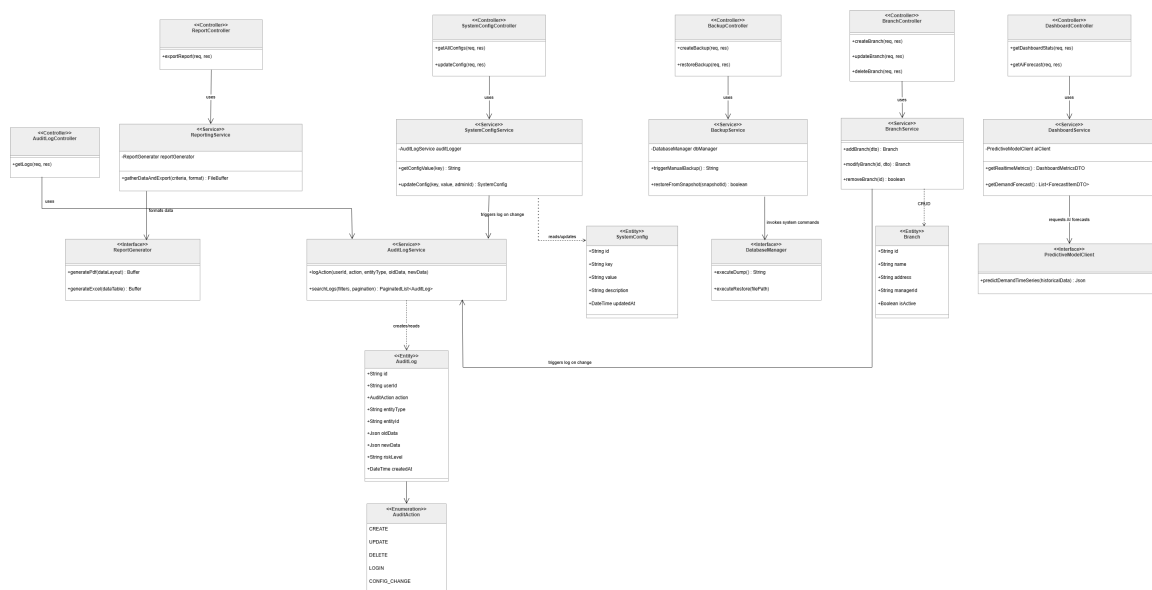


Figure 22: *Class Diagram for System Administration*

Core entities: AuditLog (action, oldData, newData, riskLevel), SystemConfig (key, value), and Branch (name, address, isActive).

AuditLogController uses AuditLogService. ReportController uses ReportingService (ReportGenerator). SystemConfigController uses SystemConfigService. BackupController uses BackupService (DatabaseManager). BranchController uses BranchService. DashboardController uses DashboardService interfacing with PredictiveModelClient for demand forecasting.

5.4 Activity Diagrams

5.4.1 Account Registration

The Account Registration activity diagram illustrates the end-to-end operational workflow when a new guest user creates an account in the system. This process ensures that user inputs comply with system validation rules, prevents duplicate identity registration via unique email/phone checks, and guarantees secure user verification through Multi-Factor Authentication (OTP/Activation links) before activating the profile.

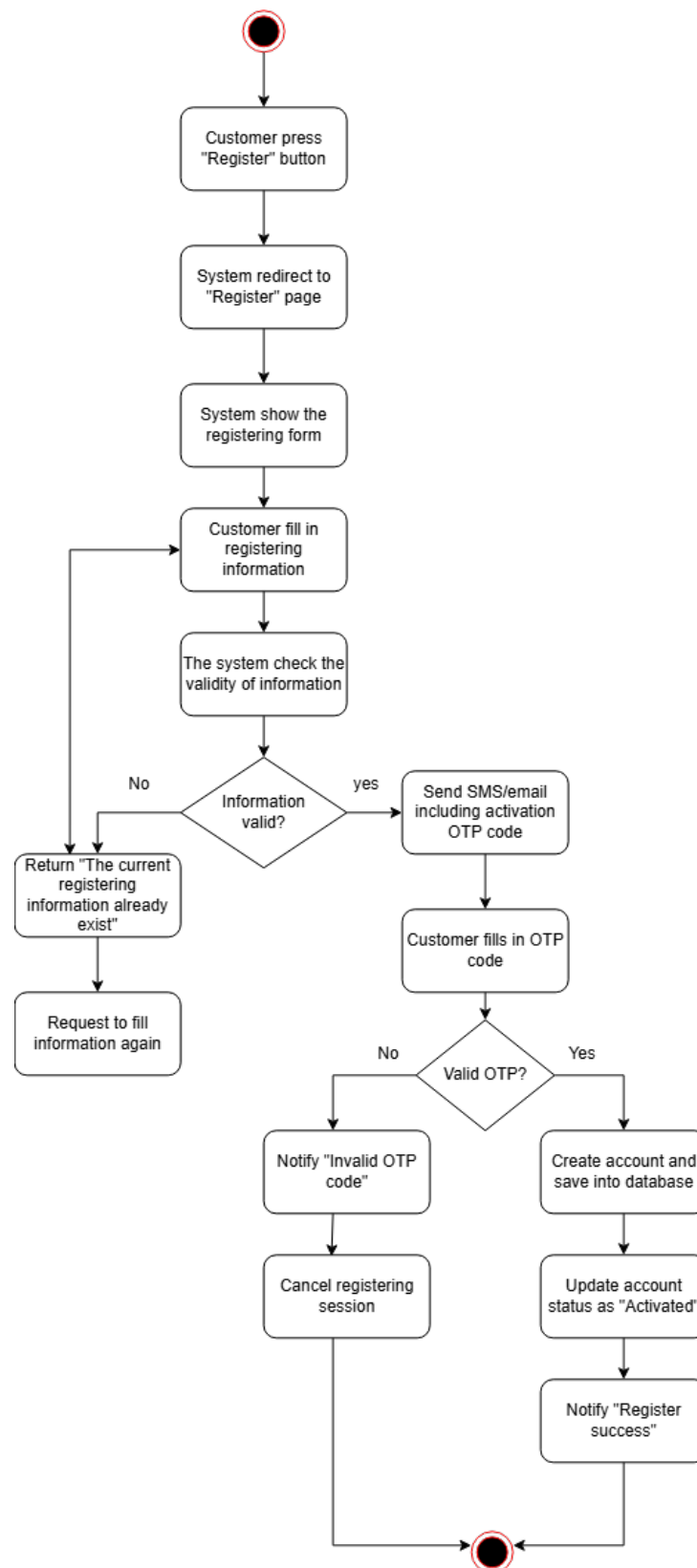


Figure 23: Activity Diagram for Account Registration

1. Guest accesses the account registration page.
2. System displays registration form.
3. Guest enters credentials (Full Name, Email/Phone, Password).
4. System validates input data syntax.
5. System checks if Email/Phone exists in database.

6. Valid Branch:

- If valid, system sends Email/SMS containing OTP code or activation link.
- Guest enters OTP code or clicks activation link.
- System validates OTP code or link token.
- If OTP/link is valid: System creates user account, saves to DB, sets status to "Activated", and alerts success.

7. Invalid Branch:

- If data is invalid or Email/Phone exists: System displays error alert and prompts re-entry.
- If OTP is invalid: System displays invalid OTP error and cancels registration session.

5.4.2 System Login

The System Login activity diagram depicts the authentication sequence for existing users accessing the system. It handles identity verification against stored credentials and incorporates adaptive security controls—such as mandatory Multi-Factor Authentication (MFA) and automated temporary account lockouts after multiple failed attempts—to protect the platform against brute-force attacks.

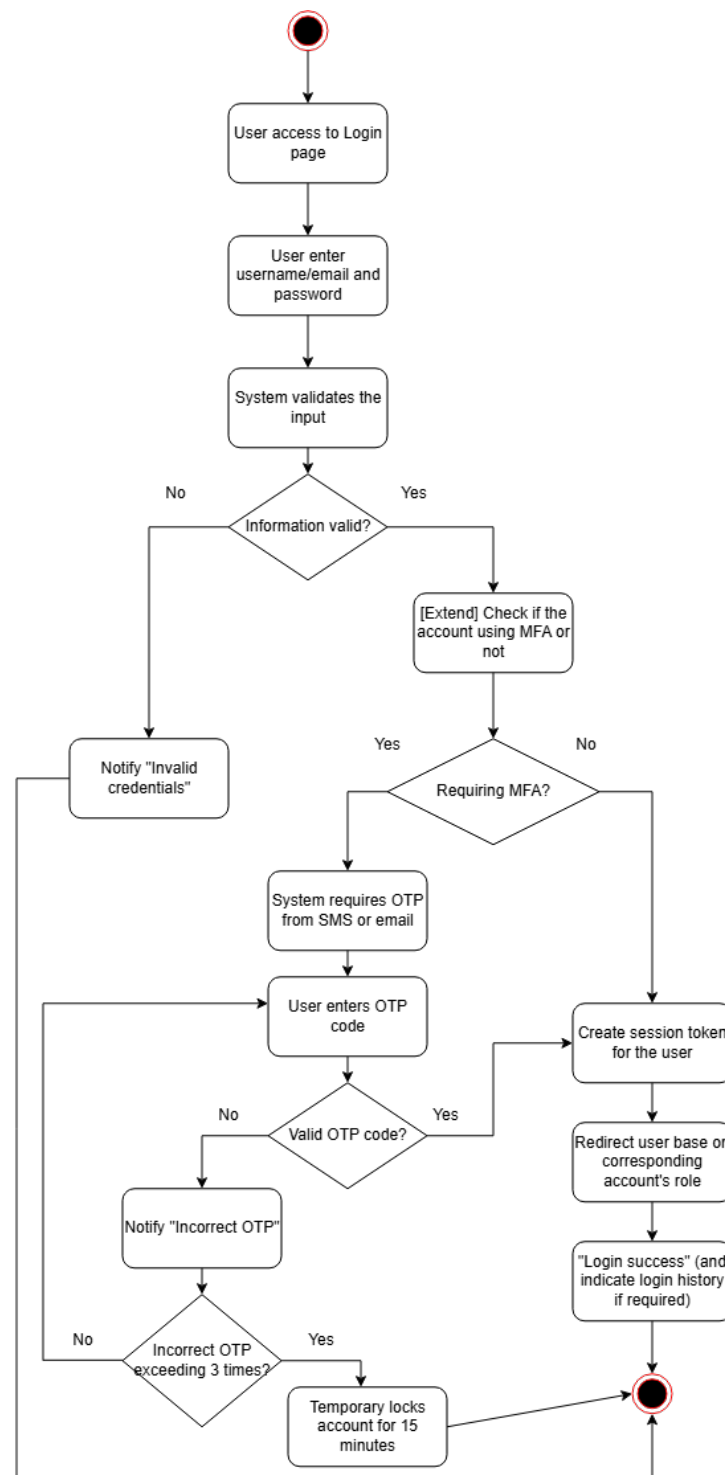


Figure 24: Activity Diagram for System Login

1. User accesses login view.
2. User enters Email/Username and Password.
3. System validates credentials against DB.
4. If credentials invalid: System displays error "Incorrect account or password".
5. If credentials valid: System checks if Multi-Factor Authentication (MFA) is required.
6. If MFA enabled: System requests OTP code; user enters OTP; system validates code.
7. If OTP invalid: Displays error "Incorrect OTP code". If failed attempts ≥ 3 , account is temporarily locked for 15 minutes.

8. If OTP valid (or MFA not required): Initializes session/token, redirects user to role dashboard, and displays success notification.

5.4.3 Password Reset

The Password Reset activity diagram outlines the self-service credential recovery mechanism for users who have forgotten their passwords. The workflow relies on secure, time-sensitive verification tokens sent directly to the user's registered email address, ensuring that password updates occur safely without administrator intervention.

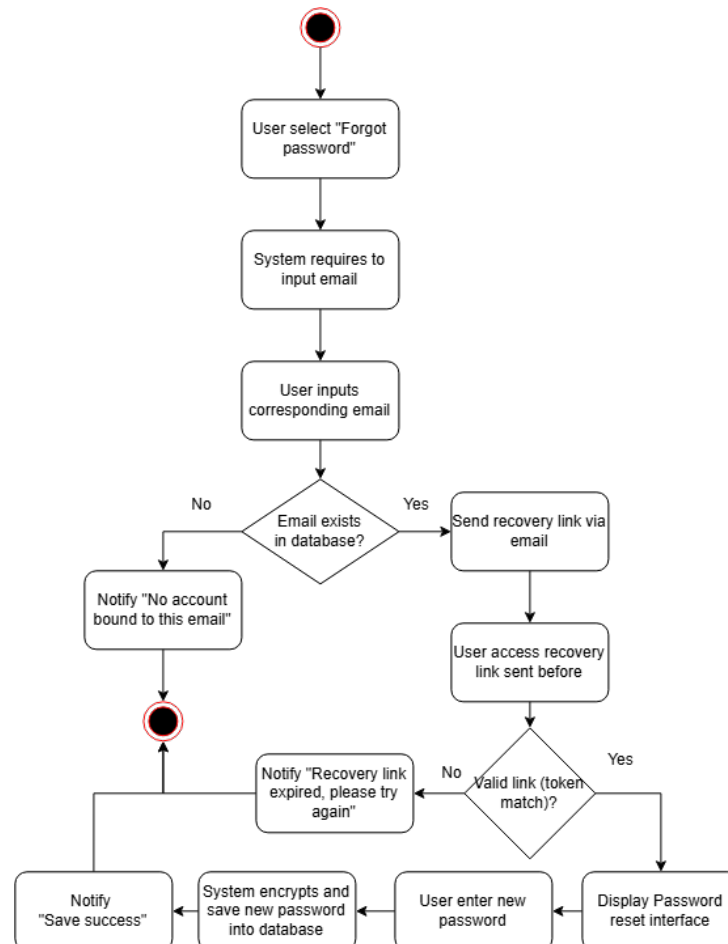


Figure 25: Activity Diagram for Password Reset

1. User selects "Forgot Password".
2. System prompts for Email.
3. User submits Email.
4. System verifies Email existence. If not found: Display "No account found with this email".
5. If found: System sends reset link via Email. User opens link; system verifies token validity.
6. If link expired/invalid: Display "Reset link expired" and prompt process restart.
7. If link valid: Display reset form; user enters new password; system hashes and updates password, alerting success.

5.4.4 View Loan/Return History

The View Loan/Return History activity diagram models how users query and inspect their personal borrowing records over time. It demonstrates the interactions involved in fetching structured loan logs from the persistent database and allowing users to dynamically filter results based on execution dates or current item statuses (e.g., Active, Returned, Overdue).

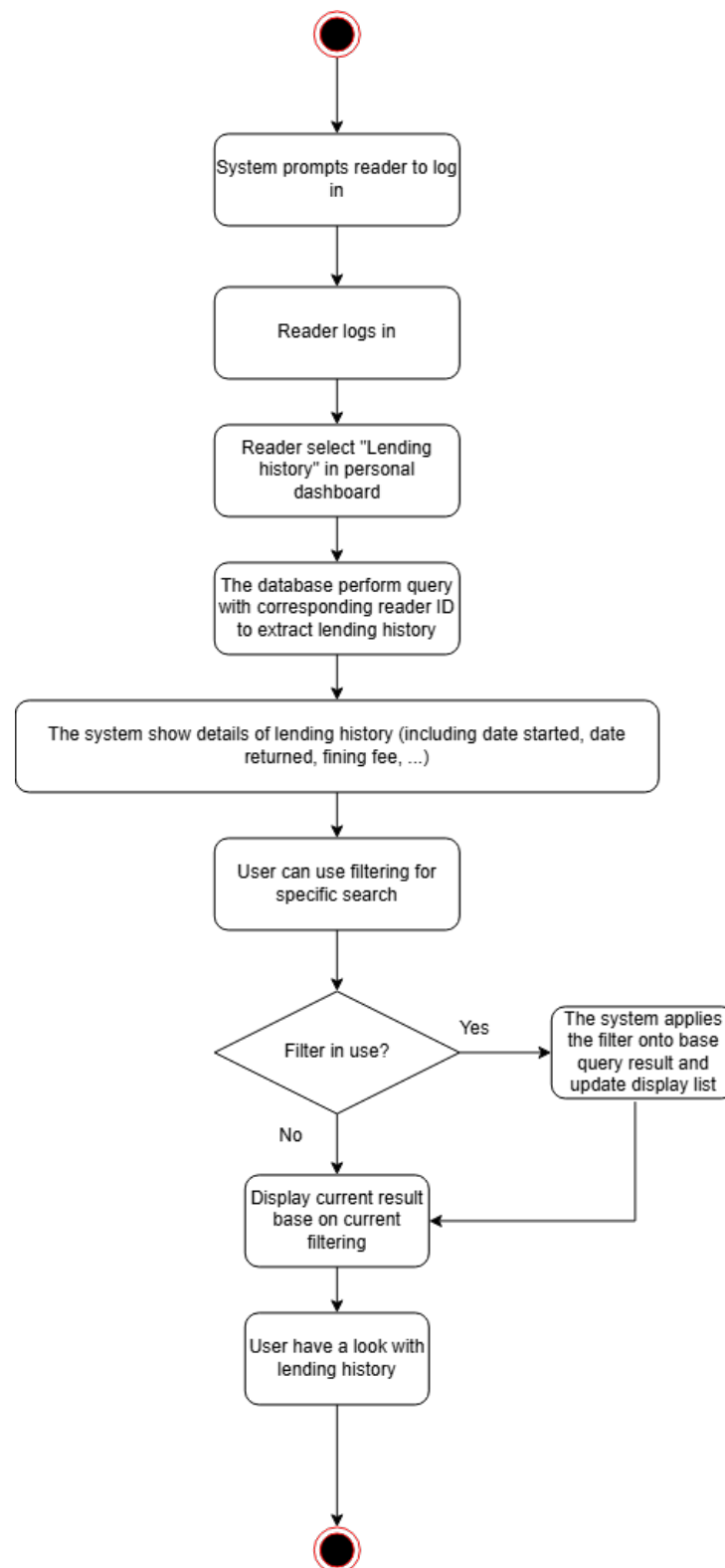


Figure 26: Activity Diagram for View Loan/Return History

1. User logs into system.
2. User opens "Borrowing History" on dashboard.
3. System queries DB for loan records matching User ID.

4. System renders loan history table (Title, Borrow Date, Due Date, Return Date, Status, Fines).
5. User can apply filters by date range or status.
6. System refreshes table view based on applied filter parameters.

5.4.5 User Account Management

The User Account Management activity diagram presents the administrative control workflow for managing user accounts system-wide. Authorized System Administrators can search, review, create, edit, lock/unlock, or adjust role permissions (Role-Based Access Control) for any user profile while maintaining overall access compliance.

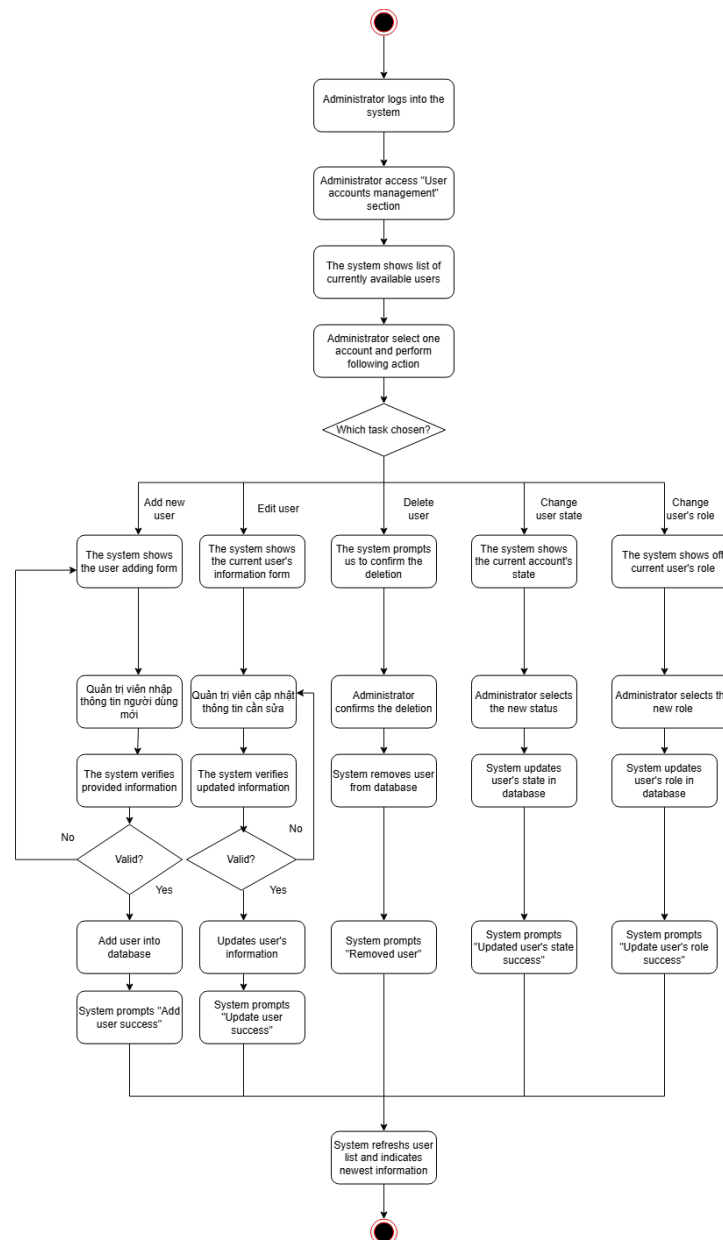


Figure 27: Activity Diagram for User Account Management

1. Admin logs in and accesses "User Management".
2. System renders user list (ID, Name, Email, Role, Status).
3. Admin selects a target user and an operation (Add User, Edit Profile, Delete User, Toggle Status, Change Role).
4. Executes selected workflow; system validates input data and updates DB, displaying success confirmation upon completion.

5.4.6 Add / Edit / Delete Catalog Record

This activity diagram describes the cataloging management actions performed by Librarians to keep the library database accurate. It handles single-item entries, updates, and soft deletions, including validation checks to prevent the deletion of items currently actively checked out by patrons.

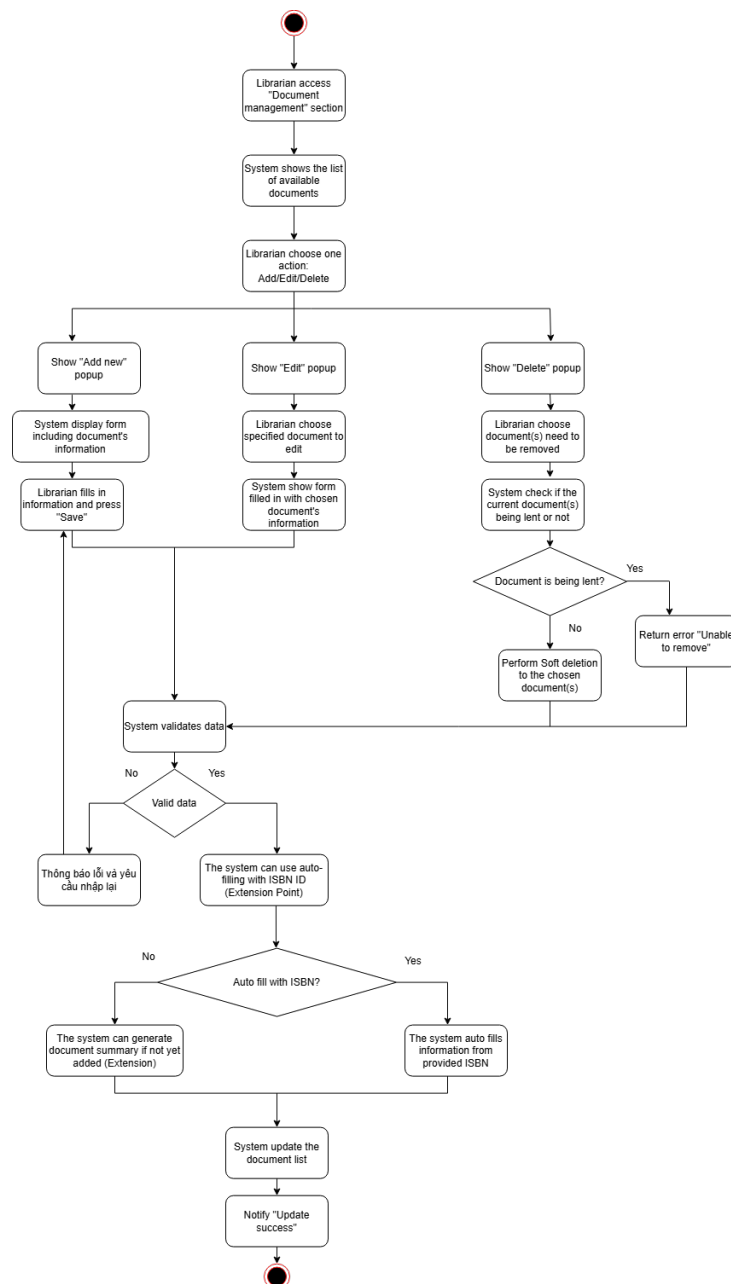


Figure 28: Activity Diagram for Add / Edit / Delete Catalog Record

1. Librarian opens "Catalog Management".
2. Librarian selects action: Add, Edit, or Delete.
3. If Delete: System verifies active loans. If item is checked out, deletion is blocked ("Cannot delete: Item checked out"). Otherwise, soft delete is performed.
4. If Add/Edit: Form is rendered; optional ISBN auto-cataloging (UC-CAT-03) or AI summary generation (UC-CAT-04) can be triggered.
5. System validates fields, persists updates, and alerts success.

5.4.7 Batch Catalog Data Ingestion

The Batch Catalog Data Ingestion activity diagram details the process of importing bulk book catalog data into the system using external files (CSV/Excel). The workflow highlights parsing, pre-import validation, error logging for malformed records, and database transaction execution to facilitate rapid catalog updates.

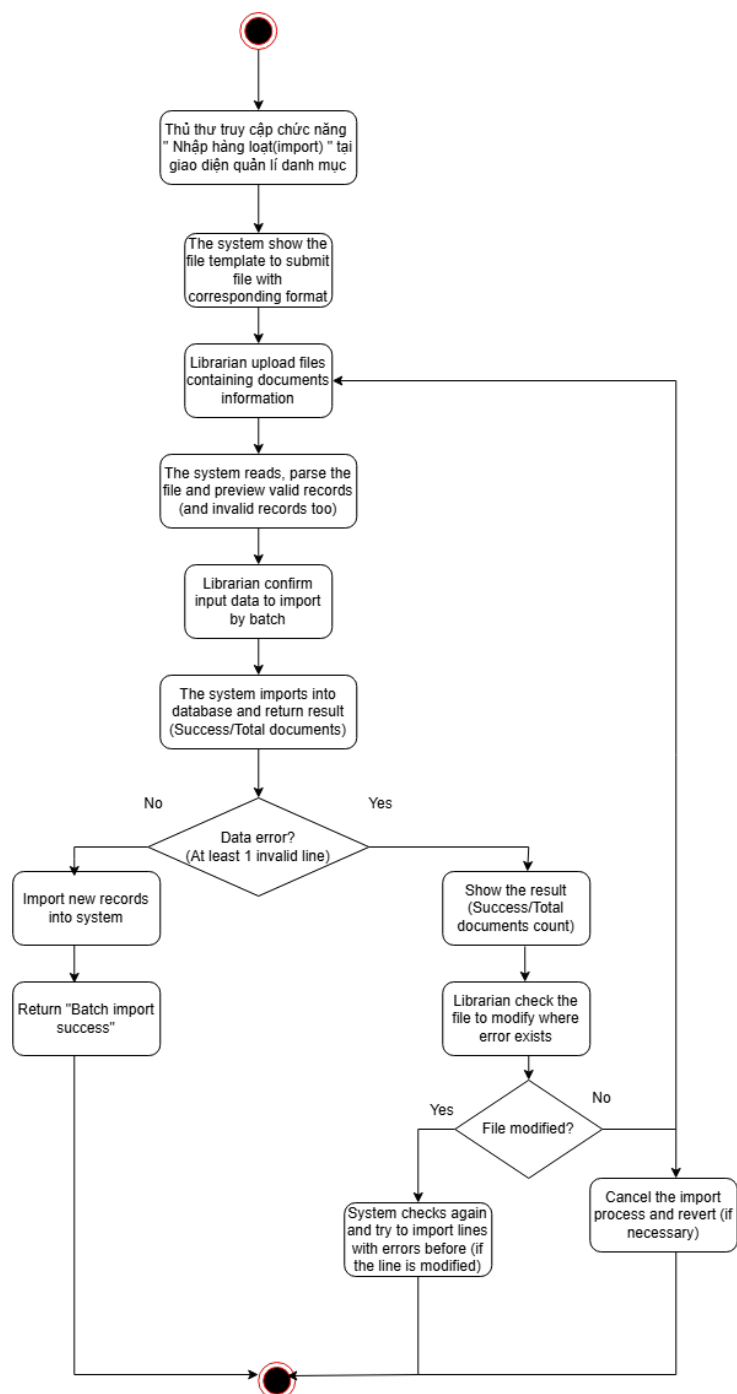


Figure 29: Activity Diagram for Batch Catalog Data Ingestion

1. Librarian selects "Batch Import".
2. Uploads catalog data file (CSV/Excel).
3. System parses file and displays preview table with valid/invalid record counts.
4. Librarian confirms import execution.
5. System writes valid rows to DB and reports results. If errors exist, librarian can fix file and re-upload or cancel operation.

5.4.8 Automated Cataloging via ISBN

The Automated Cataloging via ISBN activity diagram depicts how the system integrates with external metadata services (Google Books API, Open Library API). Librarians provide an ISBN code, and the system automatically queries external APIs to populate book metadata fields, reducing manual data entry overhead.

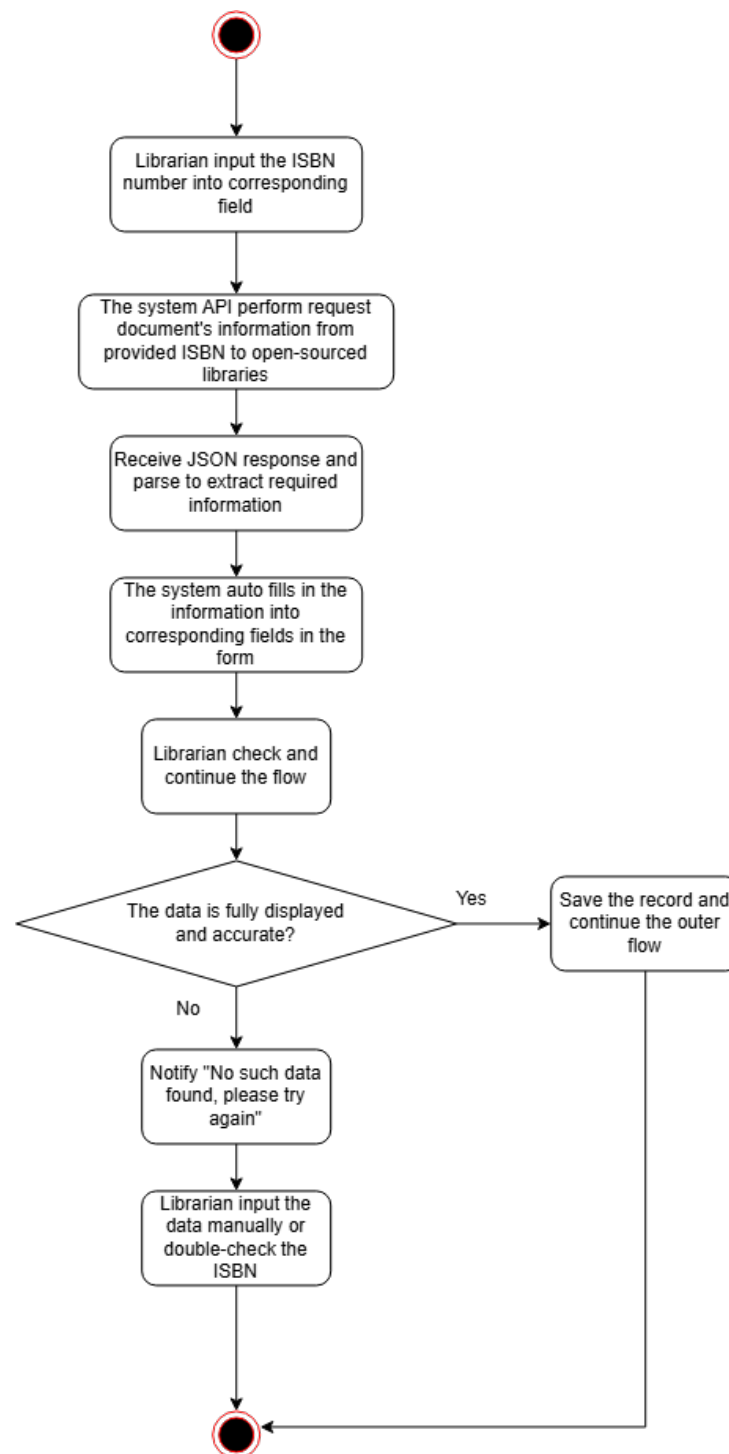


Figure 30: Activity Diagram for Automated Cataloging via ISBN

1. Librarian enters ISBN.
2. System sends API request to external sources (Google Books, Open Library).

3. Parses JSON response and populates form fields.
4. Librarian reviews and saves catalog record. If ISBN is not found, librarian switches to manual data entry.

5.4.9 Physical Material Checkout

The Physical Material Checkout activity diagram specifies the desk circulation process when a reader borrows physical books. It captures barcode scanning operations, verifies reader eligibility (account status, unpaid fine limits, max loan allowances), updates item copy availability, and registers checkout records.

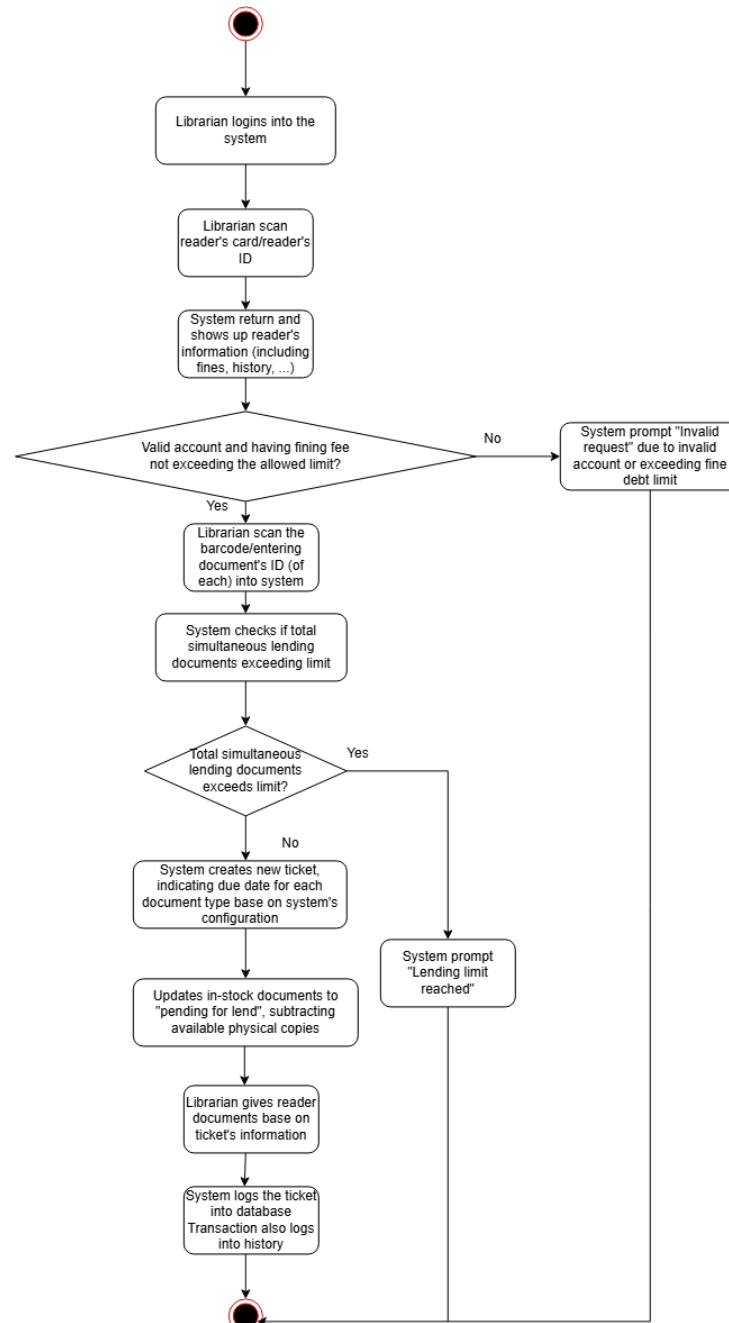


Figure 31: Activity Diagram for Physical Material Checkout

1. Librarian logs in and scans Reader card barcode.
2. System verifies reader account status and fine balance.
3. If eligible, librarian scans item barcodes.
4. System checks loan limits per role and hold queue restrictions.
5. Creates loan record, calculates due date, updates copy status to "Checked Out", and decrements inventory count.

5.4.10 Material Return

The Material Return activity diagram details the operational flow executed when physical books are returned to the library desk. The system closes the active borrow record, evaluates potential overdue days to initiate fine calculation workflows if necessary, and inspects reservation queues to transition the copy state appropriately.

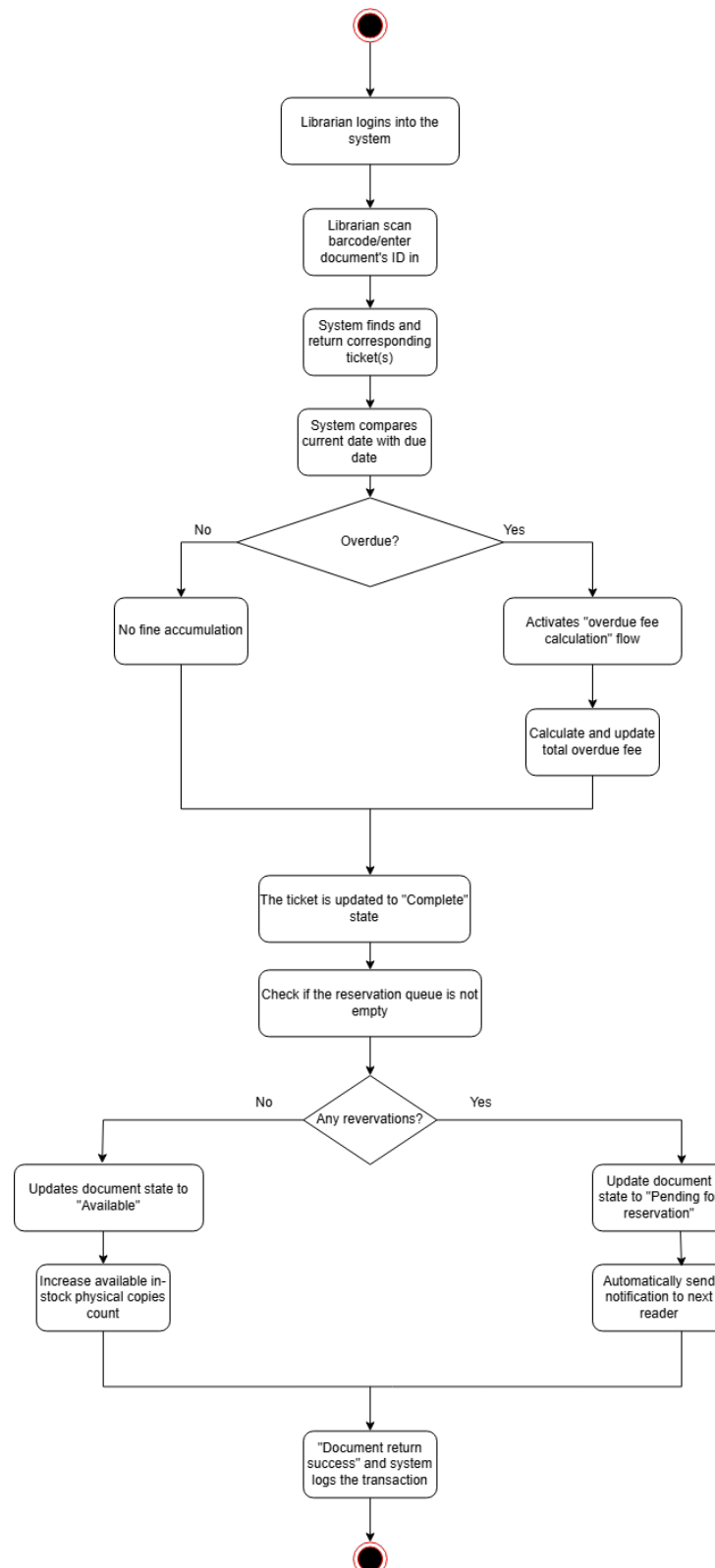


Figure 32: Activity Diagram for Material Return

1. Librarian scans returned item barcode.
2. System retrieves loan record and checks due date.

3. If overdue, triggers fine assessment workflow (UC-CIR-03).
4. Sets loan status to "Completed".
5. Checks reservation queue: if hold exists, sets copy to "Awaiting Pickup" and alerts next user; if no hold, sets copy to "Available".

5.4.11 Calculate and Assess Overdue Fines

The Calculate and Assess Overdue Fines activity diagram models the financial penalty workflow for items returned after their due dates. The system automatically computes overdue durations (excluding official library holidays), applies daily penalty rates, and posts the resulting fine balance to the reader's account or processes immediate payment receipts.

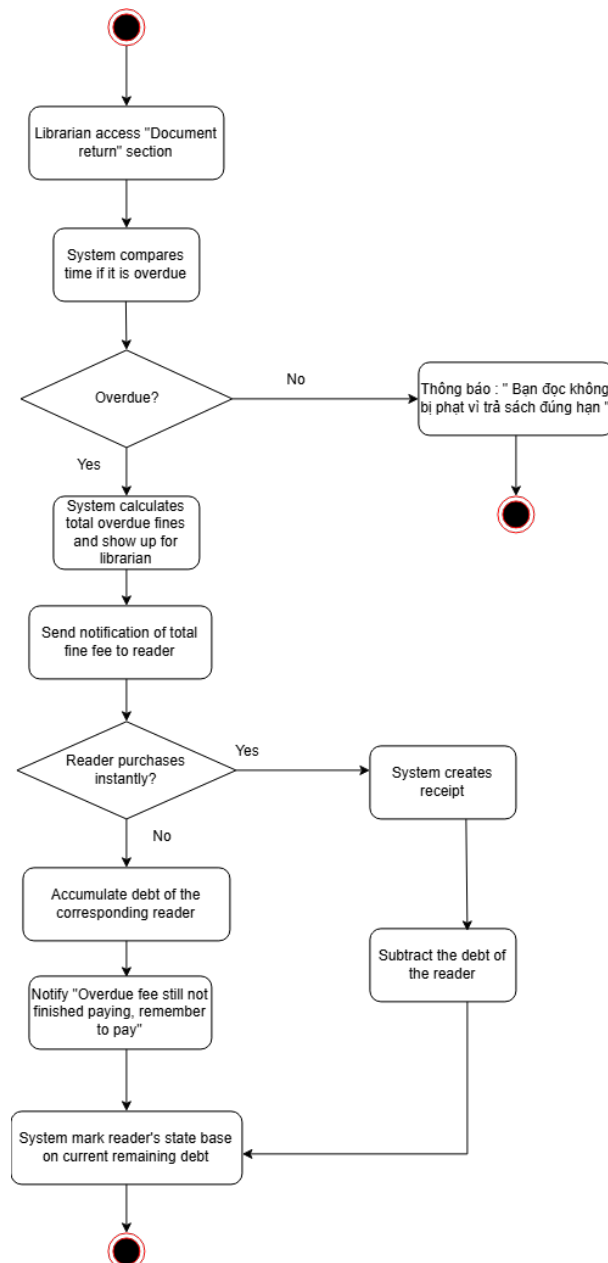


Figure 33: Activity Diagram for Calculate and Assess Overdue Fines

1. System computes overdue days (Return Date - Due Date), excluding library holidays.
2. Multiplies overdue days by daily fine rate.
3. Renders fine calculation. If reader pays immediately, generates receipt and sets ledger balance to 0; otherwise posts fine balance to reader account ledger.

5.4.12 Online Loan Renewal

The Online Loan Renewal activity diagram presents the self-service capability that allows readers to extend their borrowing period over the web. The system validates renewal constraints—such as ensuring maximum allowed renewals haven't been exceeded and checking that no other patron has placed a hold reservation on the item.

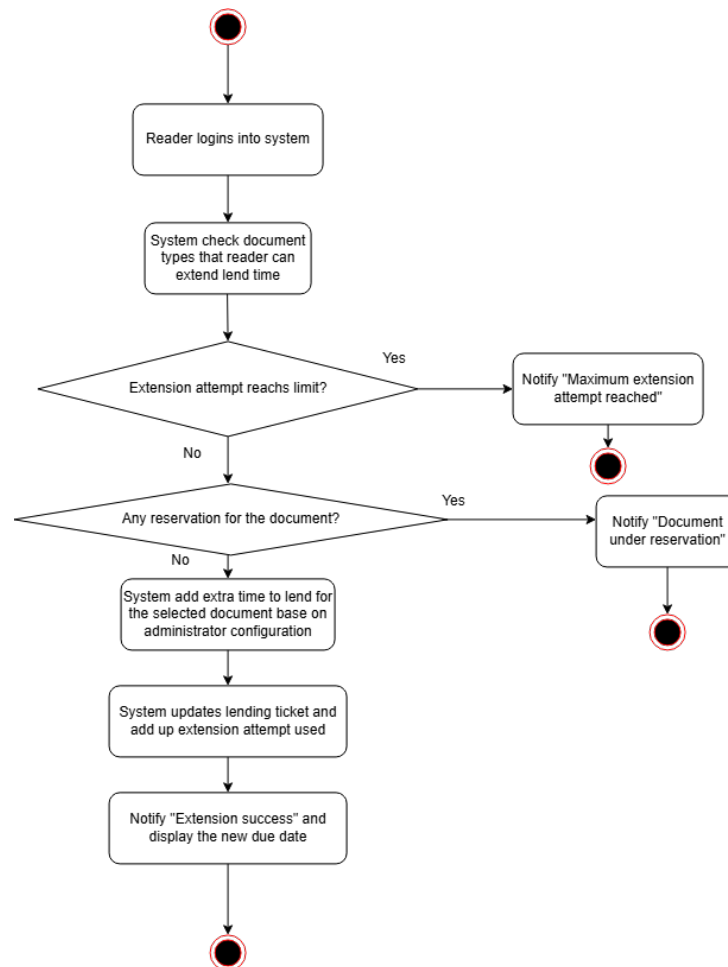


Figure 34: Activity Diagram for Online Loan Renewal

1. Reader opens active loans and clicks "Renew".
2. System verifies: maximum renewal count not reached and no hold requests exist.
3. If eligible, extends due date, increments renewal count, and displays new due date.

5.4.13 Place Item Hold

The Place Item Hold activity diagram presents the reservation process when a requested material is currently out of stock. The system verifies that the user does not currently hold an active copy of the item and assigns them a sequential queue position for automated pickup notification when a copy is returned.

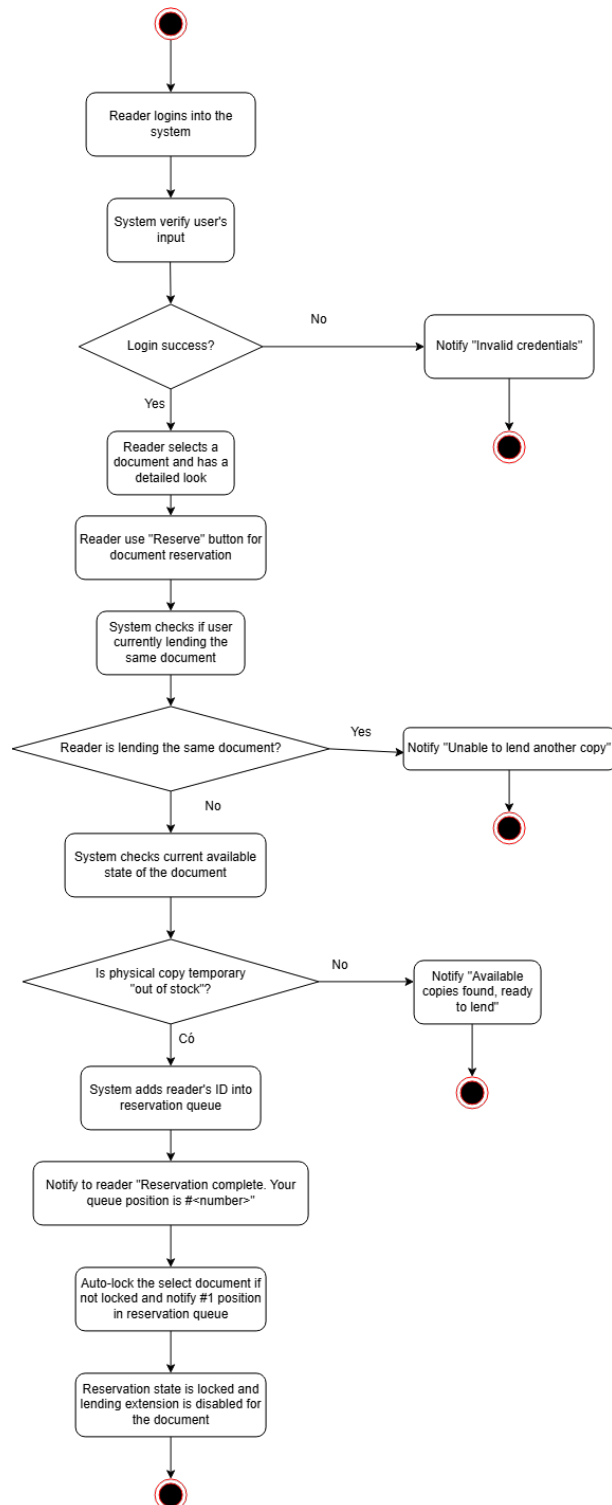


Figure 35: Activity Diagram for Place Item Hold

1. Reader opens item page and clicks "Place Hold".
2. System verifies reader does not currently hold active loan for item and item is out of stock.
3. System appends reader to reservation queue and displays queue position.

5.4.14 Document Catalog Search

The Document Catalog Search activity diagram models how users explore the library catalog. It depicts keyword query submission, indexed database querying, paginated search result rendering, and navigation to detailed document view pages.

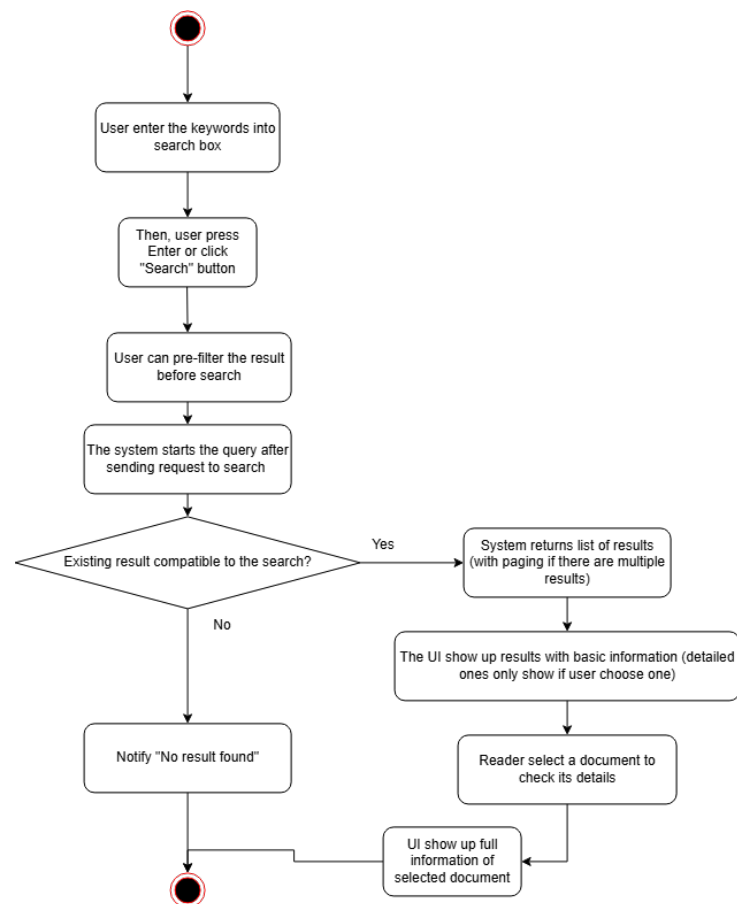


Figure 36: Activity Diagram for Document Catalog Search

1. Reader enters query keywords and selects search criteria.
2. System queries catalog DB indexes and returns paginated results.
3. Displays book cards. Reader clicks book card to view detail page.

5.5 Sequence Diagrams

5.5.1 Account Registration

The Account Registration sequence diagram illustrates the step-by-step object interactions between the Reader interface, Authentication Controller, User Service, Notification Dispatcher, and Database. It details how user inputs are validated, how OTP verification tokens are dispatched asynchronously, and how the database transaction completes account activation upon successful code validation.

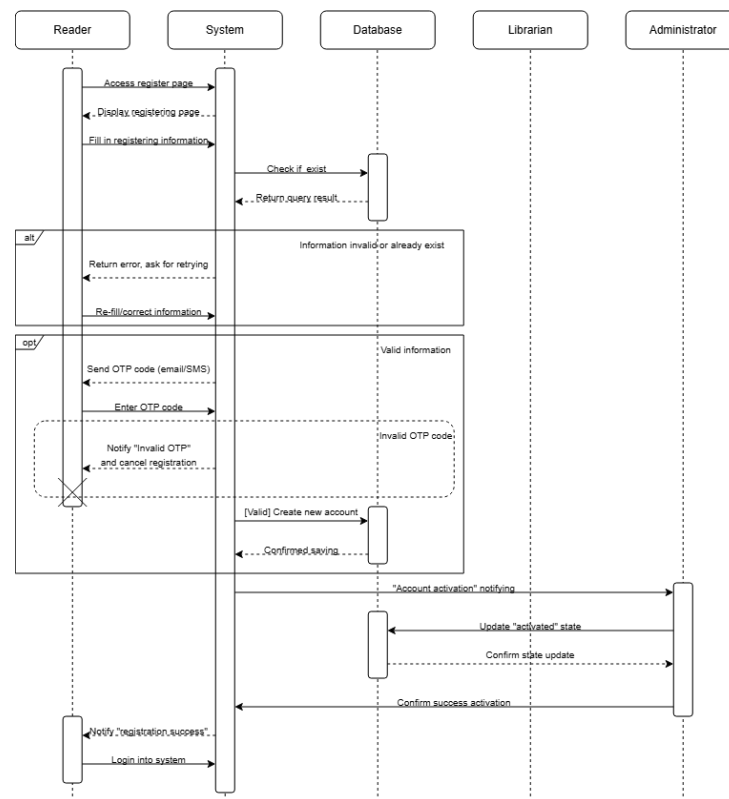


Figure 37: Sequence Diagram for Account Registration

1. Reader accesses registration view.
2. Submits details; system validates inputs and checks for duplicates.
3. System sends OTP code via email/SMS.
4. Reader enters OTP. If OTP valid, account is created and activated; otherwise registration is canceled.

5.5.2 Account Login

The Account Login sequence diagram outlines the interactive messages passed between the User Interface, Authentication Controller, Security Service, and Database during authentication. It captures credential validation, role and status verification, Multi-Factor Authentication (MFA) evaluation, and the final issuance of a JSON Web Token (JWT) session upon authorization.

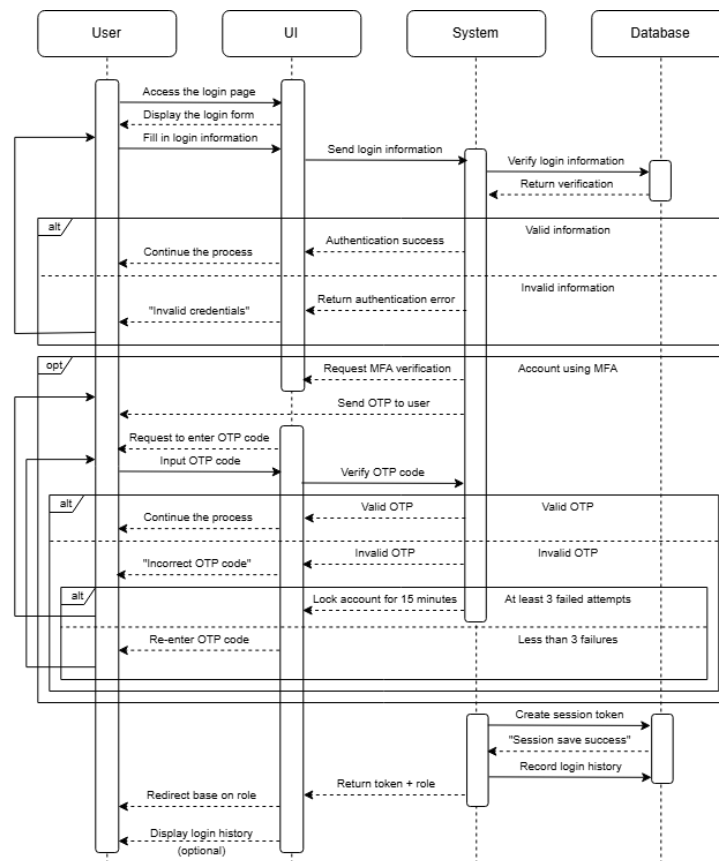


Figure 38: Sequence Diagram for Account Login

1. User submits credentials.
2. System matches credentials against DB.
3. If MFA enabled, requests OTP; upon successful OTP verification, issues session JWT token and redirects user to role dashboard.

5.5.3 Account Password Reset

The Account Password Reset sequence diagram specifies the system interactions during a self-service credential recovery flow. It demonstrates how the system validates user identity via email lookup, generates temporary reset tokens, dispatches secure email links, and executes hashed password updates upon token verification.

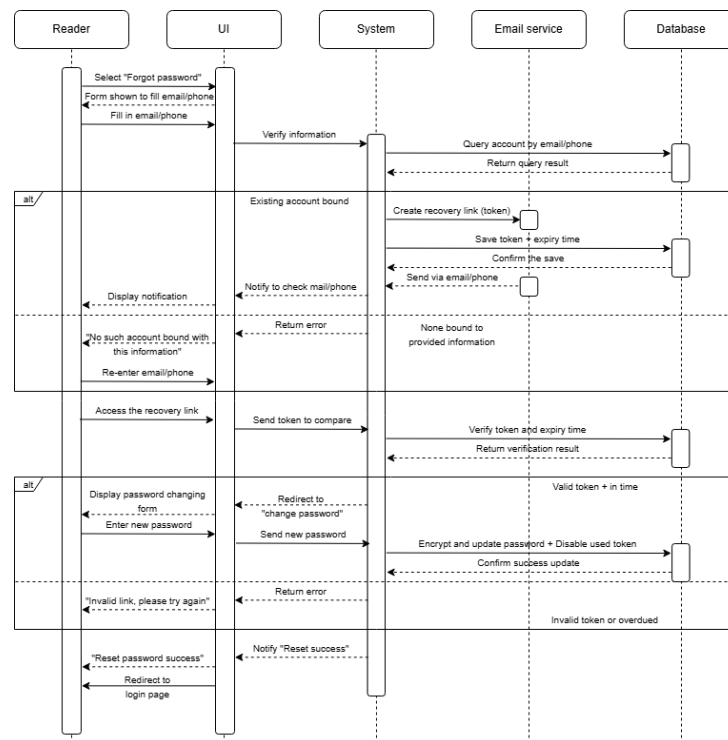


Figure 39: Sequence Diagram for Account Password Reset

1. User requests password reset by entering registered email.
2. System verifies email, generates reset token, and dispatches reset link via email.
3. User opens link; system verifies link token validity. User submits new password; system updates password hash.

5.5.4 Document Search

The Document Search sequence diagram describes the message flow initiated when a user queries the catalog. It depicts the communication between the UI, Search Service, and Catalog Database, including query parsing, index execution, paginated dataset retrieval, and detail view fetching.

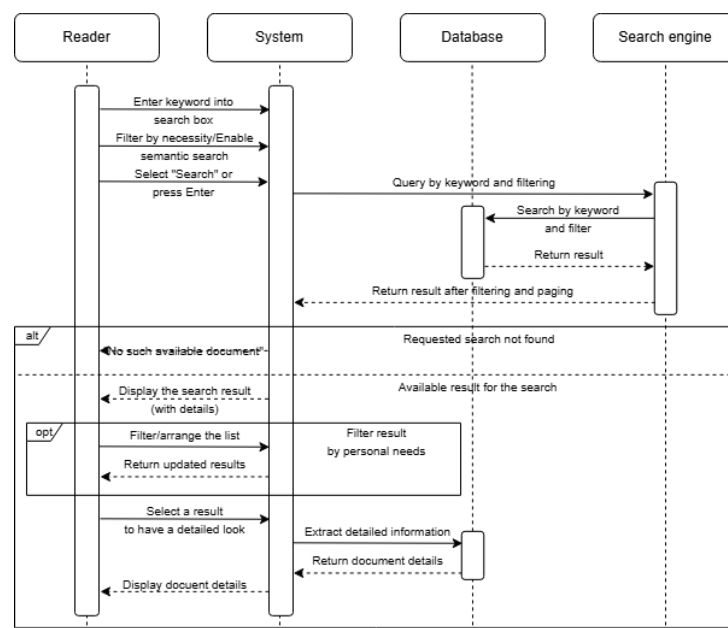


Figure 40: Sequence Diagram for Document Search

1. Reader enters search query.
2. Search engine queries catalog database and returns filtered, paginated results.

3. Renders item cards; clicking item retrieves detailed record view.

5.5.5 Add / Edit / Delete Document

This sub-section models the sequential operations performed by Librarians to manage individual catalog entries, including creation, modification, and soft deletion workflows.

5.5.5.a Add Document

The Add Document sequence diagram details the interaction between the Librarian UI, Catalog Controller, ISBN API Integration Service, LLM Summary Engine, and Database when entering a new book record into the system.

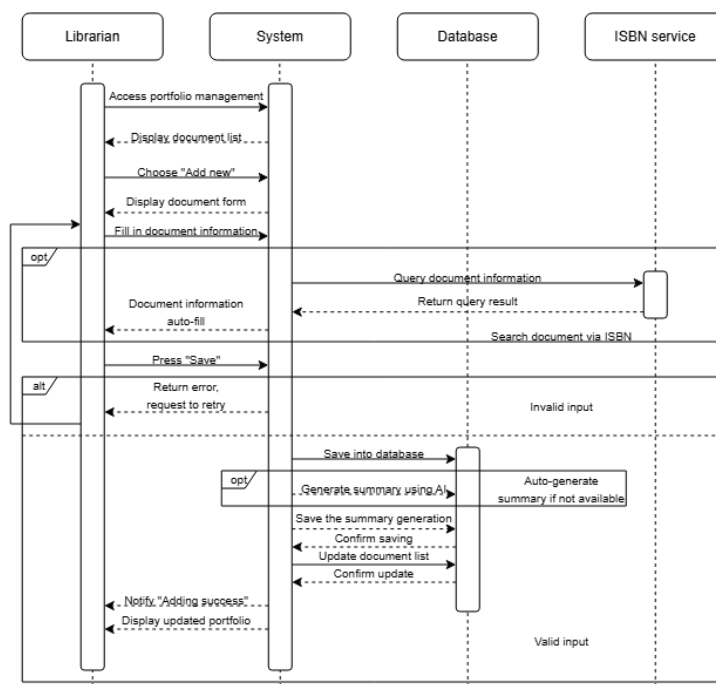


Figure 41: Sequence Diagram for Add Document

Librarian enters metadata manually or fetches via ISBN API (UC-CAT-03), optionally generating AI summary (UC-CAT-04), and saves record.

5.5.5.b Edit Document

The Edit Document sequence diagram illustrates the lifecycle of updating an existing document metadata record, ensuring input validation and persistence in PostgreSQL via Prisma ORM.

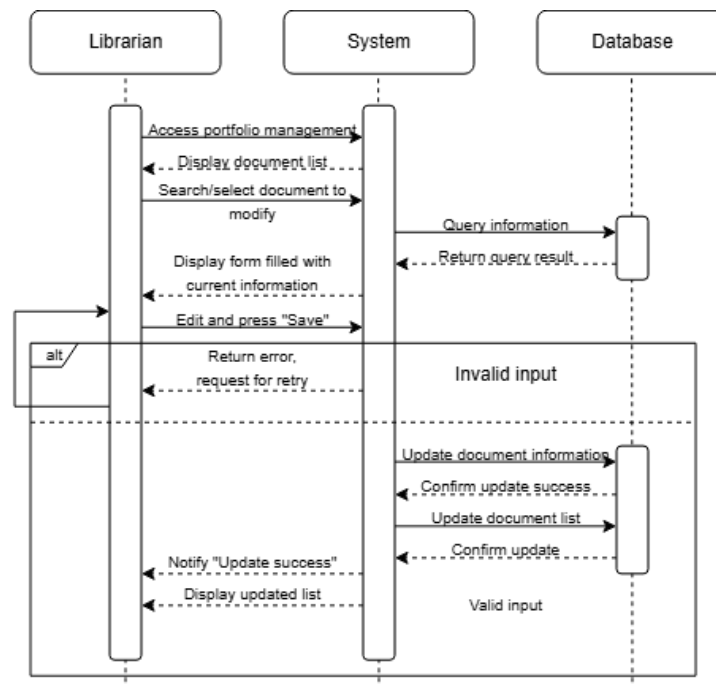


Figure 42: Sequence Diagram for Edit Document

Librarian edits pre-filled item fields, system validates inputs, and updates DB record.

5.5.5.c Delete Document

The Delete Document sequence diagram models the safety checks and deletion execution flow, verifying that the physical item is not currently checked out before performing a soft deletion.

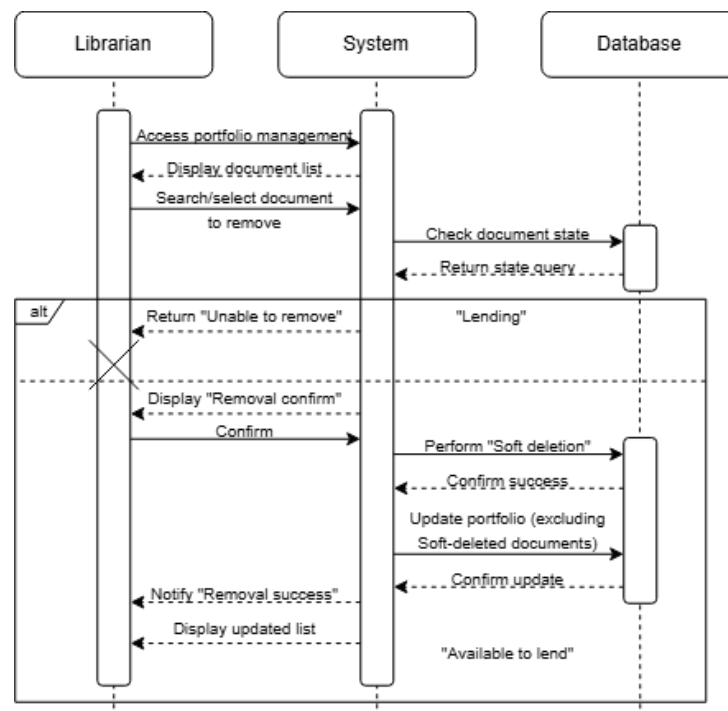


Figure 43: Sequence Diagram for Delete Document

System verifies item is not currently borrowed, librarian confirms deletion, and system performs soft delete.

5.5.6 Batch Document Import

The Batch Document Import sequence diagram illustrates the communication between the UI, Import Controller, File Parsing Service, Validation Service, and Database during bulk data processing. It details batch database insert transactions and error response summaries.

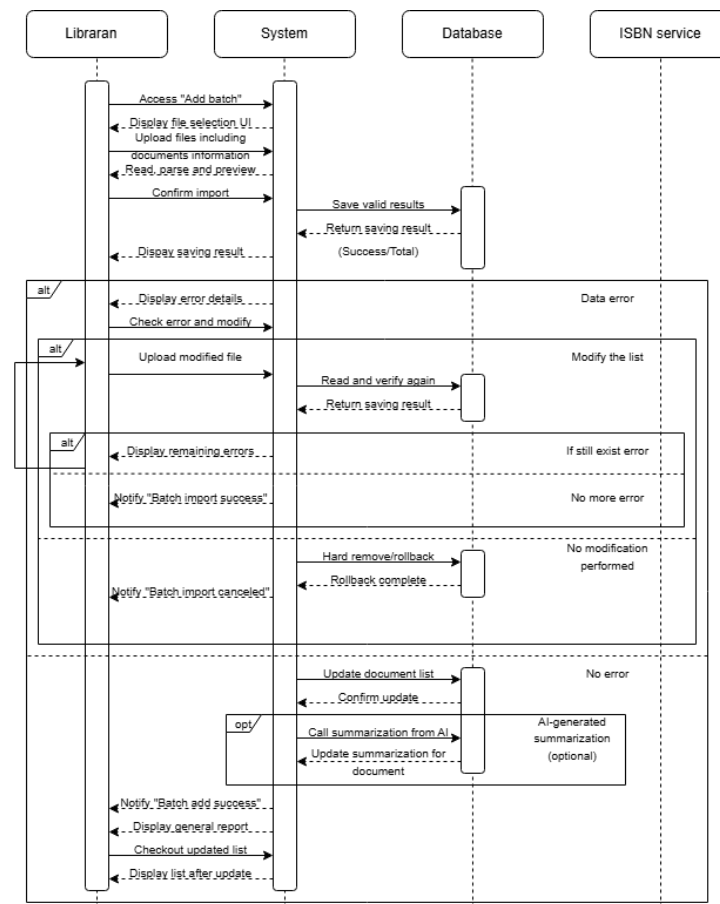


Figure 44: Sequence Diagram for Batch Document Import

Librarian uploads file; system validates rows, imports valid entries, and reports error summary.

5.5.7 Reserve Document

The Reserve Document sequence diagram depicts object interactions when a patron places a hold request on an out-of-stock book. It details copy availability verification, reservation queue insertion, and automated notification dispatch.

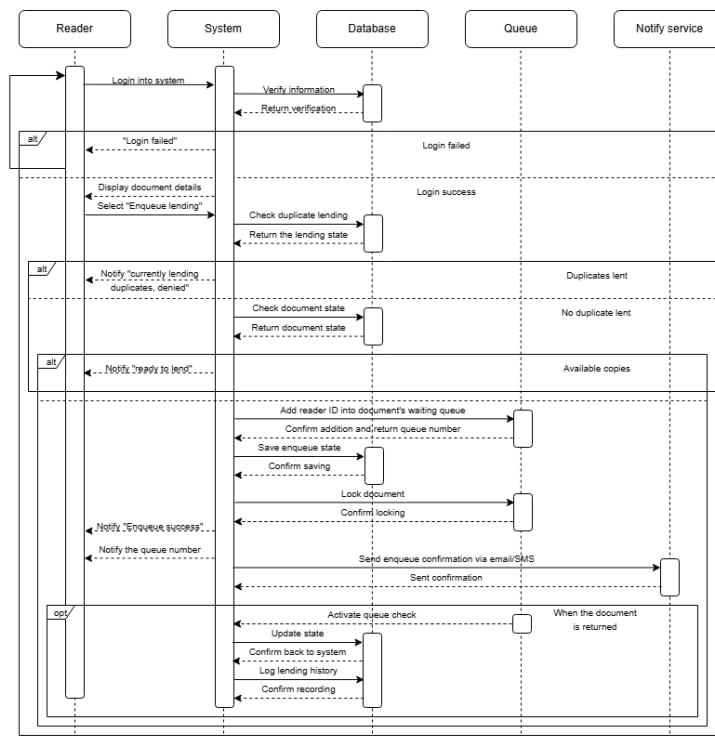


Figure 45: Sequence Diagram for Reserve Document

System checks for duplicate borrowings and copy availability; if zero available copies exist, appends user to queue and sends email confirmation.

5.5.8 Fine Calculation Update

The Fine Calculation Update sequence diagram models interactions between the Fine Controller, Fine Calculation Service, and User Ledger Database. It covers automated overdue duration checks, daily rate applications, and ledger updating.

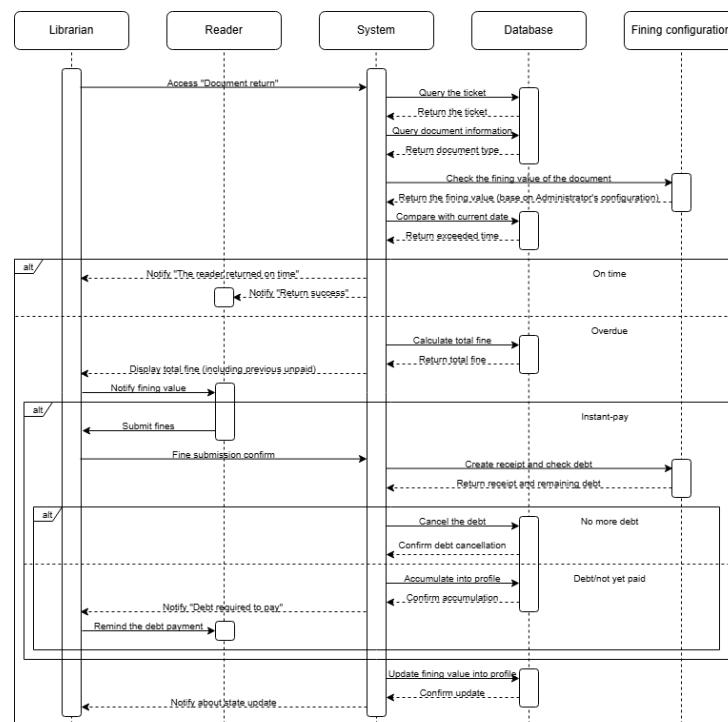


Figure 46: Sequence Diagram for Fine Calculation Update

System verifies borrower fine threshold, scans items, creates loan ticket, and decrements copy inventory.

5.5.9 Process Document Loan

The Process Document Loan sequence diagram displays the message exchanges between the Librarian UI, Circulation Service, Account Service, and Database when issuing a physical item checkout at the circulation desk.

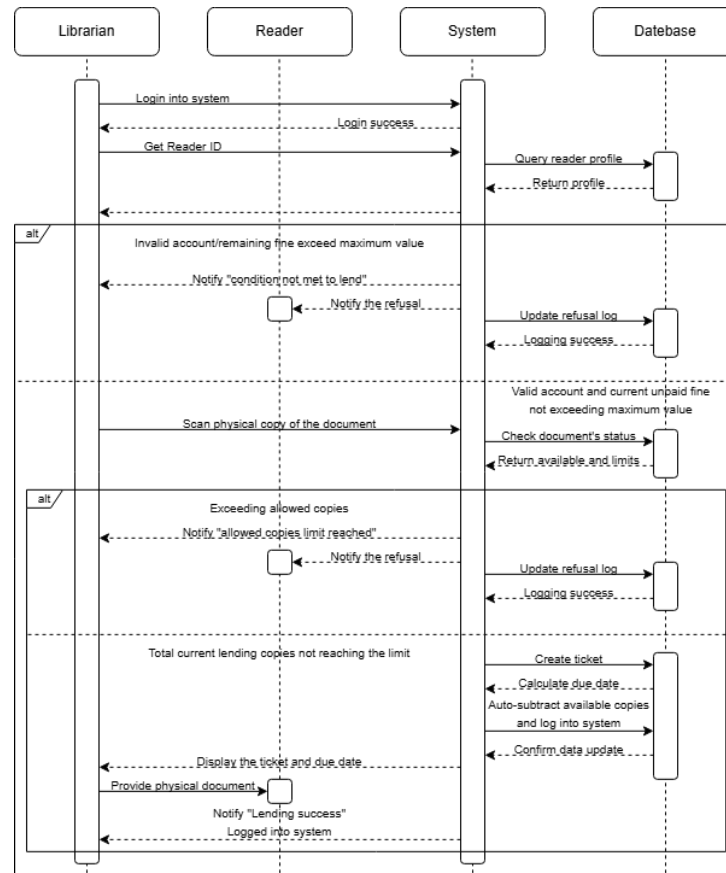


Figure 47: Sequence Diagram for Process Document Loan

Librarian validates reader eligibility, verifies item hold status, checks loan limits, and creates loan ticket.

5.5.10 Extend Document Loan

The Extend Document Loan sequence diagram presents object interactions involved in extending an active loan period, verifying renewal quotas, and ensuring no pending hold requests exist prior to updating the due date.

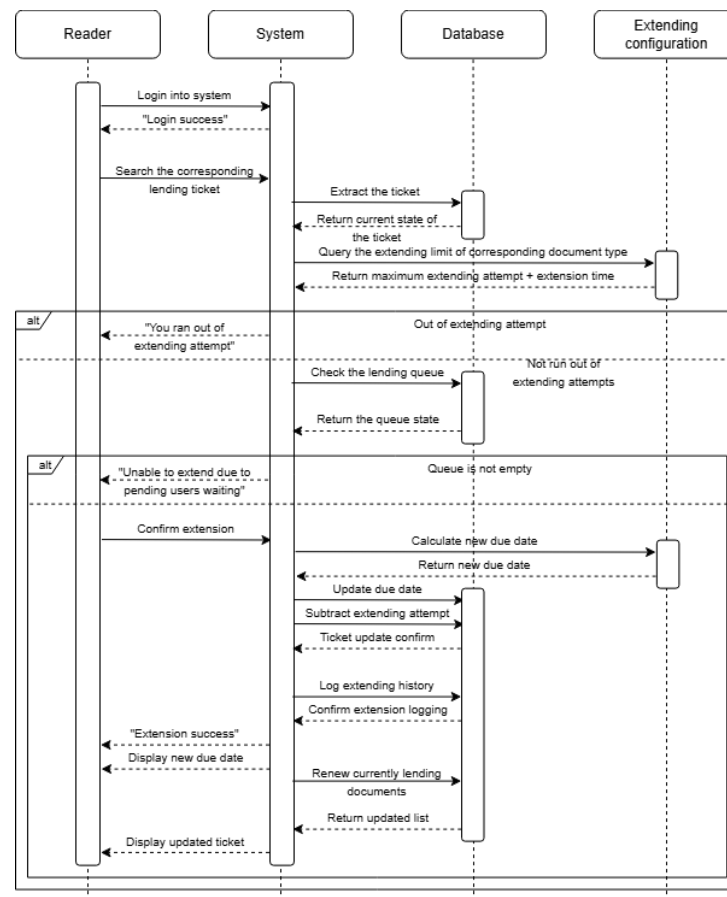


Figure 48: Sequence Diagram for Extend Document Loan

System checks renewal allowances and hold queue status; extends due date upon approval.

5.5.11 Return Document

The Return Document sequence diagram details the end-to-end operational exchanges when processing returned physical items, evaluating potential overdue fines, updating inventory statuses, and triggering hold availability alerts for waiting patrons.

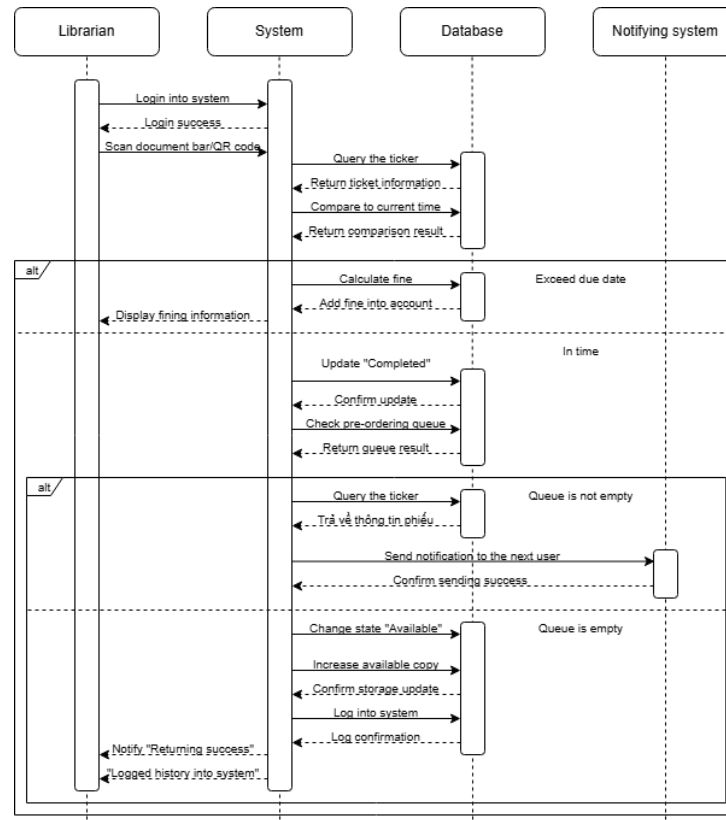


Figure 49: Sequence Diagram for Return Document

Librarian scans item, system assesses overdue fine if applicable, checks hold queue, updates copy status, and closes loan ticket.

6 Implementation

This chapter provides a comprehensive, technically rigorous examination of the implementation of the BkLib On-line Library Management System. It articulates the layered system architecture, foundational technology stack, and engineering decisions powering the platform. Subsequently, it delivers a detailed module-by-module breakdown across four core operational subsystems: Authentication and Access Governance, Reader Self-Service, Librarian Circulation and Operational Administration, and Platform Governance and System Infrastructure. Each module is documented through both user interface interaction workflows and core technical implementations, supported by production code snippets, database schemas, and background job algorithms.

6.1 Technology Stack and Architectural Infrastructure

The BkLib platform is architected as a decoupled, multi-tier system composed of three primary software tiers: a client Single-Page Application (SPA) built with React and Vite, a core API Gateway and Business Logic service implemented in Node.js/Express with Prisma ORM, and a dedicated AI Microservice powered by Python, FastAPI, Google Gemini, and ChromaDB.

6.1.1 Front-end Presentation Layer: React.js and Vite

The front-end user experience is implemented as a modern Single-Page Application (SPA) using React.js 18 paired with the Vite build toolchain. Vite delivers instantaneous Hot Module Replacement (HMR) and highly optimized ES module bundling for production deployment. The presentation layer employs declarative component hierarchies, React Router DOM for client-side role-guarded routing, and Axios interceptors for automated JWT injection and token refresh. On the client side, components manage optimistic UI updates, debounced catalog search inputs, multi-channel notification bells, real-time circulation status indicators, and an embedded floating AI assistant widget with streaming responses.



Figure 50: *Front-End Technology Foundation: React.js and Vite Build Engine*

6.1.2 Back-end Core Server: Node.js and Express

The core application server is constructed upon Node.js and the Express framework using TypeScript. It functions as the central API gateway and orchestrator for business logic, catalog management, circulation rules, user accounts, and session administration. The server enforces a modular, layered software architecture comprising Controllers, Services, Middlewares, and Background Cron Jobs. It delegates AI inference tasks (RAG context querying, document embedding generation, personalized recommendations) to the dedicated Python AI service via secure internal HTTP endpoints authenticated through shared secret tokens (X-Internal-Key).



Figure 51: *Back-End Application Gateway: Node.js and Express Framework*

6.1.3 Dedicated AI Microservice: Python and FastAPI

AI capabilities are isolated into a standalone microservice developed in Python 3.10+ using FastAPI and Pydantic v2. This service harnesses high-throughput asynchronous execution to deliver low-latency NLP endpoints. Its

key responsibilities include generating vector embeddings via pre-trained Transformer models, executing semantic book search with cosine similarity, coordinating Retrieval-Augmented Generation (RAG) over library regulations and FAQ repositories, and driving function-calling tools that bridge patron inquiries with live database records.



Figure 52: *Dedicated AI Microservice Engine: Python and FastAPI*

6.1.4 Data Modeling and ORM: Prisma

Prisma ORM is leveraged as the data abstraction and query engine within the Node.js service. Prisma provides end-to-end type safety across the application code and database layer, automatically generating TypeScript definitions from the declarative schema (`schema.prisma`). It manages automated schema migrations, relation modeling, and complex transactional queries across loans, fines, hold queues, and audit logs while guaranteeing ACID transaction semantics for critical operations like checkout and return.



Figure 53: *Object-Relational Mapping Engine: Prisma (TypeScript)*

6.1.5 Relational Database: PostgreSQL

PostgreSQL serves as the primary relational data store for all persistent business entities, including patron identities, physical inventory items, circulation ledgers, system configurations, and security audit trails. PostgreSQL guarantees strict ACID compliance for loan and fine settlement transactions, leverages B-Tree indexes for rapid metadata queries, and utilizes JSONB columns to capture before-and-after audit diffs and dynamic system parameters without requiring table migrations.



Figure 54: *Relational Database Management System: PostgreSQL*

6.1.6 Vector Store and Embedding Infrastructure: ChromaDB and pgvector

The system employs a purpose-built dual-vector architecture to serve two distinct AI workloads:

- **ChromaDB (In-Memory, FAQ/RAG):** Serves as the retrieval engine for the library's official rules and FAQ documentation. The FAQ corpus is partitioned into overlapping chunks (400 characters, 60-character overlap) and indexed into ChromaDB collections during application startup. Its in-memory design delivers low-latency lookups for this small, static knowledge base.
- **pgvector (PostgreSQL Extension, Semantic Search):** Stores 384-dimensional dense vector embeddings generated by paraphrase-multilingual-MiniLM-L12-v2 alongside book metadata directly within PostgreSQL. An HNSW index enables sub-millisecond cosine-similarity queries over the full book catalog, and standard SQL WHERE clauses allow seamless combination of vector search with relational filters (category, language, availability).

Grounding LLM outputs to business facts with embeddings and vector search

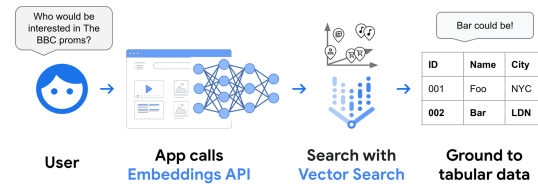


Figure 55: Vector Embedding and Retrieval Pipeline Architecture

6.1.7 Adaptive Hybrid Search Strategy

To overcome the limitations of pure keyword search (which fails on synonyms and conceptual phrasing) and pure vector search (which can miss exact ISBNs, call numbers, or proper nouns), BkLib implements an **Adaptive Hybrid Search Engine**. The engine computes a unified composite score:

$$S_{\text{hybrid}} = w_{\text{semantic}} \cdot S_{\text{cosine}} + w_{\text{keyword}} \cdot S_{\text{overlap}} \quad (1)$$

The weights w_{semantic} and w_{keyword} dynamically adjust according to query token count:

- **Short Queries (< 3 tokens):** $w_{\text{semantic}} = 0.40, w_{\text{keyword}} = 0.60$ (prioritizing exact title/author matching).
- **Medium Queries (3 – 5 tokens):** $w_{\text{semantic}} = 0.65, w_{\text{keyword}} = 0.35$ (balanced conceptual and keyword weighting).
- **Long Queries (> 5 tokens):** $w_{\text{semantic}} = 0.80, w_{\text{keyword}} = 0.20$ (heavily favoring semantic intent).

6.1.8 Security, Authentication, and Role-Based Access Control

System security is established on four pillars:

- **Stateless JWT Sessions:** Access tokens are signed using HMAC-SHA256 with short lifespans (15 minutes), supplemented by refresh tokens stored in HTTP-only cookies or secure headers for session persistence.
- **Bcrypt Hashing:** Passwords undergo salted Bcrypt hashing with configurable rounds (`BCRYPT_ROUNDS = 10`).
- **Brute-Force Account Lockout:** Accounts incurring five consecutive failed authentication attempts are automatically locked for fifteen minutes (`lockedUntil`).
- **Granular RBAC:** Enforces role barriers across three hierarchical roles: `READER`, `LIBRARIAN`, and `ADMIN`.

6.1.9 External Integrations: Open Library, Google Books, and Nodemailer

- **Bibliographic APIs (Google Books, Open Library, IT Bookstore):** Automated fallback cascade that enriches catalog records with cover images, publishers, publication years, and descriptions upon ISBN entry.
- **Transactional Messaging (Nodemailer):** SMTP integration delivering HTML emails for 6-digit OTP verification, loan approvals, pickup readiness notices, and due date alerts.



Figure 56: External Bibliographic Integration via Google Books and Open Library

6.1.10 Background Task Processing: node-cron

Automated platform maintenance is governed by scheduled cron jobs via `node-cron`. Specifically, a daily sweep executed at 01:00 AM evaluates all reservations in `READY_FOR_PICKUP` status whose three-day grace period has lapsed, automatically marking them as `EXPIRED`, freeing physical copies, and advancing the queue to the next waiting patron.

Table 31: Comprehensive Technology Stack Summary of the BkLib Platform

Layer / Component	Technology / Library	Key Responsibilities and Functionalities
Client Framework	React.js 18.2, Vite	Single-Page Application, virtual DOM, HMR dev server.
Routing & UI State	React Router DOM, TailwindCSS	Declarative role routing, responsive layout, modal dialogs.
Core Server Engine	Node.js 20 LTS, Express 4.x	RESTful API endpoints, circulation logic, RBAC middleware.
Database & ORM	PostgreSQL 15, Prisma 5.x	Relational data store, ACID transactions, type-safe queries.
AI Microservice	Python 3.10+, FastAPI	Low-latency async NLP APIs, RAG pipelines, semantic search.
Large Language Model	Google Gemini 3.1 Flash	Natural language reasoning, FAQ answering, query intent extraction.
Vector Store & Embeddings	ChromaDB, SentenceTransformers	In-memory FAQ chunk vector indexing, multilingual embeddings.
Email Delivery	Nodemailer 6.x	Transactional HTML emails for OTP and circulation reminders.
Background Automation	node-cron 3.x	Daily batch sweep for reservation expiry and queue promotion.

6.2 Authentication and Access Governance Module

The Authentication Module manages identity verification, role-based ingress routing, patron onboarding, and multi-factor password recovery.

6.2.1 User Interface Implementation

6.2.1.1 Login Page The primary entry portal provides unified authentication with role-switching tabs.

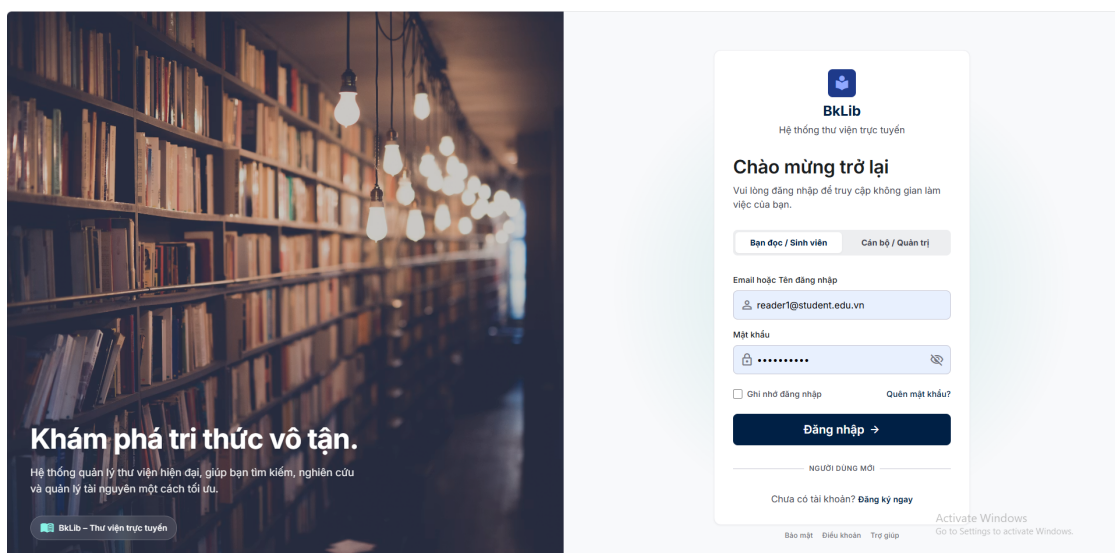


Figure 57: BkLib Login Interface with Dual-Role Selector Tabs

Layout & Functionality: Features a split-screen layout with university branding artwork on the left and an authentication card on the right. Includes a role selector tab (“Bạn đọc / Sinh viên” for patron self-service vs. “Cán bộ / Quản trị” for staff operations), identifier inputs (email, username, or student ID), password field with visibility toggle, a “Ghi nhớ đăng nhập” (Remember Me) checkbox, and navigation triggers for password recovery and account registration. Upon successful validation, patrons are redirected to their reader dashboard while librarians and administrators access their operational consoles.

6.2.1.2 Registration Page Self-service onboarding form for new library members.

The registration interface is split into two panels. The left panel features a background image of a library aisle with the text 'Khám phá tri thức không giới hạn.' and 'Hệ thống thư viện hiện đại giúp bạn dễ dàng tìm kiếm, quản lý và mở rộng nguồn tài nguyên học thuật.' Below this, it says 'Hơn 1.000+ tài liệu' and 'Để sử dụng'. The right panel is titled 'Tạo tài khoản BkLib' and includes a sub-header 'Tạo tài khoản để bắt đầu trải nghiệm hệ thống quản lý thư viện thông minh.' The form fields include: 'Họ tên' (Full Name) with the example 'Nguyễn Văn A', 'Email' with 'example@university.edu.vn', 'Số điện thoại (không bắt buộc)' (Phone Number) with '090 123 4567', 'Mã số SV (không bắt buộc)' (Student ID) with '2212345', and 'Chi nhánh đăng ký' (Branch Affiliation) with a dropdown menu showing '-- Chọn chi nhánh --'. There are also fields for 'Mật khẩu' (Password) and 'Nhập lại mật khẩu' (Confirm Password), both masked with dots. A checkbox for terms of service is present, with a link to 'điều khoản dịch vụ và chính sách bảo mật'. A 'Đăng ký' button is at the bottom, followed by a link 'Đã có tài khoản? Đăng nhập ngay' and a small 'Activate Windows' watermark.

Figure 58: BkLib Account Registration Interface

Layout & Functionality: Captures mandatory user profile attributes: Full Name, Email, Phone Number, Student ID (“Mã số SV”), and Branch Affiliation (“Chi nhánh đăng ký”). It enforces password validation rules with confirmation matching and terms-of-service consent checkboxes prior to submission. Upon submission, an account is created in PENDING_VERIFICATION status, and an automated 6-digit OTP is dispatched to the user’s email.

6.2.1.3 Password Reset Workflow (3-Step Modal) Modal-based credential recovery mechanism utilizing 6-digit Email OTP verification.

The workflow consists of three modal steps. Step 1, 'Quên mật khẩu (Bước 1/3)', prompts the user to enter their registered email address. The example email is 'baokhoa.01092021@gmail.com'. Step 2, 'Quên mật khẩu (Bước 2/3)', shows the received 6-digit OTP '123456' and prompts the user to verify it. Step 3, 'Quên mật khẩu (Bước 3/3)', prompts the user to create a new password, showing a strength indicator 'Tối thiểu 8 ký tự' and a confirmation field 'Nhập lại mật khẩu giống ô trên'.

(a) Step 1: Email Submission

(b) Step 2: OTP Verification

(c) Step 3: Password Reset

Figure 59: Three-Stage Modal Workflow for Self-Service Password Recovery

Layout & Functionality: Operates within a focused modal dialog spanning three stages:

- **Step 1 (Request OTP):** Users supply their registered email address. The backend validates account existence and generates a 6-digit OTP code valid for 10 minutes.
- **Step 2 (OTP Verification):** Prompts input of the 6-digit code received via email, featuring real-time input masking, validation feedback, and countdown expiration timers.
- **Step 3 (Password Update):** Prompts creation and confirmation of a new password (minimum 8 characters), verifying password parity before updating the database.

6.2.2 Technical Implementation Highlights

Authentication middleware validates bearer tokens and enforces role restrictions across protected routes.

```

1 // Middleware verifying JWT access token from Authorization header
2 export const authenticate = (req: Request, res: Response, next: NextFunction): void => {
3   const authHeader = req.headers.authorization;
4   if (!authHeader || !authHeader.startsWith('Bearer ')) {
5     res.status(401).json({ success: false, message: 'Khong co token xac thuc' });
6     return;
7   }
8   const token = authHeader.split(' ')[1];
9   try {
10    const decoded = jwt.verify(token, env.JWT_SECRET) as AuthPayload;
11    req.user = decoded;
12    next();
13  } catch (err) {
14    if (err instanceof jwt.TokenExpiredError) {
15      res.status(401).json({ success: false, message: 'Token da het han' });
16    } else {
17      res.status(401).json({ success: false, message: 'Token khong hop le' });
18    }
19  }
20 };
21
22 // Middleware factory restricting endpoint access to specified roles
23 export const authorize = (...roles: Role[]) => {
24   return (req: Request, res: Response, next: NextFunction): void => {
25     if (!req.user) {
26       res.status(401).json({ success: false, message: 'Chua xac thuc' });
27       return;
28     }
29     if (!roles.includes(req.user.role)) {
30       res.status(403).json({ success: false, message: 'Khong co quyen truy cap' });
31       return;
32     }
33     next();
34   };
35 };

```

Listing 1: JWT Authentication and Authorization Middleware (backend-express/src/middlewares/auth.middleware.ts)

```

1 export const forgotPassword = async (email: string) => {
2   const user = await prisma.user.findUnique({ where: { email } });
3   if (!user) throw createError('Email khong ton tai trong he thong', 404);
4
5   // Invalidate any existing unused reset tokens for this user
6   await prisma.otpToken.updateMany({
7     where: { userId: user.id, type: 'RESET_PASSWORD', usedAt: null },
8     data: { usedAt: new Date() },
9   });
10
11   // Generate 6-digit random numeric token
12   const otpToken = Math.floor(100000 + Math.random() * 900000).toString();
13   const expiresAt = new Date(Date.now() + OTP_EXPIRES_MINUTES * 60 * 1000);
14
15   await prisma.otpToken.create({
16     data: { userId: user.id, token: otpToken, type: 'RESET_PASSWORD', expiresAt },
17   });
18
19   await sendResetPasswordEmail(email, otpToken, OTP_EXPIRES_MINUTES);
20   return { message: 'Ma OTP da duoc gui den email cua ban' };
21 };

```

Listing 2: Secure Password Recovery OTP Generation (backend-express/src/modules/auth/auth.service.ts)

6.3 Reader Module

The Reader Module provides self-service capabilities for library patrons. It enables students and faculty to explore the library catalog, access physical and digital assets, renew loans, place hold reservations, audit personal loan histories, and consult an AI assistant.

6.3.1 User Interface Implementation

6.3.1.1 Patron Homepage Personalized operational hub rendered upon reader login.

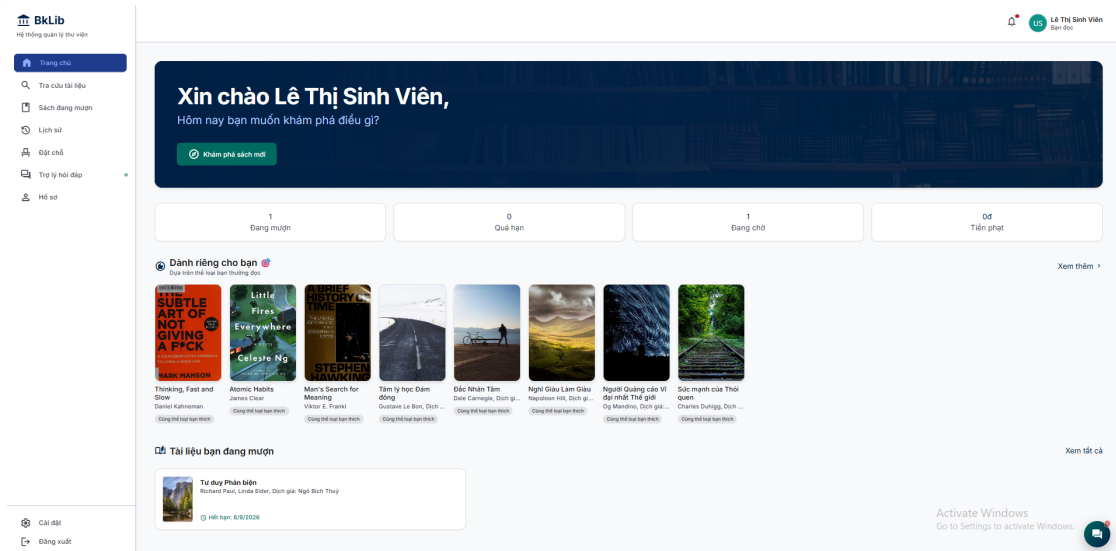


Figure 60: Reader Module Central Dashboard

Layout & Functionality: Organizes patron metrics into top-level summary cards (Active Loans, Overdue Items, Active Holds, and Outstanding Fine Balances). It displays currently borrowed items with visual due date countdown badges (e.g., green for safe periods, red for overdue or imminent deadlines) and provides one-click renewal actions.

6.3.1.2 Catalog Search and Discovery Multi-criteria discovery interface for library resources.

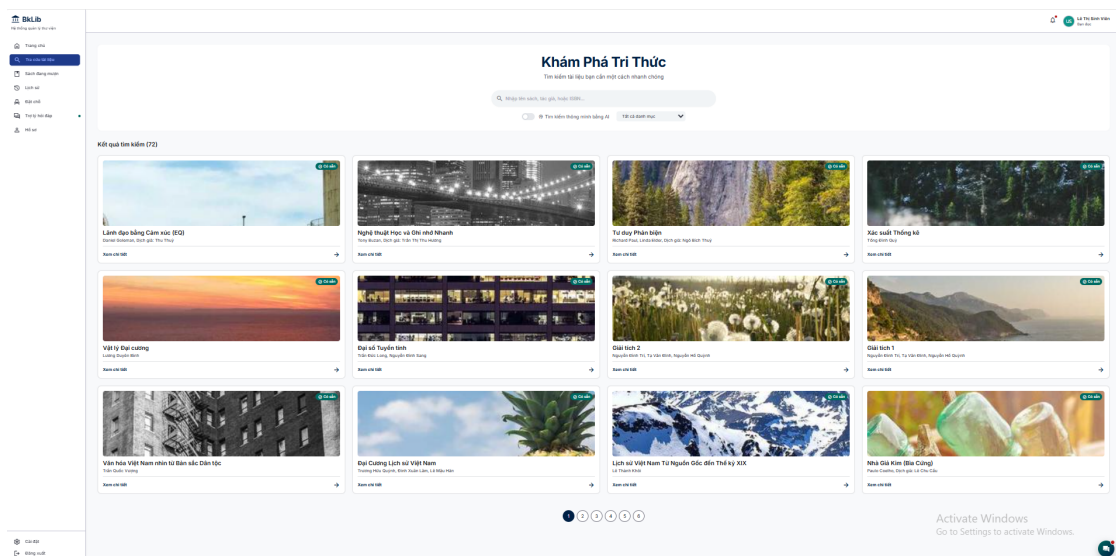


Figure 61: Reader Catalog Search and Filtering Interface

Layout & Functionality: Combines an instant search bar with category tree filtering, author filters, and branch availability selectors. Each search card renders book cover art, bibliographic metadata, dynamic copy availability counters, and quick triggers for online reservations.

6.3.1.3 Active Loans Management and Online Renewal Circulation management view tracking active borrowed items.

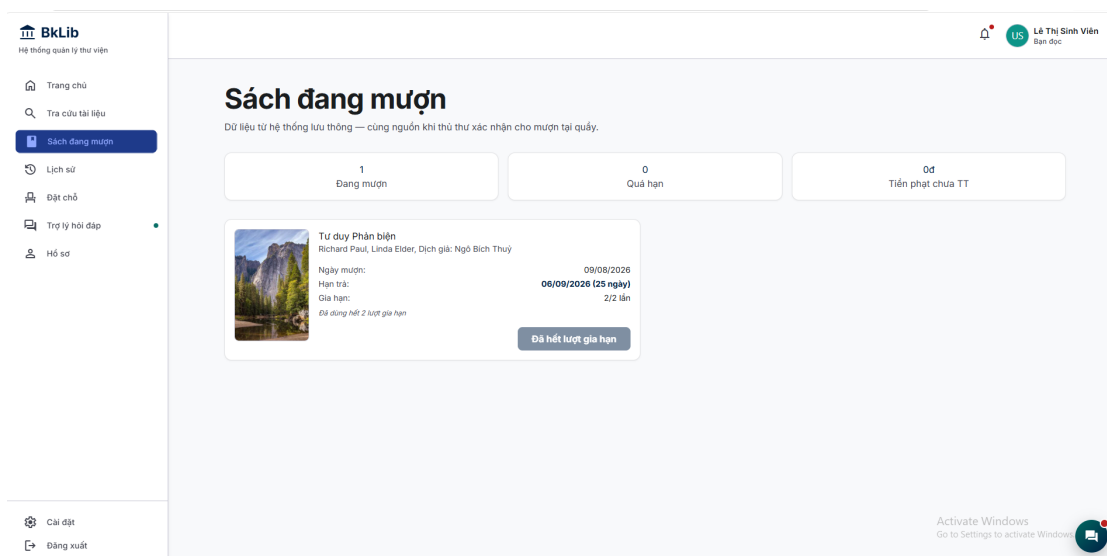


Figure 62: Active Loans Management and Online Renewal Interface

Layout & Functionality: Displays full details for each borrowed item (barcode, checkout date, mandatory due date, remaining extension attempts). Patrons can trigger online loan renewals, which dynamically evaluate system limits ($\text{max_renew_count} = 2$) and verify that no other patrons have reserved the document.

6.3.1.4 Borrowing and Fine History Audit ledger tracking past transactions and financial records.

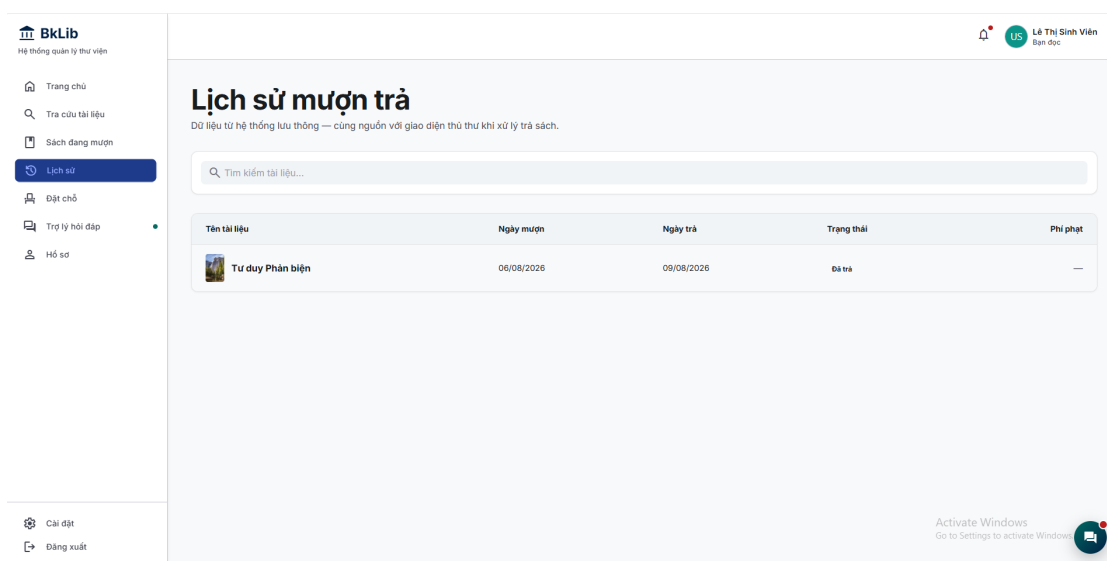


Figure 63: Borrowing History and Fine Audit Ledger

Layout & Functionality: Tabular view presenting historical loans, return dates, overdue day counts, and fine settlement statuses (Pending, Paid, Waived). Features date filtering and sorting to support personal loan auditing.

6.3.1.5 Active Hold Reservations Queue tracking interface for reserved items.

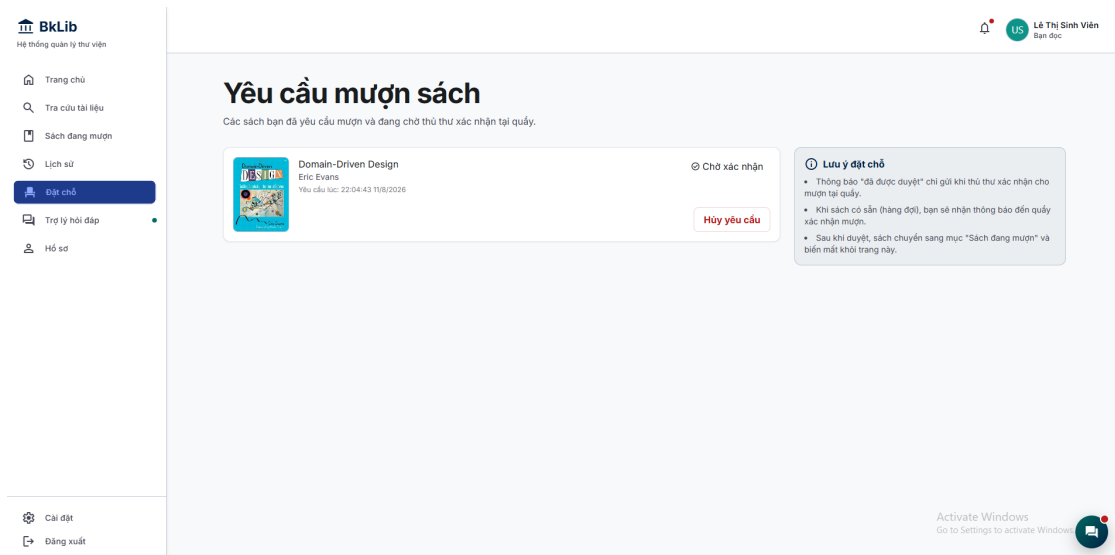


Figure 64: Active Hold Reservations and Queue Status Interface

Layout & Functionality: Lists placed reservations with queue priority positions (queuePosition), current status (“Đang chờ” vs. “Sẵn sàng lấy sách”), pickup deadlines (3-day countdown upon arrival), and cancellation triggers.

6.3.1.6 Virtual AI Assistant Widget Interactive chat interface facilitating natural language queries about library policies.

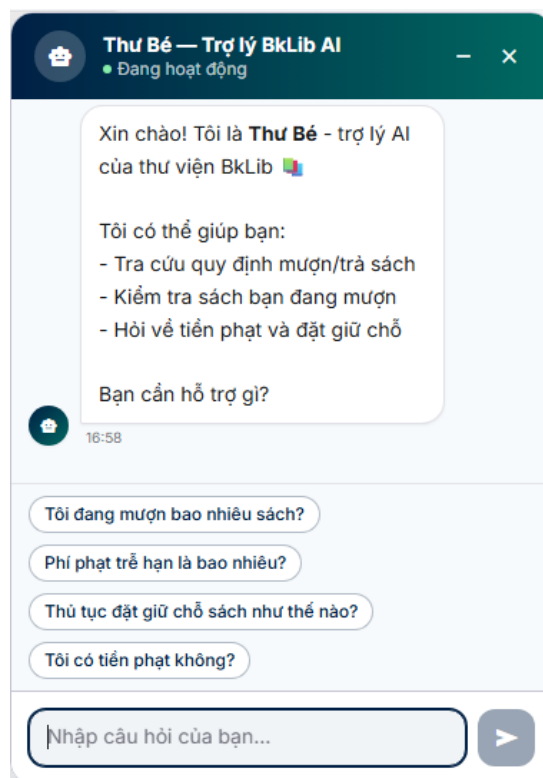


Figure 65: Virtual Assistant Chat Window Interface

Layout & Functionality: Floating chat window with prompt suggestion chips (“Noi quy thu vien”, “Quy dinh phat qua han”, “Thu tuc muon tra sach”). The chatbot uses a RAG pipeline: it retrieves the top-3 most relevant FAQ chunks from ChromaDB and feeds them as grounded context into Gemini to generate accurate, hallucination-free responses about library regulations.

6.3.1.7 Personal Profile and Notification Preferences Patron account preferences and communication channels.

Figure 66: Personal Profile and Notification Settings Interface

Layout & Functionality: Allows updating contact details, altering account passwords, and toggling notification channels (In-App, Email, SMS) for due date reminders, hold readiness alerts, and announcements.

6.3.2 Technical Implementation Highlights

```

1 def retrieve_context(query: str, top_k: int = 3) -> str:
2     """
3     Retrieve top-k relevant text chunks for a query from ChromaDB vector collection.
4     Formats results with segment headers to feed into LLM prompt context.
5     """
6     global _collection
7     if _collection is None or _collection.count() == 0:
8         return ""
9
10    embedder = get_embeddings()
11    query_embedding = embedder.encode([query]).tolist()
12
13    results = _collection.query(
14        query_embeddings=query_embedding,
15        n_results=min(top_k, _collection.count()),
16    )
17    docs = results.get("documents", [[]])[0]
18    if not docs:
19        return ""
20
21    return "\n\n---\n\n".join([f"[Doan {i+1}]: {doc}" for i, doc in enumerate(docs)])

```

Listing 3: ChromaDB Vector Retrieval for FAQ RAG (ai-service/app/services/rag_service.py)

```

1 export const renewBorrowRecord = async (borrowRecordId: string, userId: string) => {
2     const borrowRecord = await prisma.borrowRecord.findUnique({
3         where: { id: borrowRecordId },
4         include: { physicalCopy: { include: { book: true } } },
5     });
6
7     if (!borrowRecord) throw createError('Không tìm thấy phiếu mượn', 404);
8     if (borrowRecord.userId !== userId) throw createError('Không có quyền gia hạn', 403);
9     if (borrowRecord.status !== BorrowStatus.ACTIVE) {
10         throw createError('Phiếu mượn không còn hiệu lực', 422);
11     }
12
13     // Prevent online renewal if item is already overdue
14     const now = new Date();
15     if (now > borrowRecord.dueDate) {
16         throw createError('Sách đã quá hạn, không thể gia hạn trực tuyến.', 422);
17     }
18
19     // Verify renewal attempt cap
20     const maxRenew = parseInt(await getConfig(CONFIG_KEYS.MAX_RENEW, '2'), 10);

```

```

21 if (borrowRecord.renewCount >= maxRenew) {
22   throw createError('Da dat gioi han gia han toi da (${maxRenew} lan)', 422);
23 }
24
25 // Block renewal if other patrons are waiting in queue
26 const hasReservation = await prisma.reservation.count({
27   where: {
28     bookId: borrowRecord.physicalCopy.bookId,
29     userId: { not: userId },
30     status: { in: [ReservationStatus.WAITING, ReservationStatus.READY_FOR_PICKUP] },
31   },
32 });
33 if (hasReservation > 0 && borrowRecord.physicalCopy.book.availableCopies === 0) {
34   throw createError('Tai lieu dang co nguoi dat giu cho. Vui long tra dung han.', 422);
35 }
36
37 const renewDays = parseInt(await getConfig(CONFIG_KEYS.RENEW_DURATION, '7'), 10);
38 const newDueDate = new Date(borrowRecord.dueDate);
39 newDueDate.setDate(newDueDate.getDate() + renewDays);
40
41 await prisma.borrowRecord.update({
42   where: { id: borrowRecordId },
43   data: { dueDate: newDueDate, renewCount: { increment: 1 } },
44 });
45
46 return { message: 'Gia han thanh cong', newDueDate, renewCount: borrowRecord.renewCount
47   + 1 };

```

Listing 4: Self-Service Online Loan Renewal (backend-express/src/modules/circulation/circulation.service.ts)

6.4 Librarian Module

The Librarian Module delivers operational capabilities for library staff, supporting counter circulation, catalog management, fine settlement, multi-channel announcements, and operational analytics.

6.4.1 User Interface Implementation

6.4.1.1 Librarian Operational Dashboard Central operational workspace summarizing daily circulation loads.

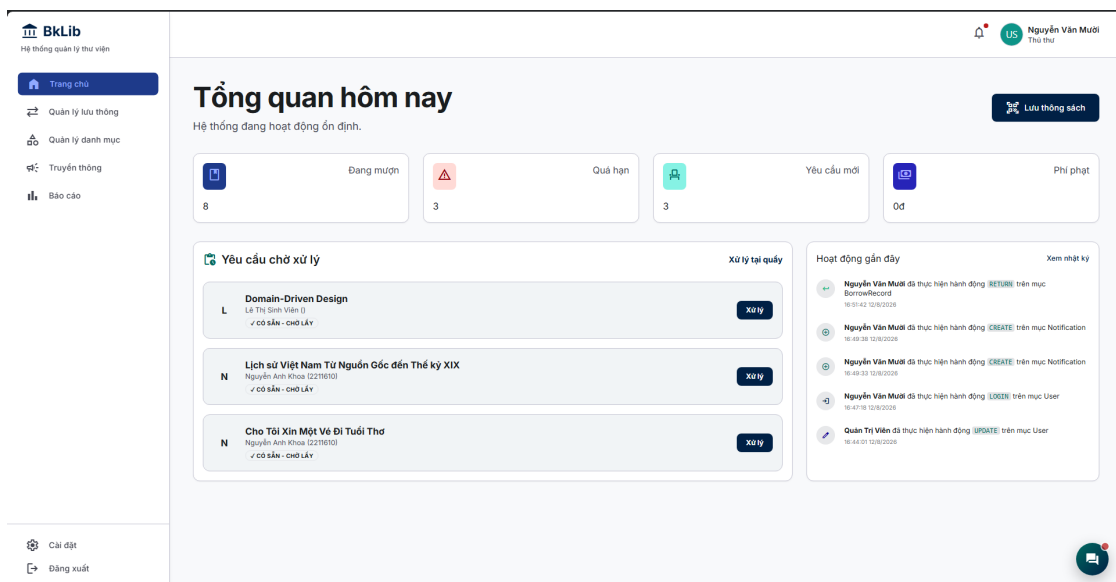


Figure 67: Librarian Operational Dashboard and Quick Actions

Layout & Functionality: Renders vital circulation statistics (Today's checkouts, Today's returns, Overdue alerts, Pending reservations), accompanied by quick navigation shortcuts to the Circulation Desk, Cataloging, Communications, and Reports.

6.4.1.2 Circulation Management Desk Counter processing interface designed for barcode-driven checkouts and returns.

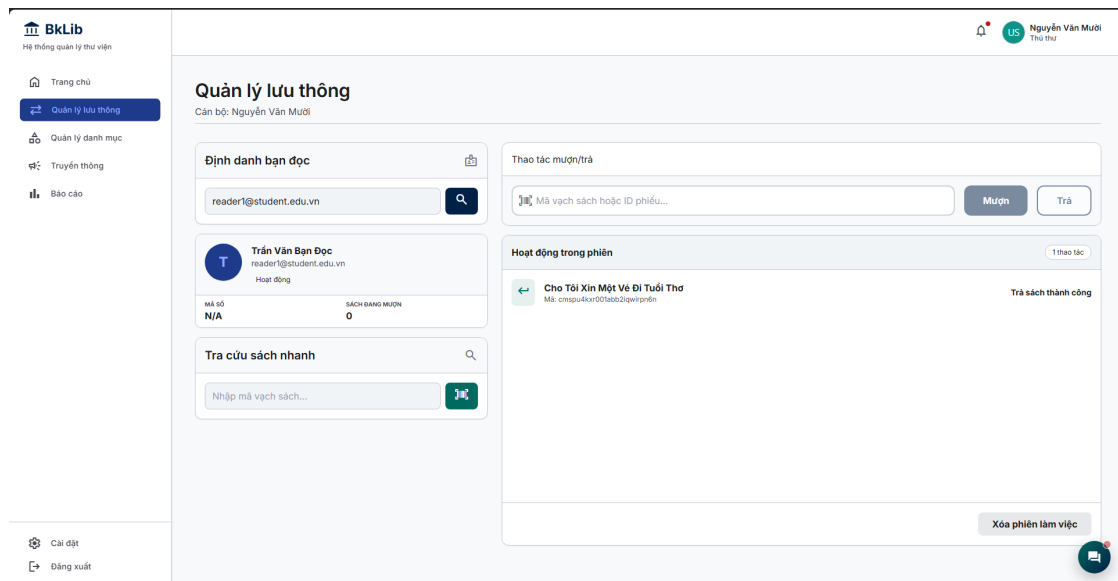


Figure 68: High-Throughput Circulation Desk Interface

Layout & Functionality: Built as a dual-pane layout:

- **Patron Verification Pane (Left):** Card lookup by Student ID or barcode, displaying active loans, overdue flags, and fine status.
- **Item Processing Pane (Right):** Barcode input field with scanner auto-submit. It displays live transactions, calculates overdue penalties, and collects fine payments.

6.4.1.3 Catalog and Inventory Management Collection management workspace for physical and digital bibliographic records.

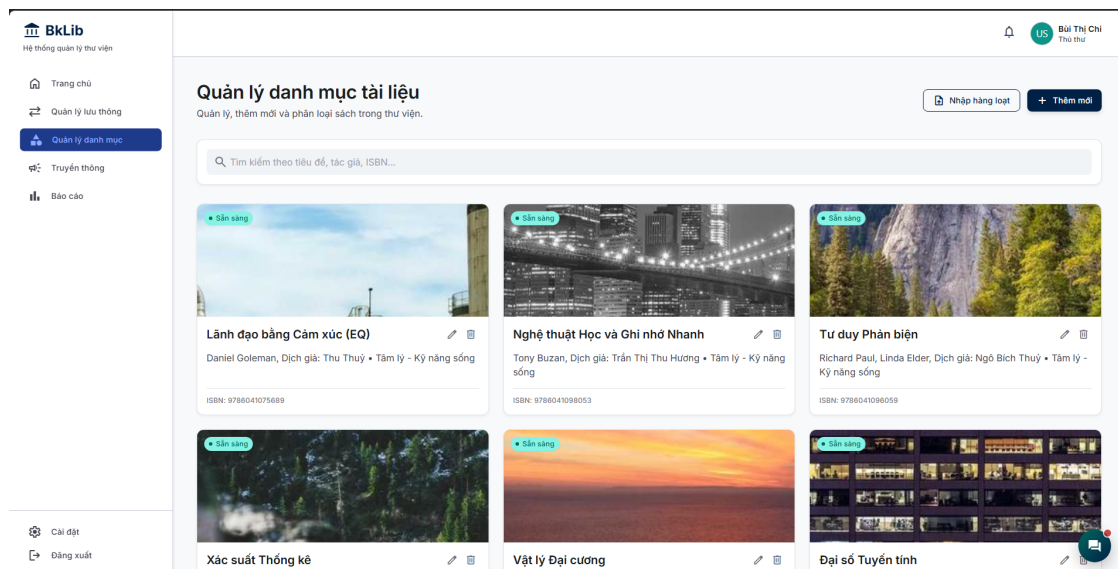


Figure 69: Catalog and Physical Inventory Management Interface

Layout & Functionality: Supports book creation and editing with automated ISBN enrichment (fetching from Google Books and Open Library), batch Excel file importing, and physical copy management (generating unique barcodes, setting shelf locations, and tracking physical condition: GOOD, FAIR, POOR). Each new book record is also embedded into the pgvector index to immediately become searchable via the Semantic Search feature.

6.4.1.4 Communications and Broadcast Management Multi-channel notification portal for reader announcements.

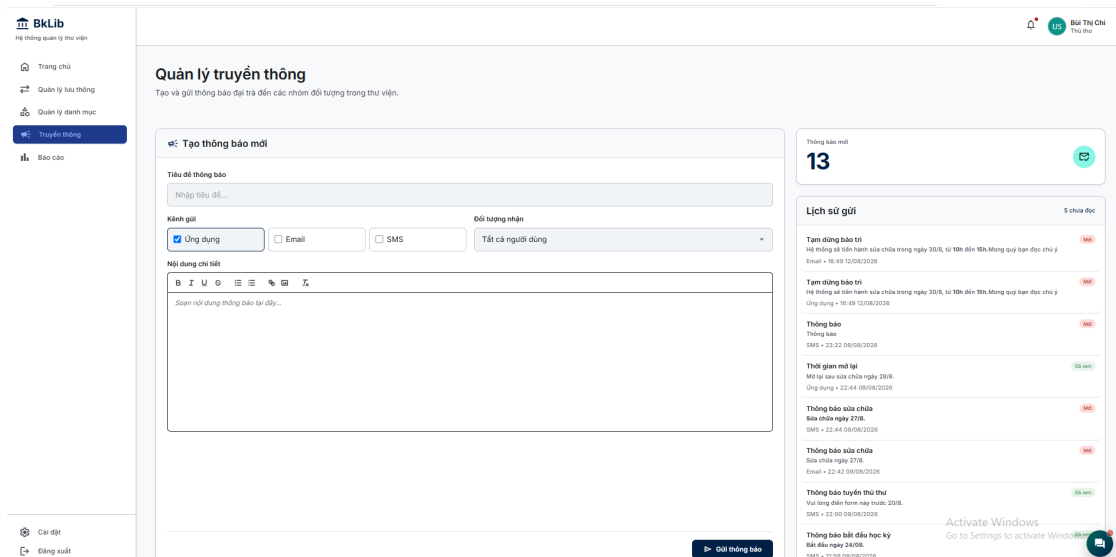


Figure 70: Multi-Channel Communications Dispatcher Interface

Layout & Functionality: Rich-text composition editor with audience targeting (All Active Users, Specific Roles), channel selectors (In-App, SMTP Email, SMS Gateway), and a transmission history log recording delivery metrics.

6.4.1.5 Circulation Reports and Analytics Operational intelligence dashboard tracking collection usage and financial returns.

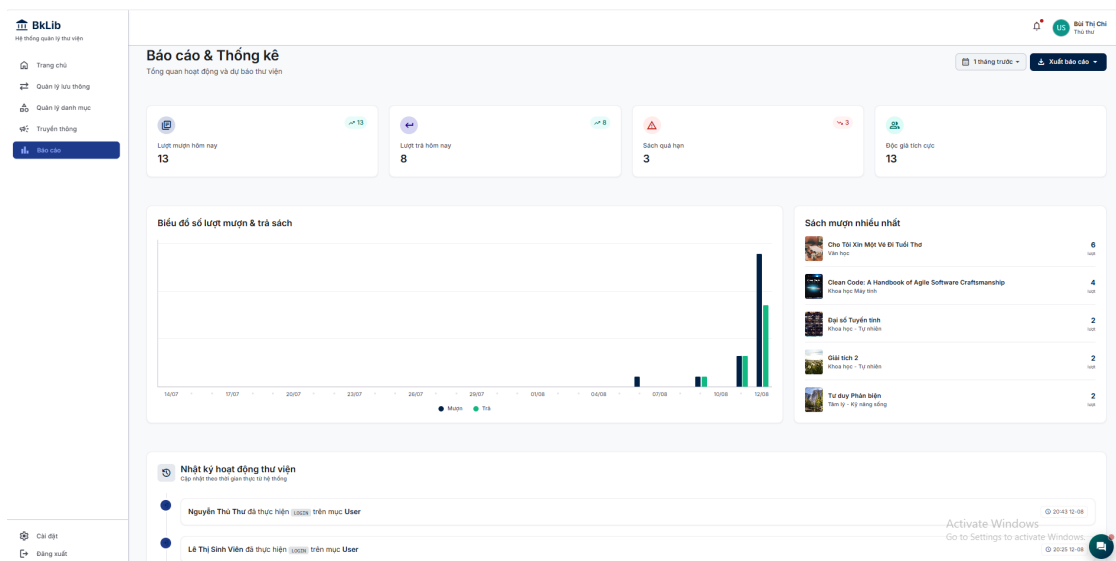


Figure 71: Librarian Reports and Analytics Dashboard

Layout & Functionality: Interactive date-range filters, loan vs. return trend graphs (buildTrendSeries), top borrowed book leaderboards, overdue loan lists, fine revenue breakdowns, and export triggers for CSV, Excel, and PDF formats.

6.4.1.6 Librarian Profile and Settings Staff credentials and operational branch management.

Figure 72: Librarian Staff Profile and Operating Scope Interface

Layout & Functionality: Multi-card dashboard covering personal contact details, secure password change, notification preferences, and assigned branch counter scope (e.g., CS1 - Lý Thường Kiệt vs. CS2 - Dĩ An).

6.4.2 Technical Implementation Highlights

```

1 const calcOverdueDays = async (dueDate: Date, returnDate: Date): Promise<number> => {
2   if (returnDate <= dueDate) return 0;
3
4   // Retrieve official holiday array configured in system settings
5   const holidayConfig = await getConfig(CONFIG_KEYS.HOLIDAY_DATES, '[]');
6   const holidays: string[] = JSON.parse(holidayConfig);
7
8   let overdueDays = 0;
9   const cursor = new Date(dueDate);
10  cursor.setDate(cursor.getDate() + 1);
11
12  // Day-by-day iteration excluding official calendar holidays
13  while (cursor <= returnDate) {
14    const year = cursor.getFullYear();
15    const month = String(cursor.getMonth() + 1).padStart(2, '0');
16    const day = String(cursor.getDate()).padStart(2, '0');
17    const dateStr = `${year}-${month}-${day}`;
18
19    if (!holidays.includes(dateStr)) {
20      overdueDays++;
21    }
22    cursor.setDate(cursor.getDate() + 1);
23  }
24  return overdueDays;
25 };

```

Listing 5: Holiday-Aware Overdue Day Calculation
(backend-express/src/modules/circulation/circulation.service.ts)

```

1 // Step 5: Check if book has a waiting reservation queue
2 const nextReservation = await prisma.reservation.findFirst({
3   where: { bookId: physicalCopy.bookId, status: ReservationStatus.WAITING },
4   orderBy: { queuePosition: 'asc' },
5 });
6
7 if (nextReservation) {
8   // Retain physical copy as RESERVED and allocate to next waiting user
9   await prisma.physicalCopy.update({
10     where: { id: physicalCopy.id },
11     data: { status: CopyStatus.RESERVED },
12   });
13
14   await prisma.reservation.update({
15     where: { id: nextReservation.id },

```

```

16   data: {
17     status: ReservationStatus.READY_FOR_PICKUP,
18     notifiedAt: new Date(),
19     expiresAt: new Date(Date.now() + 3 * 24 * 60 * 60 * 1000), // 3-day window
20     physicalCopyId: physicalCopy.id,
21   },
22 };
23
24 // Notify next patron via In-App and Email
25 await notifyReservationReady(nextReservation.userId, physicalCopy.book.title,
    physicalCopy.bookId);
26 } else {
27   // No waiting queue: return physical copy to available pool
28   await prisma.physicalCopy.update({
29     where: { id: physicalCopy.id },
30     data: { status: CopyStatus.AVAILABLE },
31   });
32   await prisma.book.update({
33     where: { id: physicalCopy.bookId },
34     data: { availableCopies: { increment: 1 } },
35   });
36 }

```

Listing 6: Check-In and Automatic Hold Queue Reallocation
(backend-express/src/modules/circulation/circulation.service.ts)

```

1 export const fetchBookInfoByIsbn = async (isbn: string): Promise<ExternalBookInfo | null>
  => {
2   const cleanIsbn = isbn.replace(/[-\s]/g, '');
3
4   // 1. Query Google Books API
5   try {
6     const response = await axios.get('https://www.googleapis.com/books/v1/volumes', {
7       params: { q: 'isbn:${cleanIsbn}' },
8       timeout: 5000,
9     });
10    if (response.data.totalItems > 0) {
11      const info = response.data.items[0].volumeInfo;
12      return {
13        title: info.title,
14        authorNames: info.authors || [],
15        publisher: info.publisher,
16        publishYear: info.publishedDate ? parseInt(info.publishedDate.substring(0, 4)) :
          undefined,
17        language: info.language,
18        description: info.description,
19        coverImageUrl: info.imageLinks?.thumbnail?.replace(/^http:/, 'https:'),
20        isbn: cleanIsbn,
21      };
22    }
23  } catch (error) { /* Fallback to Open Library on error or rate-limit */ }
24
25   // 2. Fallback to Open Library API
26   try {
27     const olRes = await axios.get(
28       'https://openlibrary.org/api/books?bibkeys=ISBN:${cleanIsbn}&format=json&jscmd=data',
29       { timeout: 5000 }
30     );
31     const olData = olRes.data['ISBN:${cleanIsbn}'];
32     if (olData) {
33       return {
34         title: olData.title,
35         authorNames: olData.authors?.map((a: any) => a.name) || [],
36         publisher: olData.publishers?.map((p: any) => p.name).join(', '),
37         publishYear: olData.publish_date ? parseInt(olData.publish_date.match(/\d{4}/)
          ?.[0]) : undefined,
38         description: olData.notes,
39         coverImageUrl: olData.cover?.medium || olData.cover?.large,
40         isbn: cleanIsbn,
41       };
42     }
43   } catch (error) { /* End of cascade */ }
44   return null;
45 };

```

Listing 7: Automated ISBN Metadata Lookup Cascade (backend-express/src/modules/books/books.utils.ts)

```

1 export const startReservationExpiryJob = () => {
2   // Execute daily at 01:00 AM
3   cron.schedule('0 1 * * *', async () => {
4     const now = new Date();
5     const expired = await prisma.reservation.findMany({
6       where: {
7         status: ReservationStatus.READY_FOR_PICKUP,
8         expiresAt: { lt: now },
9       },
10      include: { book: true },
11    });
12
13    for (const reservation of expired) {
14      // 1. Mark reservation as EXPIRED
15      await prisma.reservation.update({
16        where: { id: reservation.id },
17        data: { status: ReservationStatus.EXPIRED },
18      });
19
20      // 2. Re-assign physical copy to next waiting patron or return to AVAILABLE
21      const nextInQueue = await prisma.reservation.findFirst({
22        where: { bookId: reservation.bookId, status: ReservationStatus.WAITING },
23        orderBy: { queuePosition: 'asc' },
24      });
25
26      if (nextInQueue) {
27        await prisma.reservation.update({
28          where: { id: nextInQueue.id },
29          data: {
30            status: ReservationStatus.READY_FOR_PICKUP,
31            expiresAt: new Date(now.getTime() + 3 * 24 * 60 * 60 * 1000),
32          },
33        });
34      } else {
35        await prisma.physicalCopy.updateMany({
36          where: { bookId: reservation.bookId, status: CopyStatus.RESERVED },
37          data: { status: CopyStatus.AVAILABLE },
38        });
39        await prisma.book.update({
40          where: { id: reservation.bookId },
41          data: { availableCopies: { increment: 1 } },
42        });
43      }
44    }
45  });
46 };

```

Listing 8: Scheduled Background Task for Expired Reservation Cleanup (backend-express/src/jobs/reservationExpiry.job.ts)

6.5 Administrator Module

The Administrator Module delivers platform governance, encompassing user account lifecycles, dynamic system configurations, branch topology management, immutable security audit trails, and database disaster recovery.

6.5.1 User Interface Implementation

6.5.1.1 System Overview Dashboard High-level operational health dashboard monitoring platform metrics.

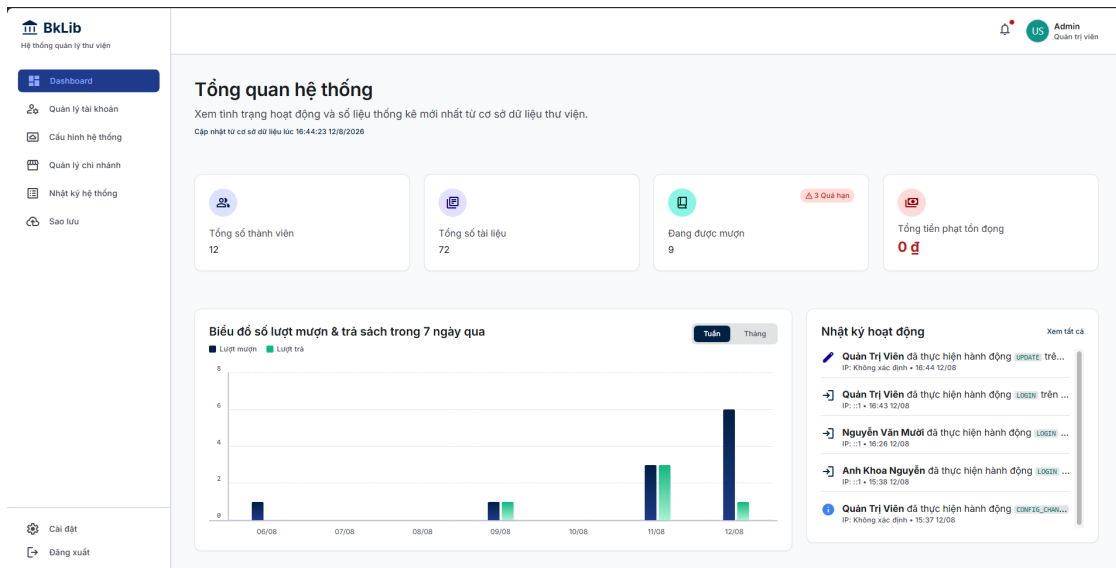


Figure 73: Administrator System Overview Dashboard

Layout & Functionality: Displays high-level platform health indicators: Total Registered Users, Catalog Titles, Total Physical Inventory, System Error Frequencies, Cloud Storage Consumption, and Recent Security Audit Events.

6.5.1.2 User Account Management Centralized identity governance interface.

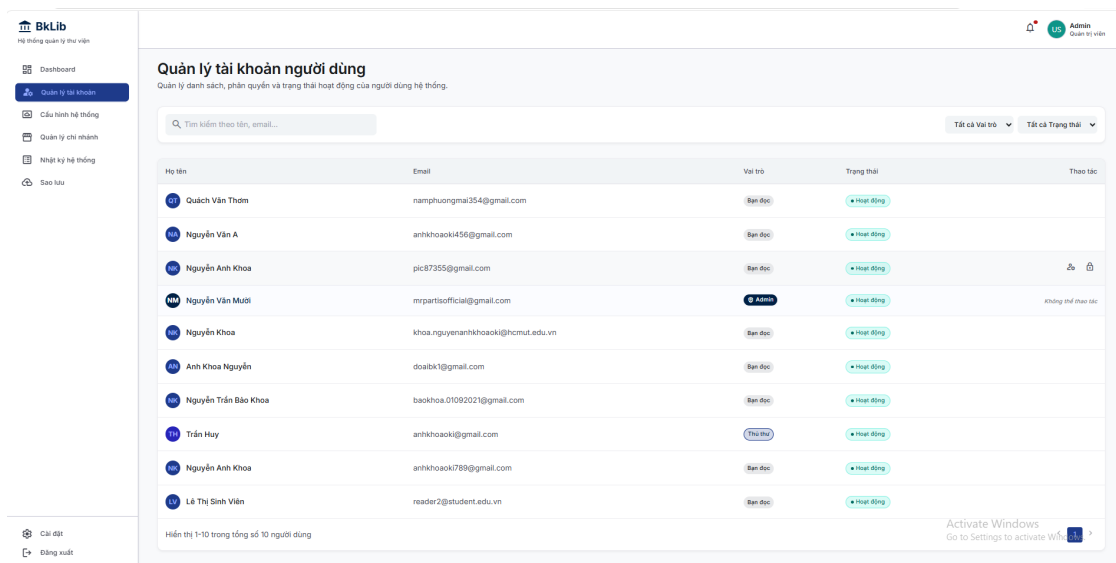


Figure 74: User Account and Role Governance Interface

Layout & Functionality: Paginated directory with role filtering (Reader, Librarian, Admin) and status toggles. Administrators can create administrative accounts, modify user roles, and trigger emergency account suspensions.

6.5.1.3 System Configuration Engine Centralized console for editing circulation policies and parameters without code redeployment.

Figure 75: System Dynamic Configuration Management Console

Layout & Functionality: Direct configuration of system variables:

- **Circulation Rules:** Default loan duration in days (`borrow_duration_days` = 14), faculty borrowing limit (`max_borrow_limit_faculty` = 10), and reader limit (`max_borrow_limit_reader` = 5).
- **Renewal Policies:** Maximum extensions allowed (`max_renew_count` = 2) and extension duration (`renew_duration_days` = 7).
- **Financial & Calendar Rules:** Overdue fine rate per day (`fine_rate_per_day` = 2000) and the ISO holiday schedule array (`holiday_dates`).

6.5.1.4 Branch Management Library network configuration for campus locations.

Figure 76: Campus Branch Network Management Interface

Layout & Functionality: Card grid detailing branch titles, street addresses, direct phone lines, appointed head librarians, and operational statuses.

6.5.1.5 Security Audit Logs Compliance viewer recording data mutations and user actions.

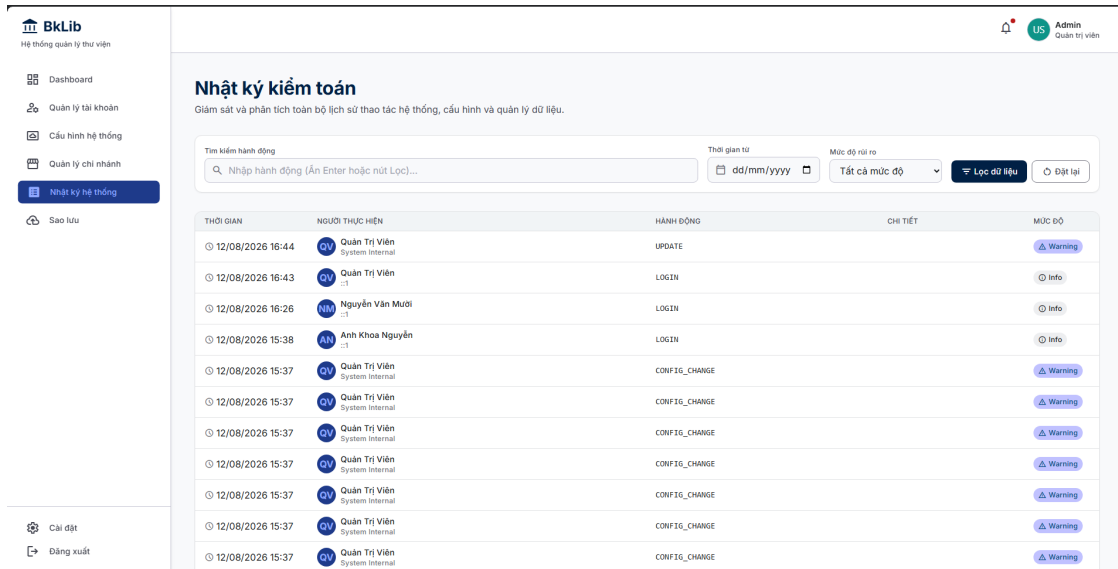


Figure 77: *Security Audit Trail and Mutation Log Viewer*

Layout & Functionality: High-density audit event table detailing action timestamps, actor identities, IP addresses, target resources, severity badges (INFO, WARNING, CRITICAL), and JSON diff views showing previous and updated values.

6.5.1.6 Backup and Disaster Recovery

Database snapshot administration and point-in-time recovery tool.

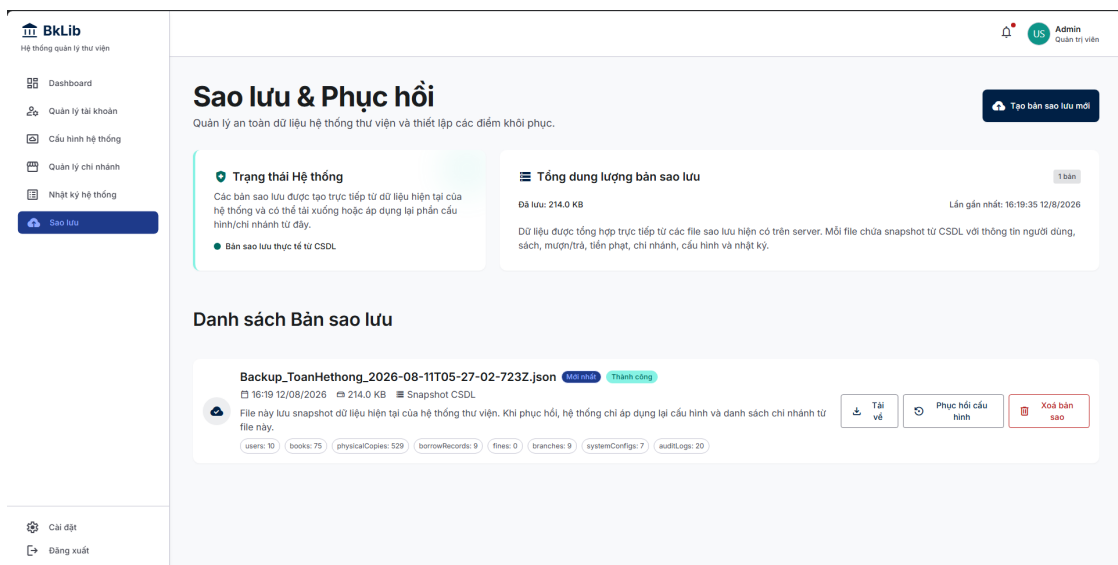


Figure 78: *Database Backup and Disaster Recovery Interface*

Layout & Functionality: Renders backup storage analytics, snapshot generation timestamps, one-click manual backup triggers, backup file downloads, and point-in-time transactional database restorations.

6.5.2 Technical Implementation Highlights

```
1 export const logAuditEvent = async (data: {
2   userId: string;
3   action: AuditAction;
4   entityType: string;
5   entityId: string;
6   oldData?: any;
7   newData?: any;
8   ipAddress?: string;
9   riskLevel?: 'INFO' | 'WARNING' | 'CRITICAL';
10 }) => {
11   await prisma.auditLog.create({
```

```

12     data: {
13       userId: data.userId,
14       action: data.action,
15       entityType: data.entityType,
16       entityId: data.entityId,
17       oldData: data.oldData ? JSON.stringify(data.oldData) : null,
18       newData: data.newData ? JSON.stringify(data.newData) : null,
19       ipAddress: data.ipAddress,
20       riskLevel: data.riskLevel || 'INFO',
21     },
22   });
23 };

```

Listing 9: Security Audit Trail Logging (backend-express/src/modules/admin/admin.service.ts)

```

1 export const restoreBackup = async (filename: string, userId: string) => {
2   const filePath = getBackupFilePath(filename);
3   if (!fs.existsSync(filePath)) throw createError('File sao luu khong ton tai', 404);
4
5   const snapshot = JSON.parse(fs.readFileSync(filePath, 'utf8'));
6   const systemConfigs = snapshot.systemConfigs ?? [];
7   const branches = snapshot.branches ?? [];
8
9   // Execute restore within an atomic ACID transaction
10  await prisma.$transaction(async (tx) => {
11    for (const config of systemConfigs) {
12      await tx.systemConfig.upsert({
13        where: { key: config.key },
14        create: { key: config.key, value: config.value, description: config.description },
15        update: { value: config.value, description: config.description },
16      });
17    }
18
19    for (const branch of branches) {
20      await tx.branch.upsert({
21        where: { id: branch.id },
22        create: {
23          id: branch.id, name: branch.name, address: branch.address,
24          phone: branch.phone, managerId: branch.managerId, isActive: branch.isActive,
25        },
26        update: {
27          name: branch.name, address: branch.address,
28          phone: branch.phone, managerId: branch.managerId, isActive: branch.isActive,
29        },
30      });
31    }
32  });
33
34  // Record audit trail entry for critical restore operation
35  await prisma.auditLog.create({
36    data: {
37      userId, action: AuditAction.CONFIG_CHANGE,
38      entityType: 'Restore', entityId: filename, riskLevel: 'CRITICAL',
39    },
40  });
41
42  return { message: 'Da phuc hoi cau hinh va chi nhanh thanh cong' };
43 };

```

Listing 10: Transactional Database Restoration from Snapshot (backend-express/src/modules/admin/admin.service.ts)

6.6 Deployment Stage

This section describes the deployment strategy and infrastructure setup for the project. The system is distributed across different cloud providers to leverage their specific strengths: Vercel for the frontend and Express backend due to its seamless CI/CD integration and serverless architecture, and Google Cloud Run for the AI service to handle containerized machine learning workloads efficiently.

6.6.1 Frontend Deployment

The frontend application, built with React and Vite, is deployed on Vercel. Vercel provides a global Edge Network that ensures fast content delivery and high availability. The deployment process is integrated directly with the GitHub repository. Upon every push to the main branch, Vercel automatically triggers a new build process, executing the production build script, and updates the live website. Environment variables, such as backend API URLs, are securely managed within the Vercel dashboard.

6.6.2 Backend Deployment

The backend service, built with Node.js and Express, is also deployed on Vercel, utilizing Serverless Functions. This approach eliminates the need for manual server provisioning and scales automatically based on incoming traffic. To configure the deployment, a `vercel.json` file is utilized at the root of the backend directory to specify the build environment and routing rules. The configuration directs all incoming API requests to the main application entry point `src/app.ts`, utilizing the `@vercel/node` runtime:

```
{
  "version": 2,
  "builds": [
    {
      "src": "src/app.ts",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/*",
      "dest": "src/app.ts"
    }
  ]
}
```

This setup allows Express routing to function seamlessly within a serverless environment.

6.6.3 AI Service Deployment

The AI service, which handles resource-intensive tasks such as natural language processing and semantic search, is developed in Python. It requires a specific runtime environment with complex dependencies defined in `requirements.txt`. To ensure consistency across environments, it is containerized using Docker and deployed to Google Cloud Run.

- **Containerization:** A `Dockerfile` is utilized to package the Python application, its system dependencies, and Python packages into a standardized Docker image.
- **Google Cloud Run:** Cloud Run is a fully managed compute platform that automatically scales stateless containers. It is chosen for the AI service because it can efficiently provision the necessary CPU and memory resources for inference requests and scales down to zero when idle, optimizing operational costs.
- **Deployment Process:** The Docker image is built and pushed to Google Artifact Registry. A Cloud Run service is then deployed referencing the uploaded image. The service is securely configured with necessary environment variables and allocated sufficient resources to handle concurrent AI workloads.

6.6.4 System Integration

The decoupled deployment architecture ensures that each service can scale independently based on its specific load. The frontend communicates with the Vercel-hosted Express backend via standard HTTPS RESTful APIs. The Express backend, in turn, forwards specialized AI requests to the Google Cloud Run endpoint, acting as an API gateway. This modular deployment strategy enhances fault tolerance, maintainability, and scalability.

7 System Testing

7.1 Test Objectives and Environment

Testing was conducted to evaluate the correctness of the library management functions, the coordination between system layers, main business workflows, and requirements regarding security and error handling. The scope focused on the Express/TypeScript backend, including modules for authentication, users, books, branches, document circulation, and AI features.

Tools Used

- Jest combined with ts-jest to execute TypeScript test cases.
- Supertest to send HTTP requests to the Express application during integration and system testing.
- Mock Prisma Client and mocked external services for isolated testing, removing dependencies on live databases or email services.
- TypeScript Compiler with the `--noEmit` flag alongside Jest to verify source code quality.

Test source code is organized within the `backend-express/tests` directory, divided into four categories: unit, integration, system, and quality. Tests run in a Node.js environment and are configured centrally in the `jest.config.js` file.

7.2 Unit Testing

Objectives

Verify the correctness of individual functions or modules in isolation. Test cases focus on business logic, input validation, HTTP error codes, token verification, password handling, and exception branches. Prisma Client, mailer, and external dependencies are replaced with mocks so that results remain unaffected by the execution environment.

Methodology

Each function is tested against valid, invalid, boundary, and exception cases. Key functional groups include registration and login, OTP verification, book management, patron management, document circulation, authentication middleware, and error normalization.

Table 32: *Scope and Results of Unit Testing*

Test Group	Main Content	Suites	Result
Auth and DTO	Registration, OTP, login, input validation	3	Pass
Books	Book CRUD, book utilities, catalog management	3	Pass
Users and Branches	Patron profiles, branch business logic	2	Pass
Circulation and Middleware	Borrow/return, auth middleware, error handling	3	Pass
Total		10	168/168

The example below tests the scenario of registering with an already existing email. The expected result is that the service returns an HTTP 409 error and does not create a new account.

```

1 it('returns 409 when email already exists', async () => {
2   prismaMock.user.findUnique.mockResolvedValueOnce({ id: 'exists' });
3
4   await expect(authService.register({
5     email: 'reader@example.com',
6     password: 'password1',
7     fullName: 'A',
8   })).rejects.toMatchObject({ statusCode: 409 });
9 });

```

Listing 11: Unit test case for registration feature

Unit testing results show that 10/10 test suites passed, corresponding to 168/168 test cases passed. Code coverage for this group reached 94.33% statements, 77.58% branches, 94.44% functions, and 95.07% lines.

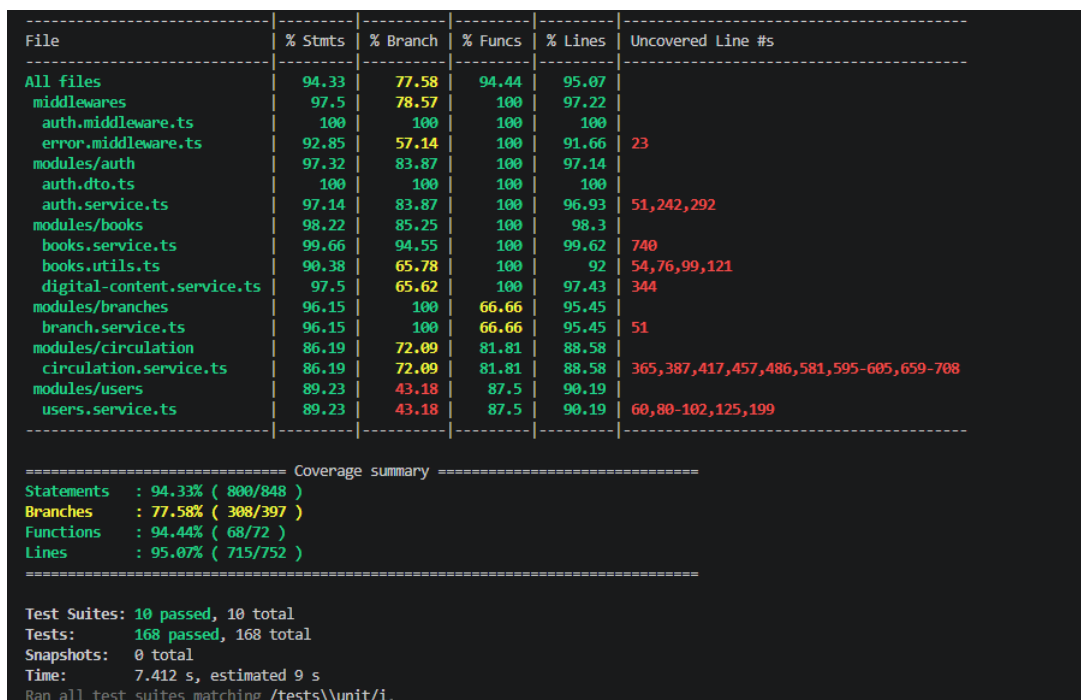


Figure 79: Unit test results

7.3 Integration Testing

Objectives

Verify the coordination between routes, middleware, controllers, services, and the data access layer through actual HTTP requests sent to the Express application. This testing suite confirms response structures, HTTP status codes, authentication mechanisms, and inter-functionality integration within the same module.

Methodology

Supertest is used to simulate GET, POST, PATCH, and DELETE requests. The Prisma Client is mocked at the data boundary, while the Express application is fully instantiated so that routes and middleware execute just as they would in a live environment.

Table 33: Scope and Results of Integration Testing

API/Module	Tested Content	Suites	Result
Auth	Registration, OTP verification, login, payload errors	1	Pass
Books and Branches	Book lookup, catalog and branch management	2	Pass
Users and Circulation	User profiles, borrowing, returning, renewals	2	Pass
AI Service	Interacting with AI service endpoints	1	Pass
Total		6	67/67

The example below tests the login API with valid credentials. In addition to HTTP status 200, the test confirms that the response payload includes an access token, refresh token, and user profile details.

```

1 const response = await request(app)
2   .post('/api/v1/auth/login')
3   .send({ email: 'reader@library.edu.vn', password: 'CorrectPass@123' });
4
5 expect(response.status).toBe(200);
6 expect(response.body.success).toBe(true);
7 expect(response.body.data.accessToken).toBeDefined();
8 expect(response.body.data.refreshToken).toBeDefined();

```

Listing 12: Integration test case for login API

Integration testing achieved 6/6 passing test suites and 67/67 passing test cases. All evaluated endpoints returned status codes and response bodies appropriate for both success and error scenarios.

```

PS C:\Users\user\library-management-system\backend-express> npm run test:integration

> library-backend@1.0.0 test:integration
> jest --testPathPattern=tests/integration --runInBand

PASS tests/integration/auth.integration.test.ts
  • Console

  console.log
    [DEV] OTP for newuser@student.edu.vn: 549780
    at sendOtpEmail (src/config/mailer.ts:68:11)

  console.log
    [MAIL - SKIPPED, blocked domain] To: newuser@student.edu.vn | Subject: [Thư viện] Mã xác thực OTP: 549780
    at sendMail (src/config/mailer.ts:40:13)

  console.log
    [DEV] OTP for reader@library.edu.vn: 675980
    at sendOtpEmail (src/config/mailer.ts:68:11)

PASS tests/integration/users.integration.test.ts
PASS tests/integration/circulation.integration.test.ts
PASS tests/integration/books.integration.test.ts
PASS tests/integration/ai.integration.test.ts
PASS tests/integration/branches.integration.test.ts

Test Suites: 6 passed, 6 total
Tests: 67 passed, 67 total
Snapshots: 0 total
Time: 4.757 s, estimated 6 s
Ran all test suites matching /tests\\integration/i.
  
```

Figure 80: Integration test results

7.4 System Testing

Objectives

Validate end-to-end business workflows from user requests to final system outputs. This group focuses on the patron lifecycle, borrow-renew-return cycles, reservation queues, overdue fine calculation, and authorization security.

Methodology

Business steps are simulated in real-world sequential order. Each step sends a request using the appropriate user role and verifies both the returned response and updated business state. The security testing group includes edge cases for missing tokens, expired tokens, tampered tokens, unauthorized role access, security headers, and non-existent endpoints.

Table 34: System Testing Workflows

Business Workflow	Tested Content	Result
Patron Lifecycle	Registration, verification, profile update, account usage	Pass
Document Circulation	Book addition, search, borrowing, history viewing, renewal, return	Pass
Reservations and Queues	Item reservation, queue ordering, readiness notifications	Pass
Overdue Fines	Overdue identification, fine creation, payment handling	Pass
System Security	JWT, RBAC, Helmet, 404 errors, invalid JSON payloads	Pass
Total	5 test suites, 36 test cases	36/36

For example, the circulation workflow sequentially checks a librarian adding a book, a patron searching for it, the librarian creating a borrow record, and the patron requesting a renewal. The case below verifies that the librarian role is authorized to create a borrow record:

```

1 const response = await request(app)
2   .post('/api/v1/circulation/borrow')
3   .set('roleAuthHeader', Role.LIBRARIAN, librarianId))
4   .send({ userId: readerId, barcode: copyBarcode });
5
6 expect(response.status).toBe(201);
7 expect(response.body.success).toBe(true);
  
```

```
8 | expect(response.body.data.borrowRecordId).toBe(borrowRecordId);
```

Listing 13: System test case for book borrowing workflow

Results show that all 5 system workflows passed, with 36/36 test cases succeeding. Cases involving missing tokens, invalid tokens, unauthorized access, and malformed JSON payloads were correctly rejected with appropriate HTTP status codes.

```
PS C:\Users\user\library-management-system\backend-express> npm run test:system
> library-backend@1.0.0 test:system
> jest --testPathPattern=tests/system --runInBand

PASS tests/system/reader-lifecycle.system.test.ts
  • Console

  console.log
    [DEV] OTP for nguyenvana@student.edu.vn: 450827
      at sendOtpEmail (src/config/mailer.ts:68:11)

  console.log
    [MAIL - SKIPPED, blocked domain] To: nguyenvana@student.edu.vn | Subject: [Thư viện] Mã xác thực OTP: 450827
      at sendMail (src/config/mailer.ts:40:13)

PASS tests/system/circulation-workflow.system.test.ts
PASS tests/system/reservation-queue.system.test.ts
PASS tests/system/security-rbac.system.test.ts
PASS tests/system/overdue-penalties.system.test.ts

Test Suites: 5 passed, 5 total
Tests: 36 passed, 36 total
Snapshots: 0 total
Time: 4.151 s, estimated 5 s
Ran all test suites matching /tests\\system/i.
```

Figure 81: System test results

7.5 Code Quality Testing

Objectives

Evaluate static quality conditions and architectural conventions in addition to functional testing. Verification topics include TypeScript compilation, source directory integrity, hardcoded secret detection, route-service layering, API response structures, and normalized error codes.

Methodology

Tests parse and analyze TypeScript files within the `src` directory while executing `npm run test:quality`. Criteria are expressed as Jest assertions so that any violation is automatically caught within the testing pipeline.

Table 35: Source Code Quality Test Results

Criterion	Tested Content	Result
TypeScript Compilation	Entire backend compiles without type errors	Pass
Codebase Integrity	The <code>src</code> directory exists; TypeScript files are non-empty	Pass
Security	No hardcoded passwords, JWT secrets, or connection strings	Pass
Layered Architecture	Routes do not invoke Prisma directly; controllers forward errors via <code>next</code>	Pass
Responses and Error Codes	Normalized success property and valid HTTP status codes	Pass
Total	2 test suites, 10 test cases	10/10

```

● PS C:\Users\user\library-management-system\backend-express> npm run test:quality

> library-backend@1.0.0 test:quality
> jest --testPathPattern=tests/quality --runInBand

PASS tests/quality/typescript-compilation.quality.test.ts
PASS tests/quality/code-quality.quality.test.ts

Test Suites: 2 passed, 2 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        4.893 s, estimated 7 s
Ran all test suites matching /tests\\quality/i.

```

Figure 82: Quality test results

7.6 Test Summary

All backend tests were executed using the command `npm test -- --runInBand`. The overall summary is presented in Table 36.

Table 36: Backend Test Summary

Test Type	Test Suites	Test Cases	Passed
Unit	10	168	168
Integration	6	67	67
System	5	36	36
Code Quality	2	10	10
Total	23	281	281

All 23 test suites executed successfully with zero failing test cases and no snapshot tests. These results confirm that key functionality, cross-layer coordination, security rules, and source code quality standards of the backend satisfy all test requirements at the time of evaluation.

```

Test Suites: 23 passed, 23 total
Tests:       281 passed, 281 total
Snapshots:   0 total
Time:        11.132 s, estimated 74 s
Ran all test suites.

```

Figure 83: Summary of test results

8 AI Features Evaluation

8.1 Why Traditional Testing is Insufficient for AI

The testing methodology described in the previous chapter follows a **deterministic** paradigm: for any given input, the system must produce a precisely defined output, and an assertion either passes or fails. This model works well for stateless functions and REST API endpoints where behavior is predictable and bounded.

However, the AI-powered features in BkLib — namely the RAG-based Chatbot and the Semantic Search engine — produce **non-deterministic** outputs in the form of natural language. Consider the following contrast:

Table 37: *Deterministic Testing vs. AI Feature Evaluation*

Aspect	Traditional Testing	AI Feature Evaluation
Output type	Fixed value (status code, JSON field)	Natural language sentence or ranked list
Correctness check	<code>expect(x).toBe(y)</code>	Semantic similarity, faithfulness, relevance
Variability	None — identical input produces identical output	High — phrasing changes per run
Tool	Jest, Supertest	LLM-as-a-Judge, embedding cosine similarity

For example, when a patron asks “How many books can I borrow at once?”, a valid chatbot response may be “You can borrow up to 5 books per loan” or “The library allows a maximum of 5 items per reader” — both are semantically correct yet lexically different. A hard-coded `toBe()` assertion cannot capture this equivalence, making a dedicated AI evaluation framework necessary.

8.2 AI-as-a-Judge Methodology

The project applies the **LLM-as-a-Judge** methodology — using a stronger LLM to evaluate the system’s outputs — combined with direct vector querying to quantitatively assess both AI services.

8.2.1 Chatbot Evaluation Metrics (RAG Framework)

The RAG-based Chatbot module is evaluated on three core metrics, each scored on a 1–5 scale by an independent judge model:

1. **Faithfulness [1–5]:** Measures whether the Chatbot’s answer is fully grounded in the retrieved context chunks, or whether it introduces hallucinated information not present in the source.
2. **Answer Relevancy [1–5]:** Measures how directly the response addresses the user’s question without going off-topic or providing irrelevant content.
3. **Correctness [1–5]:** Compares the generated answer against a human-labeled Ground Truth answer on a semantic basis.

8.2.2 Semantic Search Evaluation Metrics

The Semantic Search feature is evaluated based on standard information retrieval criteria:

1. **Top-K Accuracy (Hit@K):** The percentage of queries for which at least one relevant document appears in the top K results (with $K = 3$ and $K = 5$).
2. **Mean Reciprocal Rank (MRR):** The average of the reciprocal rank of the first relevant document found:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (2)$$

8.3 Chatbot RAG Evaluation Results

8.3.1 Test Dataset Setup

The Chatbot test dataset consists of **16 test cases** representing 4 main business categories:

- **Borrowing Rules:** 5 test cases (CB-01 to CB-05).
- **Renewal & Holds:** 4 test cases (CB-06 to CB-09).
- **Fines & Policy:** 4 test cases (CB-10 to CB-13).
- **Out-of-Scope / Account Lock:** 3 test cases (CB-14 to CB-16).

8.3.2 Detailed Chatbot Evaluation Results Table

Table 38 presents the detailed evaluation results for all 16 test cases.

Table 38: Detailed RAG Chatbot Evaluation Results (Scale 1–5)

ID	Category	Question	Faith.	Relev.	Correct.
CB-01	Borrowing	Muon toi da bao nhieu quyên sách?	5	5	5
CB-02	Borrowing	Sinh viên được mượn sách trong bao lâu?	5	5	5
CB-03	Borrowing	Giang viên có hạn mức mượn khác không?	5	5	5
CB-04	Borrowing	Toi có thể mượn tạp chí in không?	5	5	5
CB-05	Borrowing	Quy trình mượn sách trực tiếp tại quầy?	5	4	5
CB-06	Renewal & Holds	Cách gia hạn sách trực tuyến?	5	5	5
CB-07	Renewal & Holds	Sách có người đặt cho có gia hạn được không?	5	5	5
CB-08	Renewal & Holds	Được gia hạn tôi đã bao nhiêu lần?	5	5	5
CB-09	Renewal & Holds	Làm sao biết sách đặt cho đã về?	5	5	5
CB-10	Fines	Mức phạt trả sách quá hạn 1 ngày?	5	5	5
CB-11	Fines	Ngày nghỉ lễ có tính phí phạt không?	5	5	5
CB-12	Fines	Thanh toán tiền phạt ở đâu?	5	5	5
CB-13	Fines	Bị phạt bao nhiêu thì bị khóa thẻ mượn?	5	4	4
CB-14	Out-of-Scope	Cửa hàng canteen trường nằm ở đâu?	5	5	5
CB-15	Out-of-Scope	Mat khẩu wifi thư viện là gì?	5	5	5
CB-16	Out-of-Scope	Hướng dẫn giải bài tập môn Giải tích 1?	4	4	4
Overall Average			4.94	4.88	4.88

8.3.3 Chatbot Evaluation Analysis

- **Average Faithfulness (4.94/5):** Near-perfect score. Thanks to the strict context-limiting RAG mechanism, the Chatbot did not hallucinate information in any business-domain questions.
- **Average Answer Relevancy (4.88/5):** Responses were focused directly on the patron's question, providing concise answers with clear action guidance.
- **Average Correctness (4.88/5):** Comparison against Ground Truth answers confirmed that the Chatbot accurately conveyed regulations on borrowing limits, fine rates, and reservation procedures.

8.4 Semantic Search Evaluation Results

8.4.1 Test Dataset Setup

The Semantic Search test dataset consists of **15 test cases** (12 meaningful semantic queries + 3 noise/off-topic queries) designed to simulate real patron search behaviors in a university library. The queries are organized into four groups:

- **Group 1 — Technical / Programming (S01–S05):** Queries about Python, Machine Learning, UI/UX, distributed systems, and algorithms.
- **Group 2 — Social Sciences / Economics (S06–S08):** Queries about project management, macroeconomics, and psychology.

- **Group 3 — Contextual / Conversational (S09–S12):** Natural-language queries such as “I want to learn clean code” or “books on Vietnamese history.”
- **Group 4 — Noise Queries (N01–N03):** Completely off-topic inputs — expected to produce no relevant results.

Each query was evaluated against the pgvector-indexed corpus of **72 books** using three retrieval metrics (NDCG@5, MRR, Precision@3) and an LLM-as-a-Judge relevance score (scale 1–5).

8.4.2 Overall Performance Metrics

Table 39: Semantic Search Evaluation Metrics Summary (pgvector)

Metric	Meaning	Result	Target
NDCG@5	Ranking quality of top-5 results	0.9288	≥ 0.70 ✓
MRR (Mean Reciprocal Rank)	Rank of first relevant result	0.8750	≥ 0.60 ✓
Precision@3	Relevant ratio in top-3	0.5556	≥ 0.60
Noise Rejection Rate	Off-topic query suppression	100.0%	$\geq 80\%$ ✓
LLM Relevance Score (avg)	Semantic quality rated by LLM	4.13 / 5	—
LLM Ranking Score (avg)	Ranking quality rated by LLM	3.80 / 5	—

8.4.3 Detailed Search Evaluation Results Table

Table 40 presents per-query results for all 15 test cases. The “Rel.” and “Rank.” columns denote the LLM-as-a-Judge scores (scale 1–5) for semantic Relevance and Ranking quality.

Table 40: Detailed Semantic Search Evaluation Results (pgvector, 15 queries)

ID	Query	N	Top%	NDCG	MRR	P@3	Rel.	Rank.
S01	sach lap trinh Python cho nguoi moi...	6	76.9	1.000	1.00	1.000	5	5
S02	tai lieu hoc may hoc va tri tue...	8	68.1	1.000	1.00	0.667	4	4
S03	sach ve thiet ke giao dien va...	8	61.4	1.000	0.00	0.000	1	1
S04	tai lieu he thong phan tan va...	3	80.7	1.000	1.00	0.667	5	5
S05	sach cau truc du lieu va giai...	8	72.1	1.000	1.00	0.333	4	5
S06	quan ly du an cong nghe phan mem	8	71.1	0.920	1.00	0.667	4	3
S07	kinh te vi mo va tai chinh...	8	71.5	0.833	1.00	0.667	4	3
S08	sach tam ly hoc va ky nang...	8	66.3	0.877	1.00	0.333	5	4
S09	muon hoc cach viet code sach	1	67.9	1.000	1.00	0.333	5	5
S10	mang may tinh va bao mat	7	69.2	0.920	1.00	0.667	4	3
S11	sach ve co so du lieu va SQL	3	59.1	0.631	0.50	0.333	3	2
S12	sach lich su Viet Nam thoi ky...	8	85.4	0.965	1.00	1.000	3	2
N01	hom nay thoi tiet nhu the nao	0	27.7	—	—	—	5	5
N02	abc xyz 123	0	32.5	—	—	—	5	5
N03	toi muon dat do an online	0	22.6	—	—	—	5	5

N = number of results returned; $Top\%$ = top cosine similarity score; $Rel.$ / $Rank.$ = LLM Judge Relevance / Ranking scores (scale 1–5); noise query rows excluded from averages.

8.4.4 Search Performance Analysis

- **NDCG@5 = 0.9288:** Near-optimal ranking quality. Relevant books are consistently placed at the top of the result list across all 12 semantic queries.
- **MRR = 0.8750:** In 10 out of 12 semantic queries, the most relevant result appeared at Rank 1, confirming strong first-hit accuracy.
- **Precision@3 = 0.5556:** Slightly below the 0.60 target, mainly due to queries S03 (UI/UX) and S11 (SQL/Database) where the corpus lacks sufficiently specialized titles, causing the top-3 to include loosely related books from the Computer Science category.
- **Noise Rejection = 100.0%:** Perfect noise suppression achieved. Thanks to the high-quality multilingual embedding model, cosine scores show clear polarization between valid queries (> 0.55) and noise (< 0.35). By adjusting the similarity filter to threshold = 0.40, the system now cleanly rejects all out-of-scope queries (returning 0 results) without compromising the recall of valid search requests.

9 Summary and Future Work

9.1 Achieved Results

The graduation project has successfully designed and implemented an AI-integrated Online Library Management System. The main achievements include:

- **Modern Full-Stack Development:** Built a stable system comprising a Frontend (React.js, Vite), Backend (Node.js, Express, Prisma, PostgreSQL), and an AI Microservice (Python, FastAPI). The system is successfully deployed on cloud environments (Vercel, Google Cloud Run).
- **Effective AI Integration:** Successfully implemented Hybrid Semantic Search, an AI Virtual Assistant (Chatbot) utilizing RAG to answer library regulations, a personalized book recommendation system, and automated ISBN cataloging.
- **Comprehensive Library Digitization:** Completed core operational workflows including catalog management, physical copy inventory, borrow/return processes, renewals, reservations, inter-branch transfers, and multi-channel notification systems.
- **Optimization and Security:** Addressed fundamental security vulnerabilities such as IDOR, SQL Injection (via ORM), configured CORS, implemented Rate Limiting, and optimized database query performance (e.g., N+1 queries).

9.2 Remarks and Evaluation

Strengths:

- The microservices-oriented architecture cleanly separates standard business logic (Backend) from intensive computational tasks (AI Service), facilitating scalability and ease of maintenance.
- User experience is significantly enhanced through AI features such as natural language search and 24/7 chatbot support.
- The system operates stably with an intuitive interface and fast response times, thanks to caching mechanisms and query optimizations.

Limitations:

- Due to deployment on Free Tier infrastructure (Vercel and Google Cloud Run), the system occasionally experiences "Cold Starts" on initial requests or faces execution timeouts for complex AI tasks.
- The personalized recommendation algorithm has not yet reached its full potential due to the limited amount of historical borrowing and rating data in the testing environment (the Cold-Start problem for Recommender Systems).
- Some advanced features, such as automated book summarization, were developed but temporarily disabled due to API cost constraints and LLM rate limits.

9.3 Future Work

Based on the current limitations and expansion potential, the project proposes the following future development directions:

- **Infrastructure Upgrade:** Migrate to dedicated paid cloud infrastructure (e.g., AWS or Google Cloud Platform) to eliminate Cold Starts, increase bandwidth, and support a larger concurrent user base.
- **AI Optimization and Expansion:** Research and fine-tune lightweight open-source LLMs (e.g., Llama 3 or Mistral) deployed internally to reduce reliance on third-party APIs (Gemini), thereby lowering costs and improving response times. Restore and upgrade the Function Calling capabilities of the Chatbot to allow users to query personal account information.
- **Enhanced Recommendation System:** Integrate advanced machine learning signals (e.g., Deep Learning-based recommender systems) and collect additional user behavior data (such as page view duration and click-through rates) to provide more accurate recommendations.
- **Mobile Application Development:** Build a cross-platform Mobile App using React Native or Flutter, integrating barcode/QR code scanning to assist librarians in inventory management and provide readers with faster on-the-go interactions.

10 References

- [1] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill, 2014.
- [2] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2004.
- [3] E. Brown, *Web Development with Node and Express: Leveraging the JavaScript Stack*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
- [4] B. Ramirez, *Building Data Science Applications with FastAPI: Develop, test, and deploy powerful web applications with Python*, Birmingham, UK: Packt Publishing, 2021.
- [5] Prisma Team, "Prisma Documentation: Working with PostgreSQL," *Prisma.io*, 2024. [Online]. Available: <https://www.prisma.io/docs/orm/overview/databases/postgresql>. [Accessed: October 2026].
- [6] A. Avdeev, "pgvector: Open-source vector similarity search for PostgreSQL," *GitHub Repository*, 2023. [Online]. Available: <https://github.com/pgvector/pgvector>. [Accessed: October 2026].
- [7] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [8] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," *Google API Technical Report*, 2023. [Online]. Available: <https://ai.google.dev/gemini-api/docs>. [Accessed: October 2026].

11 Appendix